SCIENTIFIC COMPUTING

Peter Staab

Fitchburg State University

July 8, 2020

Contents

Preface **i**

1 Introduction to Scientific Computing **1**
1.1 Ideas needed to do Effective Scientific Computing 1
1.2 Examples of Scientific Computing 2
1.3 Writing Code for Scientific Computing 2

I Scientific Computing Basics **5**

2 Storing number and strings and using Julia syntax **9**
2.1 Numbers 9
2.2 Assignment Statement and Variables 9
2.3 Strings 10
2.4 Expressions 12
2.5 Operator Precendence 12
2.6 Comments 13

3 Introduction to Data Types **15**
3.1 Integers 15
3.2 Floating Point Numbers 16
3.3 Extending integers, the `BigInt` type 18
3.4 Extending Floating Point Numbers with `BigFloat` 18
3.5 Rational numbers 19
3.6 Complex Numbers 20
3.7 Abstract and Concrete Number Types 21
3.8 Converting numbers 22
3.9 Tuples 22

4 Introduction to Functions **25**
4.1 Simple example 25
4.2 Function Arguments 26
4.3 Returning values from a function 26
4.4 Specifying Argument Types 27

4.5 Multiple Dispatch . . . 27
4.6 Variable Number of arguments . . . 29
4.7 Multiple Return Values . . . 29
4.8 Factorial Function . . . 30
4.9 Recursive Functions . . . 30
4.10 Variable Scope . . . 31

5 Boolean Statements, Loops and Branching 33
5.1 Boolean values and if statements . . . 33
5.2 If Statements . . . 34
5.3 Ternary `if-else` statement . . . 35
5.4 Loops . . . 35
5.5 Ranges . . . 38

6 Arrays 39
6.1 Constructing arrays . . . 39
6.2 Comprehensions . . . 40
6.3 Accessing elements of an array . . . 41
6.4 Common Operations on Arrays . . . 42
6.5 Element by Element Operations . . . 42
6.6 Other Operations on Arrays . . . 43
6.7 Sorting Vectors and matrices . . . 43
6.8 Push and Pop; Array as a Stack . . . 43
6.9 Other Nice Array Functions . . . 44

7 Functional Programming and an Introduction to Writing fast code 47
7.1 Functional vs. Non-functional forms . . . 47
7.2 Reducing an array . . . 48
7.3 The `mapreduce` function . . . 49
7.4 Mapping a Function over an 2D array . . . 50
7.5 Writing fast code . . . 51
7.6 Summary of Results . . . 53
7.7 Computing Fibonacci Numbers . . . 54

8 Number theory and Algorithm Development 57
8.1 Prime Numbers . . . 57
8.2 Finding Primes . . . 58
8.3 Perfect Numbers . . . 59
8.4 Happy Numbers . . . 60
8.5 Primes can be big!! . . . 63
8.6 Speeding up `isPrime` . . . 63
8.7 Summary of `isPrime` functions . . . 66
8.8 Testing for Primes using the `Primes` library . . . 67

9 Solving Quadratics and Rootfinding in General 69
9.1 Absolute and Relative Errors . . . 69
9.2 Rounding Errors and the Quadratic Formula . . . 70
9.3 Are We Sunk? Examining the Quadratic Formula . . . 71
9.4 Newton's Method . . . 73

10 Plotting Data and Functions 77
10.1 The Plots Package . . . 77
10.2 Backends of the Plots package . . . 80
10.3 Other plots . . . 84
10.4 Other Plots and subplots . . . 87
10.5 Exercises . . . 87

11 Creating New Data Types in Julia 89
11.1 Basics of Julia's Composite Datatype . . . 89
11.2 A Card datatype . . . 90
11.3 Polynomials–A parametric data type . . . 92
11.4 Plotting the Polynomial . . . 94
11.5 A Rootfinding datatype . . . 97

II Simulation 99

12 Probability Distributions and Pseudorandom Numbers 103
12.1 Probability . . . 103
12.2 Pseudo Random Number Generator . . . 104
12.3 Calculating π using pseudo random numbers . . . 106

13 Using Random Numbers and Probability Models 107
13.1 Rolling Dice . . . 107
13.2 Flipping Coins . . . 108
13.3 Rolling 2 dice . . . 109
13.4 Other Probability Models . . . 111

14 Probability of Poker Hands 113
14.1 A user-defined package . . . 114

III Other Julia Features 115

15 Advanced Function Features of Julia 117
15.1 Testing Arguments . . . 117
15.2 Optional arguments . . . 118
15.3 Keyword Arguments . . . 119

15.4 Variable Number of arguments . . . 121
15.5 Multiple Dispatch . . . 121
15.6 Parametric Methods . . . 123

16 Creating Modules and using Unit Tests 125
16.1 Creating a Module . . . 125
16.2 Debugging a module . . . 126
16.3 Unit Tests . . . 126
16.4 Creating a test suite . . . 128
16.5 Modules and Unit Tests . . . 128

17 Parallel Computing 129
17.1 Running parallel code . . . 129
17.2 Writing a parallel card simulator . . . 131
17.3 A Parallel Map Function . . . 132
17.4 Shared arrays . . . 132

IV Data Analysis 135

18 Introduction to Data Analysis 137
18.1 Data from the `RDatasets` package . . . 137
18.2 Dataframes . . . 138
18.3 Manipulating DataFrames using the `Query` package . . . 140
18.4 Analyzing Census Data . . . 141
18.5 Analyzing information about Olympic Athletes . . . 144
18.6 Missing Data . . . 149

V Linear Algebra 151

19 Introduction to Linear Algebra 153
19.1 Vectors and Matrices . . . 153
19.2 Matrix Inverse . . . 154
19.3 Eigenvalues and Eigenvectors . . . 154

20 Advanced Topics in Linear Algebra 155
20.1 Sparse Matrices . . . 155

VI Calculus and Differential Equations 157

21 Numerical Integration 159
21.1 Trapezoid Rule . . . 159
21.2 Simpson's Rule . . . 161

21.3 Using the `NumericalIntegration` package 163

22 Differential Equations **165**
22.1 Euler's Method 166
22.2 Runge Kutta methods 167
22.3 Using the `DifferentialEquations` package 168

23 Complex Numbers and Analysis **169**
23.1 Complex Numbers 169
23.2 Fourier Transforms 169

VII Advanced Topics **171**

A Details of Using Julia **173**
A.1 using the Repl.it wesite 173
A.2 Downloading and Installing JuliaPro 173
A.3 Using Juno and the Atom Editor 174
A.4 The kernel 178
A.5 Restarting the Kernel 179

B Loading Packages **181**
B.1 Managing Packages in Julia 182

C Binary Representation of Numbers **187**
C.1 Integer Representation 187
C.2 Operations on Binary Integers 190
C.3 Subtraction of binary integers 191
C.4 Multiplication of Binary Integers 191
C.5 Extended Exercise 191
C.6 Scientific Notation 192
C.7 Floating Point Numbers of a given size 192

Preface

Solving problems is the essence of scientific computing. Many problems arise naturally from the sciences or mathematics and often to solve difficult problem relies on some computing resources. In a nutshell, this text is the blend of science, mathematics and computer science needed to solve problems.

The intended audience for this text one who is interested in writing computer code that will solve a problem that you have. There are a few basics that you should have with this text:

- Some background in writing computer programs. Your base language does not matter, but you should be familiar in writing short programs.
- Some calculus. Most of the examples here do not need Calculus, but there are topics throughout the book, that at least differential calculus is helpful.

As many textbooks, this started as a set of notes for my Scientific Computing class at Fitchburg State University. The students in the class are usually either Computer Science or Mathematics major and generally have an interest in computing. As I steered these pages from a set of notes to a textbook, I still keep those students in mind. However, I realize that an undergraduate text is often picked up by graduate students, those in industry and other faculty to build some knowledge base that one never had. I keep these readers in mind as well.

I also want to distinguish this text from that of a Numerical Analysis or Numerical Methods text. A Numerical Methods text often covers many basic algorithms in mathematics such as numerical differentiation, numerical integration, rootfinding, linear algebra, etc. and how to implement such algorithms; this can be done with pseudocode or with a particular language. Numerical Analysis often covers the same algorithms in Numerical Methods with an emphasis on the analysis of the algorithms. This text does not broadly cover all the topics in Numerical Methods, nor does it do any analysis. There are many good books listed in the references. Instead, this text covers how to solve particular problems and often the algorithms have been coded in an existing library and we will leverage those here.

Why use Julia or any particular language?

The subtitle of the text says that solving problems in Scientific Computing does not happen in a vacuum or necessarily in the land of pseudocode, so I have specifically

chosen a language to explain the concepts in the text much like I do in my class and the computing language that will be used will be Julia.

Before going through the technical reasons for choosing Julia, the following personal story is interesting. I proposed teaching Scientific Computing at Fitchburg State and the class would by in the Fall of 2014. The proceding spring, I knew that I had to choose some language for the class. There are many languages which would have been fine to use, including Maple, which our majors to learn in a required class, and that would have been an easy choice. However, one of blogs that I read regularly, ***ars technica***, had an article by Lee Philips entitled "Scientific computing's future: Can any coding language top a 1950s behemoth?"[1] The essence of the article is that in scientific computing, Fortran is king, due to two reasons 1) it is fast mainly because it was designed to be easily translated to machine language, which eventually is what runs on a computer and 2) there are many important mathematical libraries written in Fortran have been time-tested and virtually bug-free.

However, nobody wants to code in Fortran. It is missing many features of modern languages and requires compilation. In selecting Julia, as stated on the main webpage, "Julia was designed from the beginning for high performance." As problems in Scientific Computing increase in size and complexity, it is an imperative that code runs fast and Julia typically does. It is one of the fastest high-level computing languages in benchmarks (REF) and compared to other scientific computing languages, it is the fastest. At the time the article was written, Julia was in version 0.2 and was quite raw, but was fascinating to consider and in hind sight, I'm glad I have done so.

Another important aspect of Julia is that it is a scripting language like python. This means that you have an environment setup, you can just start entering julia commands and go. There is no compilation involved (like that in C or Fortran). Although compilation often finds errors, it is slow to do prototyping.

How to read this text

How might one read this text? Of course, the short answer is anyway you want. I hope you can pick it up and read what is relevant. There's a few items about the layout as well as some standard conventions.

Take a look at the table of contents. The text is broken up into a few parts. Part I consists of some basics of both the julia language as well as scientific computing. The remaining parts of the text are separate sections (Simulation, Data Analysis, Linear Algebra) and some advanced features of julia. Once Part I is read, the remaining parts are independent, however there are some ideas in Part III that are very helpful later in the test. The following diagram shows the chapter dependencies:

In addition, because there are code snippets throughout the text, you should run the code as you read the text. Chapter 1 is the beginning to getting you started to run julia, but it is recommended to read Appendix A for more details. All julia code is written in

[1] https://arstechnica.com/science/2014/05/scientific-computings-future-can-any-coding-language-top-a-1950s-behemoth/

`monospaced font` and is either embedded in the text, like `x=2` or written in a separate block like

```
?? PythonTeX ??
```

Any blocks of code should be able to be copied and pasted directly into the julia REPL or a jupyter notebook.

Also, output from julia is also in a fixed-width font but generally in a block like this:

```
"This is a string of output"
```

Here's some of the coding style that is used throughout the text as well. This helps with readability:

- functions are written in `camelCase`, like `findRealRoot`.
- if a function returns a boolean (`true` or `false`) it should start with `is` or `has`, like `isFullHouse` from Chapter 14
- other variables are written in `snake_case` like `num_trials`

Chapter 1

Introduction to Scientific Computing

Scientific Computing is

- a field of study in which problem solving is done via writing programs (algorithms and code) to solve the problems.
- a field which requires math or science knowledge to be effective.

Scientific Computing is not

- just writing code
- a substitute for math or science
- a magic bullet to solve any problem

Where is Scientific Computing Used

- making predictions (weather, stock market, traffic, etc.)
- handling and processing gobs of data from scientific experiments
- by individuals testing ideas
- in production software

1.1 Ideas needed to do Effective Scientific Computing

Scientific Computing is generally used to solve problems in Mathematics and various Science fields. There are a number of important things that you need to know to solve problems effectively.

- **Find/develop code that runs quickly.** This all depends on the problems that you are solving. If a simple program runs that solves your problem and only takes 5 seconds, it's probably fine. If instead, it takes a few hours to run, you probably want to investigate your solution algorithm.

- **Find/develop code that uses an appropriate amount of memory.** Another important aspect is that of memory consumption. It's not hard to find datasets these days that are 1TB or more in size, however few desktop/laptop machines have more than 16 or 32Gb of RAM, so you can't load the entire file into memory. Such a dataset would need to be processed in chunks.

- **Make sure you have known solutions/unit tests** How do you know that your solution is correct? It is important to have relatively simple cases that you can test your code on before solving more complex things.

1.2 Examples of Scientific Computing

- Consider a list of 5 cities that you need to visit and return to where you started. If you know the distances between each pair of cities, what is the most efficient path (in terms of distance or time) that gets you to each city and return.

- Poker is a card game in which certain sets of cards (hands) beat other hands. Determine the probability (or chance) of getting any particular hand in Poker.

- Given a dataset of Major League Baseball scores from 2000-2017, find the game with the largest score, most innings played, biggest winning margin, etc.

We will look at these an other problems in this course.

1.3 Writing Code for Scientific Computing

Look back at the list of requirements for Scientific Computing projects. A general rule of thumb is to use a computing language that you know will work well for those tasks. Typically, most students have learned one or two languages and aren't quite sure which to use. In most cases it doesn't matter what to use, but for complex problems it will matter.

One of the best languages for scientific computing has been Matlab over the past 3 or 4 decades. It is used extensively in engineering firms throughout the world (and the headquarters is in nearby Natick, MA). We won't be using Matlab here though. One of the nice things about taking a class is exploring new languages. Here we will use Julia, which is a very new language that a lot of people in scientific computing are excited about. We will give examples using Julia, but the ideas here should be applicable to other languages.

This is a general getting started chapter with some specifics that can be run in Julia. To get started with being able to run julia on your own computer, see Appendix A. If you haven't do so, please read that and then come back. Everything else will make more sense. Trust me.

Julia was designed as a scientific computing language, but in short is a modern language. There are number of aspects that makes julia a good language for this. Julia is

- *a scripting language with dynamic types*. This means you can get started right away–there's no need to learn about compilation–and you can prototype things quickly.
- *a language with just-in-time compilation*. Most of the time scripting languages are slow, however with modern under-the-hood tools, languages can be compiled on the fly to create very fast runtimes. There is a webpage on benchmarks comparing julia to other standard languages.
- *open source.* Although often open-source implies free but not high-quality, the free part holds, but more important is that anyone can contributed to the code. The Julia community is committed to creating a high-quality piece of software and many discussion revolve around writing code that will improve the speed or other aspects. This is very opaque and unclear if it happens with commercial software.
- *easy to use*. The syntax of julia is similar to that of python, a very popular language, and is often intuitive.
- *a joy to use*. Additionally, the structure of the language makes it easy to start with, but as projects get more complex, it can be written in a simple way.

Part I: Scientific Computing Basics

This part of the text explores the basics of scientific computing. If you flip through this part, you will notice that it is heavily dependent on writing computer code. At a fundamental level, all scientific computing is writing code–albeit in way to solve mathematical and scientific problems. Also, as mentioned above, we will be using julia throughout this text and all of the examples are in julia and much of the discussion explore julia's structure in order to write code that will run fast or in a way that is helpful for solving scientific computing problems.

Chapter 2

Storing number and strings and using Julia syntax

Solving Scientific Computing problems ultimately boils down to manipulating data and at the most basic is that of strings and numbers. We begin with understanding these data types and how to store values in them.

We also show some of julia syntax, which looks like other languages (like python). Hopefully you have some basic knowledge of computing, but no assumption of any particular language is necessary.

2.1 Numbers

Not surprisingly, the most important data type in scientific computing–at least at the most atomic level is numbers. Numbers in computation mimic those in mathematics with some important differences. Julia like most computing languages have two main number types, integers and floating points. Julia's integers and much like mathematical integers in that they store numbers like $0, 10, -400$. Floating-point numbers generally are approximations to real or decimal numbers and more details will be covered in Chapter 3.

Julia also has a rational data type for numbers commonly though of as fractions like $-\frac{2}{3}$ or $\frac{22}{7}$. Recall that typically i represents $\sqrt{-1}$, the base imaginary number. Complex numbers like $2 + 3i$ are also native to julia. We will explore both rational and complex numbers in greater depth in Chapter 3.

2.2 Assignment Statement and Variables

Anything can be stored as a variable using the single equal sign like **??**. This is an assignment operator, which creates the number 6 and stores it under the name `x`.

And now that the variable `x` is stored, we can use it in calculations. For example

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**.

Variables in julia, much like other languages are primarily sequences of alphanumeric characters as well as an underscore `_`. Primarily, a variable needs to start with a alphabetic character or `_` and after the first character can contain numbers.

Julia also allows many unicode symbols in variable names, however not everything. For example, all of the greek letters are allowed, so `α=45` is valid.

To get a greek letter in Jupyter or the REPL, type `\alpha`, hit the TAB key and it will be turned into an **|**.

2.2.1 Storing Variable in a Virtual Whiteboard

The details of storing variables in computer hardware isn't necessary, however, thinking of storing as writing variables and values on a whiteboard is a helpful paradigm. Imagine a whiteboard with a column of variable names and a column of values. For example, if we have

```
?? PythonTeX ??
```

then you can think of the whiteboard looking like:

x	6
y	−1
z	8.5

If we evaluate the expression **??**, then the value of `x` is looked up and the value 6 is substituted into the expression or `6+3`.

If we change one of the values, like **??**, this means that the right hand side is evaluated to the value 4, then the 4 is placed into the whiteboard, which will now look like:

x	6
y	4
z	8.5

If you are thinking of how a piece of code works, often you will need to get to the point of writing down a version of the whiteboard.

2.3 Strings

In many Scientific Computing fields, such as Data Science, strings arise often and it is important to understand some of the basics of them. In Julia, a string is a sequence of characters surrounded by `""` (double quotes). For example:

```
?? PythonTeX ??
```

and if you enter **??** then you should see **?? PythonTeX ??**. The individual parts of the string are called characters, which have type `Char` and are by default Unicode Characters (which will we see are super helpful). A few other helpful things about strings are

- The length of a string is found using the `length` command. `length(str)` returns **?? PythonTeX ??**.
- To access the first element of the string, type `first(str)`, the last is found by `last(str)` and the 3rd character for example is `str[3]`. In julia, string indexing starts at 1.
- To turn other data types into string, use `string`. For example `string(3.0)` returns the string `"3.0"`.
-

2.3.1 String Operations

We saw how to access elements of a string. Another helpful operation is that of concatentation, or the merging of two strings.

If

```
?? PythonTeX ??
```

we can concatenate in two ways, with the `*` operator symbol or the `string()` function. Both

```
?? PythonTeX ??
```

returns **?? PythonTeX ??** I find the second option clearer in that `*` is an odd choice for string concatenation. Many languages including java and ecmascript (javascript) use `+` instead for string concatenation.

Another cute operation for strings is the caret `^` operation. This could be helpful and a (not so helpful) example is

```
?? PythonTeX ??
```

returns **?? PythonTeX ??** . Other important functions related to strings can be found at Julia's documentation on strings

2.3.2 String Interpolation

Mixing strings and other variables together is often needed. If you have a variable `x` and would like to insert it at the end of

```
?? PythonTeX ??
```

we can use concatenation as above to do this, but instead we will use string interpolation by putting a `$` in front of a variable.

```
?? PythonTeX ??
```

and the result will be **?? PythonTeX ??**

2.4 Expressions

An expression is a combination of variables, data elements (like numbers and strings), operations (like + or *) and functions (like `length`). We've seen a number of expressions throughout this chapter so far like

```
?? PythonTeX ??
```

In short, writing things in julia will consist of writing expressions (and slightly more complicated structures).

2.5 Operator Precendence

When we type out an expression like **??**, it is important to understand the order in which operators are performed. For mathematics, the PEMDAS pnemonic is helpful to rememember in that the order is:

- *Parentheses*: The expression inside the ( ) are done first. For the example above, the `4+3` is the first operation done.
- *Exponentials*: The `^` is done next. Raise the 7 from above to the power of 3.
- *Multiplication and Division*: In this example, the `2*(343)` is done next
- *Addition and Subtraction*: Lastly add 11 to the result.

In any computing language, there are other operators as well and there is order to that precedence, so we will see that there are other things to think about. For example, the assignment operator, `=` has the lowest precedence. That is when assigning something to a variable, all calculations are done on the right side of the = before the assignment.

Details on all this can be found on the Julia documentation page on operator precedence.

2.6 Comments

A comment in computer code is sequences of characters which are ignored. The purpose of a comment is to alert a human on what is going on. You may have been told to write comments so that someone else who reads your code understands what you are doing. However, I have found that the person mostly like to read your code is you at a later date. You should add comments for yourself.

In julia, a comment is anything to the right of a **#**, pound sign or hash tag. For example:

```
?? PythonTeX ??
```

Both lines 1 and 3 have comments. On line 1, the entire line is ignore since the line starts with **#**. On line 3, everything after the 2 (the power) is ignored. Also, notice that there are two hash tags on line 1 and 1 on line 3. This is simply different style. Since anything after a single **#** is a comment, everything after the first one is ignored.

Chapter 3

Introduction to Data Types

In Chapter 2, we saw a little bit about number and string data types. This chapter goes into greater detail about numbers and other datatypes. It is important to understand how integer and floating point number are stored as their binary representation. We will cover this as well.

3.1 Integers

Recall that mathematically, an ***integer*** is a counting number (1,2,3, ...) along with 0 and the negative counting numbers $(-1, -2, -3, \ldots)$. Mathematically thinking, there is no largest (or smallest) integer, however, in reality if we are storing a number on a computer (which is a finite device), there must be a limit on the smallest and largest integers to be stored. Practically speaking, we will limit an integer to some number of bits and the standard sizes are 8, 16, 32, 64 and 128.

3.1.1 Unsigned Integers

First, we will examine unsigned integers and typically these are thought of as the nonnegative integers (0,1,2,3, ...). For example, 8-bit unsigned integers have a total of $2^8 = 256$ nonnegative numbers and since the smallest is 0, the largest is 255.

In julia, the datatypes for unsigned integers are `UInt8, UInt16, UIint32, UInt64` and `UInt128`. We can get the smallest and largest value for these with the `typemin` and `typemax` functions. **??** and
?? return **?? PythonTeX ??** and **?? PythonTeX ??** respectively.

Appendix C go over many of the details of representation integers in binary and performing basic operations. We will cover the a superficial level of integer represenation and operations here in this chapter, but for those with desire for more depth see Appendix C.

In julia, we can use the `bitstring` function to give the binary representation of integers and floating points. For example

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**. Notice that **??** returns **?? PythonTeX ??**.

Similarly, the unsigned integers with more bits work the same with largest range of integers. For example **??** returns **?? PythonTeX ??**which is a string of length 64.

3.1.2 Signed Integers

In julia, the signed integers are `Int8, Int16, Iint32, Int64` and `Int128`. Also, there is a integer type `Int` which defaults to the sized integer of the typical integer size on your machine. This is generally `Int64`.

Let's look in detail about 8-bit signed integers. The largest and smallest values that can be stored with `Int8` can be found with **??** and **??**, which returns **?? PythonTeX ??** and **?? PythonTeX ??** respectively. Bascially the number between 0 and 127 are identical between `Int8` and `UInt8`.

3.1.3 Overflow and Underflow of integer operations

Again, unlike mathematical integers, any computer-based integer has a maximum and minimum values. In short, if an operation results in a number above the maximum, then there is an overflow error and less than the minimum there is an underflow error.

Here's a simple example with 8-bit integers. Let **??** and **??**. The sum of 95 and 70 is 165 and above the maximum value for `Int8`. However, entering

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**, not the expected error.

What just happened? If you want to know why the value of **?? PythonTeX ??** arose, dig into the details in Chapter C, but the reason why there was no overflow error is that julia does not automatically check for such errors, due to the fact that there is overhead in checking, which will slow down operations.

If you want to check, there are a suite of operations that will check, therefore:

```
?? PythonTeX ??
```

will result in **?? PythonTeX ??**

Go to julia's documentation on `checked_add` which starts a list of functions that will check for over and underflow. If there is any chance of overflow/underflow errors, then the results may be wrong. Keep this in mind as in Chapter 16 we will write tests for code.

3.2 Floating Point Numbers

Many fields in scientific computing rely on using decimals and the standard way to store these in a computer is with ***floating point numbers***. Details on floating-point num-

bers are in Appendix C. Julia has 16-,32- and 64-bit floating point numbers called `Float16, Float32` and `Float64` and by default on most systems is the `Float64`.

There are two limitations to any floating-point number. First, the number of digits stored in the number and secondly, the maximum and minimum values. Each built-in type splits the number of bits into storing both and there is a balance between these. A rule of thumb is that

- `Float16` stores 4 decimal digits and the max is about 32,000.
- `Float32` stores 8 decimal digits and the max is about 10^{38}.
- `Float64` stores 16 decimal digits and the max is about 10^{307}

We can using the `bitstring` function in julia to find the binary representation. Notice that

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**. Again, details are in Appendix C but, in short, a floating-point number is stored in scientific notation with the abscissa, exponent and the sign all combined together.

Unlike integers, most numbers cannot be stored exactly with a floating-point number. For example, 1/3 divides 1 by 3 and results in the floating-point number closest to the fraction $\frac{1}{3}$. In julia this is **?? PythonTeX ??**and also note that **??** results in **?? PythonTeX ??**

Notice that there are non-zero bits throughout the number in this case that didn't occur with 8.625. This is because as a fraction 8.625 has a denominator of 8, which is a power of 2. If a fraction can be written with such a denominator, the number in binary has 0s that pad the right end of the number.

What does this matter? Well, consider the following:

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**, which is not 1.[1] This is an example of the limitations of floating-point numbers and 1) either we deal with it or 2) use a different data type (in this case either a `BigFloat` or `Rational` would be better).

Unless you know you have some reason to choose otherwise, choose `Float64` for most floating-point numbers. There are still underflow and overflow errors associated with it, but as we will see in Chapter 9, generally round-off error associated with floating-point number is more detrimental to calculations.

[1]This occurred because the closest floating point to the fraction 1/9 was just slightly above 1/9 and adding up 9 of those numbers results in the extra amount

3.3 Extending integers, the `BigInt` type

In Chapter 8, we will explore prime numbers and it is common for them to exceed the maximum allowable `Int64` or even `Int128`. If this is needed, there is a type called `BigInt` with no maximum or minimum. Here's the number one google

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??** and this is too big to fit on one line of this page. Note: the command `big` create a `BigInt` and generally normal operations with integers result in `BigInt`s as well.

It's very important to understand how **??** works. First a number of type `BigInt` is made with a value of 10. Then that is raised to the 100th power. As noted earlier, a `BigInt` doesn't have a upper or lower limit on the number. It can grow as needed.

If we did **??**, the result is **?? PythonTeX ??**, a surprising result, however note that because of order of operations, first `10^100` is calculated in standard `Int64` and then turned into a `BigInt`. Again, for details on what happens here, look at Appendix C, in short this continually multiplies the number ten, 100 times.

It is recommended only to use a `BigInt` if needed. Operations with them are significantly slower than `Int64` or even `Int128`. Under few cases do you need to do this, however, we will point out in Chapter 8 with prime numbers when we might need to use them.

3.4 Extending Floating Point Numbers with `BigFloat`

As we discussed above, a floating point number has two limitations 1) the number of digits stored and 2) the maximum exponent used. If we need numbers when the limit of either of these is in the way, we can turn to `BigFloat`. For example, what if you want to calculate π to 100 digits.[2]

Maybe do this as an example later in the text.

To get a floating point number of type `BigFloat`, wrap the `big` function around a float. For example **??** returns **?? PythonTeX ??** and we can verify it's type with **??** which returns **?? PythonTeX ??**.

Let's revisit an example from earlier and sum 1/9 nine times. If we try to turn this into a `BigFloat` with **??**, then the result is **?? PythonTeX ??**which seems to have an accuracy of only 17 digits, which is typical for a 64-big floating point, so it doesn't appear to have improved anything. This, like above, is a case of being careful in constructing a `BigFloat`. What happens with **??**? Using your order-of-operations eye, the `1/9` is done first and since both 1 and 9 are regular integers (`Int64`), the result is a `Float64`. Then the `big` function turns the `Float64` into a `BigFloat`, but not we the accuracy expected.

Instead, if we define **??**, then we get **?? PythonTeX ??** which looks more like an expected result. To determine the number of digits of accuracy, you can count (painfully)

[2] And in fact, I calculated π to about 100,000 digits for a talk in Spring 2016 using Julia

or try **??** which will return **?? PythonTeX ??**, which is about 5 times the accuracy of `Float64`. Technically if you type `precision(x)` and this returns 256, which is the number of bits and it has 4 times the binary precision of `Float64`.

Note: the set of nested commands turns the number into a string and then gives the length of the string.

As noted at the beginning of this section, though, if we want to compute π to 100 (or even 100,000) digits, it doesn't appear to work–the above example only gave about 80 digits of accuracy. The `BigFloat` type is quite flexible. The above example used it in its default precision. We can change this with the `setprecision` function. For example:

```
?? PythonTeX ??
```

returns 1024 (or each number is stored in 1024 bits or 128 bytes of memory). Then typing **??** returns a number that, at a glance, looks to store about 4 times the number of digits. Try it!

Entering

```
?? PythonTeX ??
```

returns **?? PythonTeX ??** as the number of decimal digits. This is about 20 times the precision of `Float64`.

3.4.1 Limitations of `BigFloats`

Although a `BigFloat` stores more digits and allows a wider range of possible numbers, it still has a limitation. Once you create a `BigFloat`, it uses the given precision from `setprecision` and cannot be changed. So typically, thought needs to be put in before using `setprecision` on what the desired level of precision that is needed.

Additionally, operations with `BigFloat`s are significantly slower that the `Float64` type and unless you need the extra precision, stay with the built-in method.

3.5 Rational numbers

A rational number is the ratio of integers and more commonly called a fraction. Julia (unlike many other general computing languages) has rational numbers built-in. To put in a ratio, enter a `//` to separate the numerator and denominator. For example:

```
?? PythonTeX ??
```

are examples of rational numbers. One advantage that they have is that the numerator and denominator are stored as integers (64-bit by default) and are not subject to round-off errors that floating points are. The standard operations $+, -, \cdot, \div$ between rationals results in a rational and as we will see in this course, there are advantages to using rationals instead of floating points.

3.5.1 Exercise

Perform the following operations involving rationals in Julia:

1. $\frac{1}{2} + \frac{2}{3}$
2. $\frac{1}{2} - \frac{2}{3}$
3. $\frac{2}{3} \cdot \frac{3}{5}$
4. $\frac{2}{3} \div \frac{3}{5}$

3.5.2 The Rational Type

If you enter **??**, note that julia returns **?? PythonTeX ??**and this is called a Parametric Composite Type, which will be talked about later. In this particular case, this is a rational type, but inside it (the numerator and denominator), they are type `Int64`. For example, to make a different type of rational you need to declare a different integer type inside, enter

```
?? PythonTeX ??
```

and if you check the type of this, you will see it is of type `Rational{Int16}`.

3.5.3 Exercise

Build the rational $\frac{1}{2}$ with BigInts within it. Check that in fact it is stored as you expect.

3.5.4 Other Operations with Rationals

As long as you stay in the basic operations, $+, -, \cdot, \div$, the result will be a rational. However, many other operations are not. For example `sin(1//2)` will return a floating-point number. `(2//3)^3` will return a rational, since this is ultimately multiplication, but `(2//3)^(1//2)` will return a floating-point since raising a number to the $1/2$ is the same as square root.

3.6 Complex Numbers

Recall that the imaginary number i is defined as $\sqrt{-1}$. A complex number is a number of the form $a + bi$ for a and b in general real numbers. In julia, there is a built-in constant **??**, which can be used to create complex numbers. for example **??** has type **?? PythonTeX ??**, which is a composite type with the values of the numbers a and b are `Int64`.

Complex numbers play a huge role in many aspects of scientific computing and in some cases, formulating a problem using complex numbers can make calculations faster and easier to program. This includes the very important algorithm, fast-fourier transforms or better known as FFTs. This and other examples are explained in Chapter 23.

3.7 Abstract and Concrete Number Types

The numerical data types we have seen in this chapter are examples of **concrete data types** in that we can create data (usually numbers) with those types. These include the integer types `Int8, Int16, Int32, Int64, Int128, BigInt` and floating-point versions `Float16, Float32, Float64, BigFloat`. The rational and complex types are composite, however the internal part is one of these types.

Julia is a bit different than other languages in that there are also **abstract data types** that 1) you can't make data in the type and 2) are collections of other types.

3.7.1 Abstract Number types

For example, `Integer` is the abstract type (also called a supertype) of all integer types. The other abstract number types are:

- `Signed`: supertype of all signed integers like `Int32, BigInt`.
- `Unsigned`: supertype of all unsigned integers like `UInt32,UInt128`.
- `Integer`: supertype of all signed and unsigned integers.
- `AbstractFloat`: supertype of all floating-point numbers.
- `AbstractIrrational`: supertype of irrational numbers.
- `Real`: supertype of all floating-point, rational, irrational and integer numbers.
- `Number`: supertype of all numbers.

See a bare-bones description of all of Julia's standard number types.

3.7.2 Concrete Number Types

The numbers shown above are concrete number types like:

- `Float16`, `Float32`, `Float64`, `BigFloat` which are all subtypes of `AbstractFloat`
- `UInt8`, `UInt16`,`UInt32`,`UInt64`,`UInt128`: which are all subtypes of `Unsigned`
- `Int8`, `Int16`, `Int32`, `Int64`, `Int128`, `BigInt`: which are all subtypes of `Signed`
- `Rational` types are subtypes of `Real`
- `Complex` types of subtypes of `Number`.

To test if something is a subtype of another use the `<:` operation. For example

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**, but

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**.

3.8 Converting numbers

?? converts any type of number to a floating point, like

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**.

`parse(Type,str)` parses `str` of type `String` into a number of type `Type`.

```
?? PythonTeX ??
```

returns the integer **?? PythonTeX ??** and

```
?? PythonTeX ??
```

returns the floating point **?? PythonTeX ??**.

If you have a number in another base, you can still parse it. For example, consider the binary number 10011,

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**.

3.8.1 Converting to Integers

If you have a floating point number or rational and you want to convert to an integer, typically use the `ceil`, `floor`, `round` functions. For example

```
?? PythonTeX ??
```

returns the integer **?? PythonTeX ??**, since ceiling rounds up to the next integer.

3.9 Tuples

Another type that allows one to combine two or more elements of data is called a tuple. For example, if we want two floating points that are connected, we can make:

```
?? PythonTeX ??
```

For example, this could be a way to store a geometric point. Determining the data type with **??** returns **?? PythonTeX ??**, which is another example of a composite type. The type is a `Tuple` but there are `Int64` types with the tuple.

You can access the elements of the tuple using bracket notation. If

```
?? PythonTeX ??
```

we can get the first element of `tup` with **??**. Note, this is similar to array notation, which we will see in Chapter 6.

3.9.1 Named Tuples

Sometimes it's easier to associated the individual elements of a tuple with a name instead of an index. We can generate a ***named tuple*** as

```
?? PythonTeX ??
```

which would be like a point in three dimensions. We can access the elements using dot notation, such as

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**.

3.9.2 Tuples are Immutable

However, if we try to change one of the elements of a tuple, then an error will occur.

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**

This is because tuples are immutable, meaning once created, any part of the tuple cannot be changed.

Chapter 4

Introduction to Functions

In any language, the function is one of the most important ideas and this chapter covers the introduction of functions with examples in julia. There are three main purposes of functions in scientific computation.

1. A function in a computer language mimics that of a mathematical function, which are crucial in this field.
2. Functions simply code. Wherever code is repeated either exactly or nearly exactly, functions are one way to reduce the amount of code written and makes code conceptually easier.
3. Functions abstract code. If you can create a piece of code that does a particular task, then this helps in the abstraction process.
4. Functions allow separation of code. If there is a large code and a section of it does a specific task, making it a function will separate code.

The Julia documentation on functions is a has additional information on functions and we will cover some advanced features of functions in Chapters 7 and 15.

4.1 Simple example

A simple example of a julia function is

```
?? PythonTeX ??
```

which just returns the square of the argument, called `x`. The name of the function is `sq` and the `x` inside the parentheses is called the ***argument*** of the function. Both the function name and any argument must adhere to the rules of variables from Chapter 3.

An alternative way to do this is the following:

```
?? PythonTeX ??
```

which is often used if a function is a single line of code. This also looks a lot like a mathematical function. To make it even more mathematical, we could also write this as

```
?? PythonTeX ??
```

To call a function, it is much like that of any other language. If we type **??**, which returns **?? PythonTeX ??**, the square of 3 and **??** returns **?? PythonTeX ??**.

4.2 Function Arguments

The function arguments are the numbers or strings or an element of any data type that is passed into the function. In the function `sq` above, the number `x` is the only argument. We can have more arguments by separating by commands. The (quite unnecessary function) `theSum` will take two arguments and add the result:

```
?? PythonTeX ??
```

Typically, if a function can be written in one line, we will use this style of functions. For more complex functions, use the `function` keyword as a block of code.

4.2.1 Exercise

Write a function `theMean` that finds the mean (average) of two arguments `x` and `y`.

4.3 Returning values from a function

The functions `sq` or `theSum` returned the value that is the last line of the function. If you want to return a value before the last line you can use the `return` command. The following will return true if the number is odd and false if the number is true:

```
?? PythonTeX ??
```

The `mod` function is the remainder of `n` divided by 2 and can also be written `n % 2`. If the remainder is 1, then the number is odd. On line #3, `return true` the code stops here and exits the function and doesn't execute any of the other lines.

Note: the `==` tests for equality. This will be discussed in chapter 5.

A better way to write this (but doesn't use the return statement) just evaluates if `mod(n,2)` is 1 or not:

```
?? PythonTeX ??
```

which will return `true` if `mod(n,2)` is actually 1 and `false` if it is anything else (but the only other possibility is 0). Again, since this is just one line, we'll use the shorthand notation.

4.3.1 Indentation in Functions

You should notice that the `isOdd` function written above as a block of 5 lines has different indentation. In Julia, the spaces don't matter[1] but it is standard to indent for clarity. Notice first that in all of the functions so far, the code has been indented[2] two spaces and then the `if` block is indented again 2 spaces.

4.4 Specifying Argument Types

The `isOdd` function above should only work on integers, but if type `isOdd(3.5)` you get a result. (But does it give you want you want?). What if you enter **??**? Try it.

Since odd numbers only make sense with positive integers, declaring the type of argument makes sense. Therefore, if instead, you specify a type in the following way:

```
?? PythonTeX ??
```

where the double colon, `::` tells the type of the argument `n`. Any type that is an Integer can go in here and note that we used the abstract data type `Integer` as seen in section 3.7.

Also, make sure to restart the kernel and details on how to do this can be found in section A.4. After restarting the kernel make sure that you rerun the function `isOdd` above.

Once you have done this, entering **??** returns: **?? PythonTeX ??**

This just means that there is not function `isOdd` that takes a `Float64` as an argument, which is what we want. A similar error should now occur for **??**.

Throughout the rest of this text, we will always specific an argument type. This is not only good style and practice, we will see that this results in faster code.

4.4.1 Exercise

Rewrite the `theMean` function from above using types for the arguments. Since both floats and integers are real numbers, use the abstract `Real` type for this. Restart the kernel and test your function using both numbers (floats, integers or rationals) and non numbers (like a string).

4.5 Multiple Dispatch

Before starting this, make sure that you have completed the exercise above to write a 2-argument version of the mean and it is currently in the kernel (this means, just run it again, if in doubt.)

[1] this isn't true of all languages. Python, for example, relies significantly on indentation for blocks of code and therefore the `end` isn't needed to terminate a block.

[2] I have a preference for 2 spaces, but 3 and 4 are common as well.

It would be nice to have a mean function that takes more than 2 numbers as well, so the following is a three-argument version of the mean function can be written

```
?? PythonTeX ??
```

Depending on the number of arguments, julia will call the appropriate function. This is an example of Multiple Dispatch, in which either the number or type of arguments determine the actual function call.

If you look back at your document when you declared the functions, you should see that the second one entered said `theMean (generic function with 2 methods)`, which says that there are two functions called mean. Typing **??** results in: **?? PythonTeX ??**

Much of yours will be different depending on using the REPL or a jupyter notebook, however the important part is that you will see that there are 2 methods and the rest shows where they were defined.

Multiple dispatch also allows different types of arguments as well. Let's say we want to create a function `mean` that take a single string as a argument like:

```
?? PythonTeX ??
```

and entering it shows you that there are now 3 functions. Try typing

```
?? PythonTeX ??
```

4.5.1 Multiple Dispatch of built-in functions

Multiple dispatch allows julia to be quite nice. For example, the **+** function allows code to be written with the plus symbol and execute different code. Type **??** and the top of the results should be similar to

```
# 166 methods for generic function "+":
[1] +(x::Bool, z::Complex{Bool}) in Base at complex.jl:286
[2] +(x::Bool, y::Bool) in Base at bool.jl:96
[3] +(x::Bool) in Base at bool.jl:93
[4] +(x::Bool, y::T) where T<:AbstractFloat in Base at bool.jl:104
[5] +(x::Bool, z::Complex) in Base at complex.jl:293
[6] +(a::Float16, b::Float16) in Base at float.jl:398
[7] +(x::Float32, y::Float32) in Base at float.jl:400
[8] +(x::Float64, y::Float64) in Base at float.jl:401
[9] +(z::Complex{Bool}, x::Bool) in Base at complex.jl:287
[10] +(z::Complex{Bool}, x::Real) in Base at complex.jl:301
```

A few things about this:

- There are 166 different methods for `+`.
- The 6th line lists the line in a file where julia adds two 16-bit floating points.
- The 7th line lists the line in a file where julia adds two 32-bit floating points.

If you are using jupyter notebooks, then you can click on the filename:line number and this will take you to the line in the code for that method. This shows the power of open source code. You can actually find out exactly what is run when code is executed. This isn't possible with closed-source software and one nice aspect of this is that if you think and show that you can produce code that runs faster than a built-in version, generally it will be adopted into the codebase.

4.6 Variable Number of arguments

From the last exercise, it would be unfortunate if we have to write different functions for different number of arguments. We can write a variable number of arguments with a ... trailing the last argument. The following is a generalized version of the mean:

```
?? PythonTeX ??
```

which uses a for loop that we will discuss later. This function will now find the mean using any number of arguments. Try **??**, **??** and **??** and determine if it is returning what you expect.[3]

Also, note that the argument `x` with the ... is a tuple, (see section 3.9). An alternative way to access the individual elements of `x` would be to use brackets. For example, `x[3]` would be the third argument.

4.7 Multiple Return Values

A very nice feature of Julia functions is that of multiple return values. Instead of only being able to return a single number (or requiring to send an array or structural type), you can return more than 1 number (or other data type). For example.

```
?? PythonTeX ??
```

and if you call this, say **??** you will get the result **?? PythonTeX ??**or if you say

```
?? PythonTeX ??
```

[3]You may notice that the mean of the two rational numbers results in a floating-point number. A better way to do this would be to return a rational.

the **??** will take on the value **?? PythonTeX ??** and **??** will take on the value of **?? PythonTeX ??**.

The result of this function is a tuple as we saw in section 3.9. Although in that section we used parentheses around the tuple, it is not necessary and generally isn't used to return a tuple in a function. We will use this for the result of the quadratic formula in Chapter 9.

4.8 Factorial Function

Mathematically, we define the factorial as a function on a non-negative integer as

$$n! = n(n-1)(n-2)\cdots 3 \cdot 2 \cdot 1$$

or the product of all of the numbers from itself down to 1. There are a number of ways to program this function as we will see. Using

```
?? PythonTeX ??
```

uses a `for` loop and we will see the details of this in Chapter 5. The for loop first assigns `i` the value 1 then executes the lines, then sets the value to 2, then executes the block, and so on until `i` is `n`. Since `prod` starts as 1, this multiplies `prod` by every integer between 1 and n, and thus is the factorial. The result in `prod` is returned.

4.8.1 Exercise

- Test the function above for various positive integers.
- What happens if you put in 0 or a negative integer?
- What happens if you put in a number that is not an integer?

4.9 Recursive Functions

Any function that calls itself within its block of code is called a ***recursive function***. One of the standard examples of this is the factorial function.

Above, we saw how to compute the factorial of a number using a for loop. There's another way to do this. We can define the factorial in the following way:

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & \text{otherwise} \end{cases}$$

and this is a mathematical piecewise function that returns 1 if $n = 0$ and otherwise returns $n \cdot (n-1)!$.

The reason that this is recursive is that the factorial function is within the function. That is it calls itself. We can write the julia version of the factorial in the following:

```
?? PythonTeX ??
```

where `n == 0` tests if the variable `n` is 0 or not. If `n` is 0, then 1 is returned. Otherwise, **??** is calculated and then returned. The equality test as well as the if-then-else statement will be covered in detail in Chapter 5.

4.9.1 Exercise

Another example of a recursive function is that of a Fibonacci number. If we let $f(1) = 1$, $f(2) = 1$, and then

$$f(n) = f(n-1) + f(n-2)$$

for $n > 2$. The first few values are 1, 1, 2, 3, 5, 8, 13, 21, ...

Write a recursive function that produces fibonacci numbers. Test it on values of n that are smaller than 20[4]

4.10 Variable Scope

If you are using the command line julia (also called the REPL) or iJulia, then entering **??** will create `x` as a global variable, even without saying so explicitly. If we consider the factorial function above:

```
?? PythonTeX ??
```

then recall that `n` is the function argument and the variable `prod` is actually declared locally implicitly and it is good form to say it is local by

```
?? PythonTeX ??
```

The variable `prod` is not defined or visible outside the function. If we type `2*prod` for example, an error will occur saying that `prod` is not defined.

If we have:

```
?? PythonTeX ??
```

then this will return **?? PythonTeX ??** and typing `x` (to see it's value) returns **?? PythonTeX ??**, indicating that there are two different `x` variables.

If we indeed need a variable inside of a function to use a global, we can use the `global` keyword. If instead we have

[4] If you play with this a bit, you will that computing the fibonacci numbers in a recursive manner, although simple, is very slow as the values increase. In section 7.7, we will examine another way to compute this more quickly.

```
?? PythonTeX ??
```

then executing **??** returns **?? PythonTeX ??**, showing that it is using the global value. We will use a global variable to determine the number of function evaluations to test a function in section 7.7.

Chapter 5

Boolean Statements, Loops and Branching

The basic computer science structures of if statements, while and for loops are crucial in scientific computing. In this chapter, we cover the basics of these structures in Julia. We will see all of these in more context in later chapters, but here's the syntax and basics.

Lastly, because of the syntax of a `for` loop, we show details about ranges in julia, which is a compact way to write a set of numbers that are either sequential or sequential with skips in it.

5.1 Boolean values and if statements

A boolean value is something that is either `true` or `false`. These are built-in constants in Julia. Sometimes we will want to know if a statement is true or false, but generally, we will use them in other structures.

We often use boolean to test various conditions. For each, testing equality, or comparison of numbers we use `==,<,>,<=,>=` for equality, less than, greater than, less than or equal or greater than or equal respectively.

If we set `x=3` and then can just type `x==3`, `x<3`, `x>=3` to test a variety of comparisons.

5.1.1 Compounds boolean statements

We often want to test multiple boolean statements and can build up compound ones with either the "and" (`&&`) or "or" (`||`) operators. Recall the following table for `&&` and `||`

AND	T	F	OR	T	F
T	T	F	T	T	T
F	F	F	F	T	F

If we have **??** and **??**, if we want to test that `x` is greater than 0 and `y` is 5, by

```
?? PythonTeX ??
```

which will return **?? PythonTeX ??**, since only the first is true and both must be true for this compound statement to be true. However,

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**, because the first is true.

In both of these examples, it is important to note the order of operations or operator precedence. This was mentioned in section 2.5, however as the number of operators grows, it's important to know the precedence. In these cases the tests `==`,`<=`,`<`,`>=`,`>` have precedence over `&&` and `||`.

Also, `&&` has precedence over `||` in that if we evaluate

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**. You can think of this resulting in **??** and because of precedence the first pair is tested (to be `true`) then the result **??** results in `true`.

Often when precedence is unclear, adding parentheses can be helpful. Instead, perhaps write the above as:

```
?? PythonTeX ??
```

5.2 *If Statements*

An `if` statement is used to do different things depending on the value of a variable. A standard example of this is the piecewise version of the absolute value. Mathematically, we write:

$$|x| = \begin{cases} x & x \geq 0 \\ -x & x < 0 \end{cases}$$

We could write this as a function as

```
?? PythonTeX ??
```

Evaluate **??** and **??** to ensure this is returning expected results.

Notice that we basically had two situations here, either x was greater than or equal to 0 or else not. We can rewrite this use an `if-else` statement.

```
?? PythonTeX ??
```

Try entering this and seeing if the results are the same.

The absolute value function is quite important in mathematics and thus is built-in to julia as `abs`.

5.2.1 Further choices with if statements

You may need more than 2 choices on an if statement. Recall the quadrants of the xy-plane start at I for the upper right and increase as you traverse counterclockwise. A function could be written:

```
?? PythonTeX ??
```

and technically if you are on an axis, then you are not in a quadrant, so that is the reason for the last option.

5.3 Ternary `if-else` statement

Often, an if-else statement is quite short and you want to a value depending on a condition. The absolute value example was such an example. There is what is called a ternary `if-else` statement that has the form:

```
condition ? value_if_condition_is_true : value_if_condition_is_false
```

which returns `value_if_condition_is_true` if condition is true otherwise `value_if_condition_is_false` is returned. The absolute value example above can be written as a single line:

```
?? PythonTeX ??
```

and once you practice with this, it will be easy to read and much shorter (1 line versus 7). Be careful with the syntax of this. It is required that the expressions around the ? be padded with spaces to parse correctly. The error is reasonably clear if you don't write it correctly.

Alternatively, you can write this as

```
?? PythonTeX ??
```

We will see else syntax of this operator often throughout this course.

5.3.1 Exercise

Write the recursive factorial function above using the ternary if-then-else.

5.4 Loops

A *loop* is a series of statements that are repeated either a fixed number of times or until a condition occurs. They can be very helpful if a large number of operations need to be done in a predictable manner.

5.4.1 While Loops

Another very common construction for programming is called a while loop. Basically, we want to run a few statements while some boolean statement is true. Here's a simple, but uninteresting example:

```
?? PythonTeX ??
```

and note that the expression `n+=1` is shorthand for `n=n+1`.

A more practical example of a for loop will be the Bisection Method for finding a root.

```
?? PythonTeX ??
```

In short, this method takes a function f and an interval $[a, b]$ and continually bisects it ensuring there is a root in the resulting interval. It continues while the length of the interval is greater than `1e-6` or 10^{-6}. To test it, consider

```
?? PythonTeX ??
```

which has a root of $\sqrt{2}$. The function call

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**, which is approximately $\sqrt{2}$.

5.4.2 Infinite Loops

It is common in a `while` loop to keep running it forever. This occurs if there is some bug or you haven't considered all cases. For example, in the bisection method above, if the function doesn't have a root (like $f(x) = x^2 + 2$), then this will never stop.

Here's a few things that can help prevent or debug the code:

- Make sure something is changing in your loop. If you intend to stop the loop on an index, make sure the index is updating.
- Look at your code and see if you have something that you think will stop the loop. What ever is in the boolean statement needs to eventually switch.
- Consider an additional stopping condition. You may need to add a variable to count the number of times you've gone through the loop and stop if it hits some maximum, which is greater than what you would expect.
- Stop the code if you need to. You may need to interrupt the kernel. In the REPL, CTRL-C will stop and in Jupyter, selecting the ***Kernel*** menu then ***Interrupt*** should stop it. The square in the toolbar should work too. See section A.4 for more information.

- If you can't figure out why it is in an infinite loop, use `@show` to print out values of variables.

MAYBE CITE A DEBUG SECTION?

5.4.3 For loops

A for loop executes some code a fixed number of times and has a variable (called an index) that updates. The next few examples are not practical, but are designed to illustrate the syntax and what is possible.

The following is a simple for loop that prints out the numbers 1 to 10

```
?? PythonTeX ??
```

If you want to skip numbers or count backwards, a range in the form ***start***:***skip***:***stop*** is used. The following starts at 1 and skips by 2 up to 21:

```
?? PythonTeX ??
```

and the following starts at 10, counting down to 1:

```
?? PythonTeX ??
```

We can use the `for var in list` form to go through elements in a list or array (or tuple). The following adds all numbers in the list (array) `[1,5,7,11,20]`:

```
?? PythonTeX ??
```

Recall that in Chapter 4.6 we used this in the formation to find the mean:

```
?? PythonTeX ??
```

5.4.4 While Loops Versus For Loops

No this, isn't a smackdown between these while and for loops. A big question often is when I have a problem that I want to solve and I know a loop is needed, when should I use a `while` loop and when should I use a `for` loop. The general rule of thumb is:

- If you know that you need to run code for a fixed number of times, use a for loop
- If you don't use a while loop. Generally, the ***doing*** something in the loop will affect how many times the loop is run.

Notice in the examples above, the bisection method used a `while` loop because it is unclear when you start how many times you want to go through the loop. Instead we stopped in while the interval was small enough.

In the summing of the terms for the mean, a for loop was used because of the fixed number of values in the tuple `x`.

5.5 Ranges

In a for loop with the syntax `var=x:y:z`, such as

```
?? PythonTeX ??
```

The syntax `1:4` is called a `Range` and is shorthand for all integer between 1 and 4 inclusively. Technically, this is called a `UnitRange` because the skips between numbers is 1.

If you type **??**, it will return **?? PythonTeX ??**, where is a `UnitRange` type with integers. Note the difference using `1.0:4.0`. This is of type **?? PythonTeX ??**which is a little complicated, but basically a range with floating points.

The other ranges we say included **??** and entering **??** results in **?? PythonTeX ??**.

We will see in Chapter 6 that there is a handy way of expanding this shorthand way of writing these sets of numbers to that of an entire list (or technically array) of numbers.

5.5.1 Linear Ranges

As we see above, it easy to make ranges if we know the step between each pair. Another use would be to know the starting and ending value and the number of values. For example, if we want to start at 3 and end at 11 with 5 values, it's not too hard to compute this, but julia has a built-in way to do it with the `range` command. For example,

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**.

Another nice function that does a similar thing is the `LinRange` function.

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**.

Chapter 6

Arrays

Arrays are a fundamental data structure for nearly every computer language and it is a crucial for scientific computing as that it is an efficient way of handling large amount of the same datatype. Additionally, arrays are the computing version of mathematical vectors and matrices.

In short an array is a collection of data of (typically) the same type under the same name. We have seen arrays before. For example, if we say

```
?? PythonTeX ??
```

then we get a 1-dimensional array of length 3. Note that when returned, julia says `3-element Array{Int64,1}:` which means

- it is length 3 (since it is a 3-element)
- The internal type is `Int64`.
- it is 1-dimensional, because the 1 as the second element in the {}.

There are some basic functions that tell us some information:

- `length(arr)` returns 3, the length.
- `eltype(arr)` returns `Int64`, the type of the elements in the array.

6.1 Constructing arrays

We can create arrays in many ways. As seen above, the line

```
?? PythonTeX ??
```

creates a 1-D array of length 3. A 2D array can be made like:

```
?? PythonTeX ??
```

Note that each row of the array is separated by a semicolon and the individual elements in a row are separated by spaces. The result of this is **?? PythonTeX ??**and the size is 2 by 3 (2 rows and 3 columns). This is a 2D array in that there is now a 2 after the comma in the `{}`.

The following are useful for creating arrays of 1 or more dimensions.

- `zeros(type,dims...)` makes an array of all zeros with datatype `type` and given dimensions. For example,

  ```
  ?? PythonTeX ??
  ```

  returns an array filled with zeros (of integer type) with 4 rows and 6 columns.

- `ones(type,dims...)` is similar to `zeros` except it is filled with 1s.

- `rand(dims...)` makes a random array of floating points (uniformly distributed between 0 and 1).

- `collect(range)` takes a `Range` object (the pair or triples of numbers separated by columns) and creates an 1D array with the numbers from the range. For example, **??** returns **?? PythonTeX ??**

6.2 Comprehensions

If the elements of an array form a functional pattern, we can use what is called a ***comprehension*** to construct it in a compact manner. For example, a 1D array with the elements 1,-1,1,-1,1,-1 can be made:

```
?? PythonTeX ??
```

and the following:

```
?? PythonTeX ??
```

makes an 8 by 8 matrix where each element is the sum of the row and column number or **?? PythonTeX ??**

6.2.1 Exercise

Using the techniques in this section, create the following arrays in julia:

1.

$$\begin{bmatrix} 3 \\ 6 \\ 9 \\ 12 \\ 15 \\ 18 \\ 21 \\ 24 \end{bmatrix}$$

2.

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

3.

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}$$

4.

$$\begin{bmatrix} 1 & -3 & 4 & 5 \\ 3 & 2 & 11 & -13 \\ 17 & 0 & 4 & 2 \end{bmatrix}$$

5.

$$\begin{bmatrix} 4 & 3 & 2 & 1 & 0 \\ 3 & 2 & 1 & 0 & 1 \\ 2 & 1 & 0 & 1 & 2 \\ 1 & 0 & 1 & 2 & 3 \\ 0 & 1 & 2 & 3 & 4 \end{bmatrix}$$

Hint: there is no pattern for the array in #3, so just enter the numbers.

6.3 Accessing elements of an array

Let **??** which generates: **?? PythonTeX ??**. To access the 2nd element, we type **??**. To get an array the elements 3, 4 and 5, type

```
?? PythonTeX ??
```

or if we want the last 3 elements, we can use:

```
?? PythonTeX ??
```

where `end` is an alias for the last element, so this returns an array of the last 3 elements of `x`.

Let

```
?? PythonTeX ??
```

to access the 1st row, 3rd column, type **??**. If you want the subarray of the first and 3rd and 5th columns and all rows type:

```
?? PythonTeX ??
```

returns the matrix **?? PythonTeX ??**where the : in the first slot means all rows and `1:2:5` are the columns 1, 3 and 5.

If we want the first 2 rows and then shuffle the 2,3 and 5th columns in the order 5,3,2 type

```
?? PythonTeX ??
```

to get **?? PythonTeX ??**There are a number of different ways to return subarrays. See indexing in Julia matrices documentation for more information.

6.4 Common Operations on Arrays

There are a number of operations on array. For each `+` and `-` adds and subtracts two arrays of the same size in an element by element manner. For example if

```
?? PythonTeX ??
```

and

```
?? PythonTeX ??
```

then **??** returns the matrix: **?? PythonTeX ??**
and `B-A` returns **?? PythonTeX ??**

6.5 Element by Element Operations

Many methods exist to simplify a ***vectorized*** method that returns the operation applied element by element to the matrix. To do this, most operations have a `.` variety. For example, if we want multiply `A` and `B` in an element by element manner then

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

Note that **??** performs matrix multiplication and we will discuss this in Chapter **??**. Many other operations can be done in similar way. For example, to take the square root of every number inthe matrix `A`, we can enter **??** which returns: **?? PythonTeX ??**

6.5.1 Exercises

Let **??** and **??**.

1. Find and explain the results of
   - **??**
   - **??**
   - **??**
2. Find the sin of every number in `A`.

6.6 Other Operations on Arrays

There are also other operations on arrays. For example, summing all elements in an array is just the `sum` functions. For example:

```
?? PythonTeX ??
```

returns 55. There are also the `min`, `max` and `prod` functions, with the last one, the product of elements.

```
?? PythonTeX ??
```

6.7 Sorting Vectors and matrices

Julia can sort an array using the `sort` function. If **??**, then

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**. The `sort` function returns a sorted array. If you want a sorted version of `A` in place, use the `sort!` function instead.

Also, you may want to sort in a descending manner. If this is true use the keyword argument `rev=true` in the `sort` or `sort!` function. For example:

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**.

Check out the Julia docs on sorting and sorting algorithms for more details on how Julia does sorting.

6.8 Push and Pop; Array as a Stack

There is a computer science data structure called a stack which acts like a stack of things (paper, dishes, pokemon cards) and there are two operations on it: 1) put something on top of the stack or 2) take something off the top of the stack. Many languages including julia don't have a separate data structure but use an array like one with two operations, `push` and `pop`. In julia these have ! at the end to indicate that the operations change the array.

If **??** then

```
?? PythonTeX ??
```

returns the array **?? PythonTeX ??**. And multiple values can be pushed.

```
?? PythonTeX ??
```

results in the array **?? PythonTeX ??**.
To get an element off of the end of the array, we can use `pop!`. If **??**, then

```
?? PythonTeX ??
```

returns 5 and now the array `A` is **?? PythonTeX ??**.

6.9 Other Nice Array Functions

Julia has a ton of other nice functions that act on arrays.

6.9.1 Append

If we want to concatenate two arrays, we should use the `append!` command. For example:

```
?? PythonTeX ??
```

and now the array stored in `A` is **?? PythonTeX ??**.

6.9.2 Adding and Removing Elements in the Middle of an Array

The functions `push!` and `pop!` add and remove an element from the end of a `Vector` or 1D array. If we want to add or remove elements in the middle there is are the functions `insert!` and `deleteat!`. They both act on a position in the array and you can see how they work with the following examples. If **??**, then

```
?? PythonTeX ??
```

results in the array `A` being **?? PythonTeX ??** and then resetting the array with **??** then

```
?? PythonTeX ??
```

results in the array `A` being **?? PythonTeX ??**.

6.9.3 It Splices, It Dices!! — okay, it doesn't dice

Although you can get a long ways with `push, pop, insert` and `deleteat`, the `splice!` function is a Swiss Army knife for arrays. It can take an array, pull elements out and put elements in. We will walk through the options:

Removing elements from an array We saw using the `deleteat!` function above how to remove a single element, but if

```
?? PythonTeX ??
```

then

```
?? PythonTeX ??
```

removes and returns the 3rd and 4th elements which are **?? PythonTeX ??**. Also, the array `A` is the original array without these elements or **?? PythonTeX ??**.

If we let **??**, then

```
?? PythonTeX ??
```

returns the element in the 2nd position vector or **?? PythonTeX ??**. The array `A` is now **?? PythonTeX ??**. This is the same as `deleteat!(A,2)` we saw above.

Inserting elements from an array We saw the function `insert!` above, which will insert a single element in a 1D array. The `splice!` function will insert multiple elements. For example, if we have

```
?? PythonTeX ??
```

then to insert elements in the 3rd position (and shifting everything else to the right), we can enter

```
?? PythonTeX ??
```

then the function returns **?? PythonTeX ??**. This is an empty array of type `Int64`. The result is empty because nothing was removed in the process. The array `A` is **?? PythonTeX ??**

A few things to note about this. The second argument is a bit strange in that you insert `3:2` which is a range in which the last element is smaller than the first. This will let julia know that you don't want to remove any elements, unlike either a single number or a range with the first element smaller than the last.

Replacing elements from an array We say `splice!` is the swiss army knife, because it does even more!! (Now, how much would you pay?). This last section shows that we can both remove and insert elements into an array at the same time. If **??**, then

```
?? PythonTeX ??
```

returns **?? PythonTeX ??** and replaces the 3rd position with the number 4. The result is **?? PythonTeX ??**.

If **??**, then replacing the 3rd element with `[-1,-2,-3]` by

```
?? PythonTeX ??
```

returns 5 (the element in position 3) and now the array `A` is **?? PythonTeX ??**.

6.9.4 Filtering Arrays

A very handle array method, is `filter` which takes an array and returns only the elements that satisfy some condition.

If **??**, then

```
?? PythonTeX ??
```

returns all elements that satisfy that the mod 2 is 0 (or even numbers) or **?? PythonTeX ??**

In general, the `filter` command has the form:

```
?? PythonTeX ??
```

where the result is the array consisting of elements in `array` in which the function `cond` is true.

In addition, if you want to filter on an array with the results in the place of the original array use `filter!` instead.

6.9.5 Removing Duplicate Elements

Another handy function is the `unique` function that takes an array and returns only the unique elements.

If **??** then

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**. And if you want to update the original array, use the `unique!` function instead.

Chapter 7

Functional Programming and an Introduction to Writing fast code

This chapter introduces ideas of functional programming, which is a way to think about programming using functions. Typically this requires a language that falls into the category of a ***functional computer language***, one of which is Julia. In short, a language that has this feature has a function (or procedure) as a fundamental element of the language and one of which can be passed as arguments.

7.1 Functional vs. Non-functional forms

Often functional programming is used with arrays or lists of other data. Let

```
?? PythonTeX ??
```

Here's a simple example of taking all of the elements of `arr` and finding another array which is the square of each number. In a statement-based point-of-view:

```
?? PythonTeX ??
```

which returns the array **?? PythonTeX ??**.

From a functional point-of-view, a common method is a `map` command which takes in an array and produces another array in which a function acts on each element of the array. In this case, the function is the square.

```
?? PythonTeX ??
```

or more simply using an **anonymous function**,

```
?? PythonTeX ??
```

The key to this example is the ability to pass the function as an argument to the `map` command. Once you get a feeling for functional programming, it is often an easier way to write code.[1]

[1] An extreme example of this is to avoid using any for loops after understanding maps and other related

7.1.1 Anonymous Functions

The example above included in the first argument `x->x^2`, which is an example of a *anonymous function*. In short, it is anonymous because we didn't assign it to a name and instead just applied the function to the `map` command.

Any time that a function is used in the form `->`, it is anonymous. For example, if we want to sum the contents of the array **??**, one way is to

```
?? PythonTeX ??
```

which results in **?? PythonTeX ??**, the sum of the elements of the array. We'll see the `reduce` function in the next section. Concentrate right now on the `(x,y)->x+y`, an anonymous functions can take on more than one variable.

7.1.2 Exercise

- Write a statement-based loop to take the array `[1,2,3,4,5]` and output an array that is the reciprocal of each number.
- Write a functional-based code using the `map` command to do the same.

7.2 Reducing an array

The `map` command in general takes in an array and returns an array. Another common task with arrays is to reduce the entire array to a single value. If **??**, then we can sum the values with the `reduce` command as in:

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**.

To multiply all numbers, we can type:

```
?? PythonTeX ??
```

and the result is **?? PythonTeX ??**. In both of these cases, there is a little bit hidden, so let's look at these in more detail.

1. First, the first argument of the `reduce` command is a 2-argument (or binary) function.
2. Secondly, the operation is done on all numbers on the array, but needs to start with some value (because it is a binary function). In the sum case, the initial value is 0 (by default) and in the multiply case it is 1.

functions. Although, often this can happen, it doesn't 1) make the underlying code any faster or 2) any easier to understand, so I don't generally prescribe to this philosophy.

These are perhaps obvious examples and each of these have the built-in functions `sum` and `prod` that find the sum and product of an array.

Let's look at an example that returns the number of array elements that are greater than zero:

```
?? PythonTeX ??
```

and then if it is tested on an array of both positive and negative numbers:

```
?? PythonTeX ??
```

this returns **?? PythonTeX ??**. How this works is as follows. There are two values associated within reduce from the function. The first is `n` and the second is `val`.

1. the variable `n` is initially 0 (this is the `init=0` part of the function call)
2. On the first step, `val` takes on the first value in the array (or `-3`). It is checked if it is positive and if so, return `n+1` or `n`. Since n=0 and `-3` is not positive, then the function returns 0.
3. On the second step, `val` is 5 and this time the function returns `n+1` or 1
4. `val` is 8 and the function returns `n+1` or 2
5. `val` is -2 and the function returns `n` or 2
6. `val` is 11 and the function returns `n+1` or 3.
7. Since the array has been passed through, the result is the last value or 3.

7.2.1 Exercise

- Write a reduce function that will count the number of times the string "hi" appears in an array. Test it on **??** and other arrays of strings.
- What does **??** do?

7.3 The `mapreduce` function

The `mapreduce` function is perhaps more helpful than `reduce`. For example, if we want to sum the squares of each number in an array, then `mapreduce` can do this easily.

```
?? PythonTeX ??
```

is a short-hand way to do $1^2 + 2^2 + 3^2 = 14$. Note: julia also has a version of the `sum` function that can apply a function. For example,

```
?? PythonTeX ??
```

also returns **?? PythonTeX ??**.

7.3.1 Exercise

In calculus, an important infinite series is

$$1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16} + \cdots$$

and although we can't sum an infinite number of terms, a finite version of this is still useful.

Use `mapreduce` to sum the first 20 terms. Hint: use **??**.

7.4 Mapping a Function over an 2D array

Above, we saw the `map` function which applies a function over each element of the array. We will see in Chapter 13 that the `mapslices` functions is quite helpful.

Consider the array **??** which returns **?? PythonTeX ??**

If we want to sum down the columns of the array, we can enter

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**, which is a 1D array with the column sums. The `mapslices` function needs three arguments, a function that can take an array, a 2D array and the keyword `dims` which says how to apply the sum. Note: we can also replace the command `sum` with `+` as well.

If instead we wanted to sum along rows, then

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**which is a column vector. Although note that this is a 2D array. The function `mapslices` always returns an array of the same number of dimensions as the original.

7.4.1 Exercises

Using the matrix **??**,

- evaluate **??**. What does that function do?
- evaluate **??**. What does that function do?
- use the `mapslices` function to find the maximum element in each row.
- use the `mapslices` function to find the maximum element in each column.

7.5 Writing fast code

The last part of this chapter will show one aspect of writing fast code, an important aspect of scientific computing. We will study a number of ways to sum up the first n counting numbers and determine why the timing of things are different. Also, we are going to test the various types of integers as well to determine the speed of things with that. In each case we are going to create a function and then time it.

Attempt #1 Let's start by making an array of all of the numbers, then summing them in a for loop:

```
?? PythonTeX ??
```

On a relatively slow machine:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**.

The `@time` is called a macro, which is similar to a function, however is more flexible. In this case, it calculates the time it takes, number of allocations and memory allocated. The macro then returns the value of the function

If we repeat this for the sum of the first billion counting numbers:

```
?? PythonTeX ??
```

it takes a while: **?? PythonTeX ??**.

The big difference between these is the total memory allocation. There is a 1000-fold increase in memory allocated (note the MiB or megabytes versus GiB or gigabytes). This also shows about a 2000-fold increase in time.

Allocating memory is a time-expensive endeavor. Also, even though the machine that I ran this on has 16 gigabytes, perhaps it couldn't get all 7.5 GiB it needed for this operation at once. This is shown with the 0.05% gc time, which mean garbage collection. In short, there was some time needed to handling so much memory.

If you have some patience, try this will 2 billion or more and see the results. You will probably need more allocation time and should be at least twice as long.

Attempt #2: The big difference seemed to be the total memory allocation, so let's try a version where we don't allocate the array.

```
?? PythonTeX ??
```

and running it as **??** we get: **?? PythonTeX ??**and notice that the time has shrunk to zero–we'll see later why this is true. If we try for larger numbers, such as **??**, we get: and results in **?? PythonTeX ??**which still takes almost no time, but notice that that the result is not correct. Why? Consider the ideas from Chapter 3.

Attempt #3: Hopefully you thought about the strange result above. If you thought overflow, give yourself a gold star. To avoid this, let's write a `BigInt` version of this:

```
?? PythonTeX ??
```

and:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

Comparing this with the results at the beginning of Attempt #1, shows that BigInts are much slower as mentioned in Chapter 3.

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**which is about 10 times slower, so it appears that this is linear as possibly as expected. This isn't practical to find much larger sums though.

Attempt #4: Let's trying using the `reduce` function on BigInts:

```
?? PythonTeX ??
```

Then:

```
?? PythonTeX ??
```

returns: **?? PythonTeX ??**Notice that this allocates about 91 MiB of memory, which isn't a lot, but this is probably due to the `BigInt` allocations. Also, this is about twice the amount of time as that in Attempt #3. Notice also a significant amount of gc (garbage collection) time.

Also,

```
?? PythonTeX ??
```

returns: **?? PythonTeX ??**which is about 10 times slower that the previous one, and still 1.6 times the speed of `sum3` for the same number `n`.

Attempt #5: Let's use the built-in `sum` command. Entering:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**and note that higher powers of 10 do not increase the time much. For example,

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**What is going on? How is this so fast? Think about this and we'll answer it below.

7.5.1 Exercise

Repeat Attempts #1–#4 for `Int128`. That is, make sure that the calculations are being done using this type. For the `for` loops, start with

```
?? PythonTeX ??
```

and for the `reduce` function use **??** instead of **??**.

Compare your results with those above.

7.6 Summary of Results

There were a lot of factors going on in the above example. Here's a summary.

- ***Allocating an array is expensive (in terms of memory and time).*** In Attempt #1, we created an array using the `collect` function. The creation was why this took so much time.

- ***BigInts and `Int128` are slower than `Int64`s.*** We noticed that switching from `Int64` to `BigInt` in Attempt #2 to #3, there was a significant drop in speed. In short, `BigInt`s are slow. Only use them when needed.

  You should have noticed from the exercise that `Int128` is a viable alternative to `Int64` if you need larger integers. `Int128` is slower than `Int64`, but still much better than `BigInt`. Only use `BigInt` when you need really large integers.

- ***Sometimes julia is super smart about some operations.*** In both Attempt #2 and Attempt #5, we got much shorter times than expected. In both, you would expect summing 1000 times more numbers would take 1000 times longer, but this isn't true. In both cases, julia recognizes that integers are begin summed and is probably using the formula

  $$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

  and using this formula can be done for any number n with no summing at all.

7.7 Computing Fibonacci Numbers

In section 4.9, there was an exercise to use recursion to find the fibonacci numbers. A possible solution to this is:

```
?? PythonTeX ??
```

where we have used the ternary `if-then-else`. Note: if `n` is 1 or 2, then 1 is returned, if not it uses the recursive formula. The first 10 fibonacci numbers are found with

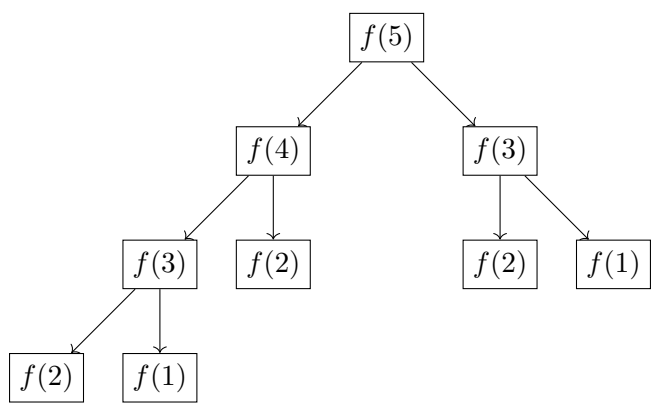
```
?? PythonTeX ??
```

and this results in **?? PythonTeX ??**.

It is fast to find the fibonacci numbers for smaller values, but consider

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**and if we find **??**, it is about twice as long. To consider what happens, let's look at **??**. Inside the function, it calls **??** and **??**. Each of these then called the previous two. This can be seen in the following tree graph where $f(n)$ is the fibonacci function:



Since each of the endpoints (either $f(2)$ or $f(1)$ requires and evaluation as does each arrow, there are a total of 13 evaluations for this. If we define:

```
?? PythonTeX ??
```

then

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**. Also

```
?? PythonTeX ??
```

returns **?? PythonTeX ??** and

```
?? PythonTeX ??
```

returns **?? PythonTeX ??** and notice that finding the 21st fibonacci number takes about 60% more operations and therefore would take about this much longer.

Thinking about this result shows that we aren't doing things efficiently. If we have already calculated the 19th and 20th fibonacci numbers, why does it take an extra 60% longer? Basically this is because we aren't saving the results to be reused.

7.7.1 A still-recursive, but much faster version of Fibonacci

As we saw above, the standard recursive version of `fibonacci` function is very slow.[2] We now attempt to find a faster version. Instead of calculating every previous fibonacci number, we will use an array and if the value is already calculated, just look it up in the array or if not, calculate it recursively. Consider

```
?? PythonTeX ??
```

and try finding fibonacci numbers with this instead. In fact, **??** returns **?? PythonTeX ??**whis is a fraction of the time it took to do the 40th fibonacci number with the original function. The exercise below explores the `fibonacci2` function.

Note that in the `fibonacci2` function, we use arrays to store all of the fibonacci numbers. We will see arrays in Chapter 6, but the point of this example is to show that 1) recursion is often a very nice and compact way to write a function, and 2) can be highly inefficient. We realized that we were never reusing the numbers that were being computed and the best way to store these were in an array.

Also, you probably noticed that we had a function called `fib` inside the `fibonacci2` function. We did this because `fib` used the array created in `fibonacci2`. There were other ways to do this, but this is a very nice example of a function within a function.

7.7.2 Exercise

1. Determine which fibonacci number results in an overflow for `Int64`. Do this by trial and error with the `fibonacci2` function.

2. Rewrite the `fibonacci2` function to return a `BigInt` version if n is greater than or equal to the number in #1.

3. Find the first 200 fibonacci numbers.

[2] in fact, it is $O(e^n)$ REALLY? as we will see.

Chapter 8

Number theory and Algorithm Development

This chapter investigates some aspects of Number Theory, which is the study of the integers. Prime numbers are one of the most important aspects of Number Theory. We will use this topic as an example of scientific computing. We will look at examples of computational number theory throughout this chapter and then spend some time investigating the efficiency of the code here.

8.1 Prime Numbers

Recall that a prime number is an integer greater than 1 whose only factors are 1 and itself. Most languages have built-in versions of testing for primes, and julia has a package for this, and we will see it toward the end of this chapter. However, we will start by writing our own versions.

Based on the definition, if we can find the factors of an integer, then we can use it to determine if the number is prime. The following function returns all of the factors of a given integer:

```
?? PythonTeX ??
```

This function starts with an array (list) consisting of only the number 1 (line 2). It then checks all numbers between 1 and n to see if each is a factor (the `mod(n,i)==0` on line 4). If it finds a factor, add it to the list with the `push` function.[1]

8.1.1 Exercise

- Type in the function above and test it on various prime and non-prime (composite) numbers.

[1] If you read carefully in Chapter 7, there was an emphasis on not using loops and using other functional-programming ideas instead. You may ask yourself, why didn't we use a `map` or `reduce` function. The answer is that in this case, an integer is passed in and an array is returned. This doesn't fit either of these. You could create an array of `collect(1:n)` to create an array, but the resulting array is smaller in sized. The `map` command always returns in an array of the same size. In short, a loop is needed for this.

- Use the function above to write a function `isPrime` that returns true or false depending on if the input is prime. Hint: recall the definition of prime and use that with the `findAllFactors` function to determine primeness.

  The template for the function should be

  ```
  ?? PythonTeX ??
  ```

8.2 Finding Primes

We'll now try to find a prime number. Many languages have a function called `nextPrime` that takes a number as an input and returns a prime number greater than that. Before presenting the code, here's some things to think about:

- What is the template for the function. We want a function that takes in an integer and returns an integer. Here's a good template for that:

  ```
  ?? PythonTeX ??
  ```

- We will need to test if `n+1` is prime (that the smallest possible one).
- If `n+1` is prime, return it, if not check the next one.
- One way to approach this is a loop (probably a while loop because we don't know how many step to do before finding a prime), but this is another great example of a recursive function.

Here's the result:

```
?? PythonTeX ??
```

where again, we have used a ternary `if-then-else`. This first checks if `n+1` is prime. If it is return it, if not call `nextPrime` on the next integer.

8.2.1 Exercise

- Test this function on some numbers that are less than 100 to see if it is working.
- Find the smallest prime less than 1 million.
- Write a function that produces the first `n` prime numbers. The template for this will look like:

  ```
  ?? PythonTeX ??
  ```

8.3 Perfect Numbers

A **perfect number** is an integer, `n` which has the sum of the factors (except itself) equal to `n`. For example, 28 is a perfect number because the factors are 1,2,4,7,14,28. The sum of the factors (except itself) is 1+2+4+7+14=28 A nice function would be to check if a number is perfect. Here's a shell:

```
?? PythonTeX ??
```

The following will go through the steps to look at this. And we will use the function `findAllFactors` that we wrote above.

First, entering **??** returns the array **?? PythonTeX ??**. To use the definition of a perfect number, we need to exclude the last element of the array above. One way to do this is to use the `pop!` function. If we define

```
?? PythonTeX ??
```

then the result is **?? PythonTeX ??**. Then we can test if the sum is 28 by **??**.

We can put all of this together as the function.

```
?? PythonTeX ??
```

We should also test this. We know that 6 and 28 are perfect, so **??** returns **?? PythonTeX ??** and **??** returns **?? PythonTeX ??**. Many other integers are not perfect, so **??** returns **?? PythonTeX ??** and **??** returns **?? PythonTeX ??**.

Although there is nothing wrong with the above function, we can shorten it a bit by not removing the last element of the array. Note that if we sum all of the factors of a perfect number n, the sum will be $2n$. The following is alternative way to do this.

```
?? PythonTeX ??
```

Note that in the function at the end is `2n`, which has a implicit multiply in there. If you multiply any variable on the left with a numerical literal (that is a number, not a variable that is a number), then multiply is implied.

8.3.1 Timing the two functions and the `BenchmarkTools` package

An objective way to determine which of the two functions is better is to test if one is faster. Let's test if 10,000 is perfect or not. Using `@time` on both of these:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**Running:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**These both seem to return about the same time, but trying running each of these a number of times. You'll notice that since the time is fairly short, the results bounce around quite a time. This is common for functions that don't take much time.

We'll examine the `BenchmarkTools` package. Recall that you need to add it using the package manager decribed in Appendix B. Once you have added the package, we'll need to tell julia that we're using it by

```
?? PythonTeX ??
```

And this package has the macro `@btime`, which is much like the `@time` macro, however is much more accurate in that it runs the code a few times, reporting the average results.

```
?? PythonTeX ??
```

returns (and you'll notice it takes a while) **?? PythonTeX ??**and

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**

Note that the μs is microseconds where 1 μs is $1/10^6$ seconds. If you rerun each of these a few times, you'll get consistent results and times. That's the point of the package `BenchmarkTools`. It runs the code many times and returns an average time.

About the results, since both versions of the `isPerfect` function are nearly identical, you may want to check a wider range of numbers (try this for numbers in the millions) to see if there is a difference.

8.4 Happy Numbers

Here's another fun example. A number n is called ***happy*** if you perform the following steps:

1. Take the digits of n and square each one.
2. Sum the squares.
3. If the sum is 1, then the number is happy. If not repeat these steps.

Note: it's also helpful that if this process results in the number 4, then you can never result in a sequence that reaches 1. You can call these number ***unhappy***.

Here's some examples:

- 13 is happy because $1^2 + 3^2 = 10$ and $1^2 + 0^2 = 1$, so the result ends in 1.
- The number 19 is also a happy number because $1^2 + 9^2 = 1 + 81 = 82$, then $8^2 + 2^2 = 64 + 4 = 68$, then $6^2 + 8^2 = 36 + 64 = 100$, then $1^2 + 0^2 + 0^2 = 1$.
- The number 4 isn't happy because
  - $4^2 = 16$, then
  - $1^2 + 6^2 = 1 + 36 = 37$, then
  - $3^2 + 7^2 = 9 + 49 = 58$, then
  - $5^2 + 8^2 = 25 + 64 = 89$, then
  - $8^2 + 9^2 = 64 + 81 = 145$, then
  - $1^2 + 4^2 + 5^2 = 1 + 16 + 25 = 42$, then
  - $4^2 + 2^2 = 16 + 4 = 20$, then
  - $2^2 + 0^2 = 4$

  and since we have returned to 4, this will continue cycling, so we stop and say 4 is ***unhappy***. Any number in this sequence is unhappy as well. (It has been proven that any unhappy number will eventually hit this cycle.)

8.4.1 Computing Happy Numbers in Julia

Julia has a built-in function to take a number and split the digits called `digits`. For example **??** returns **?? PythonTeX ??**, where the position in the array is the digit in the corresponding place in the number. For example, the 4 in the first slot in the array is the ones digit and the 1 is the thousands digit (since it is element 4 of the array).

Next, let's write a function called `isHappy` which determines if a number `n` is happy. The function should be recursive and return true if it is 1, return false if it is 4 and do the steps above otherwise. The template for the function should be

```
?? PythonTeX ??
```

Our first attempt will be with a loop. We need to go through the digits, square the numbers and find the sum. We can do this as

```
?? PythonTeX ??
```

Here's a few things to note:

- Since we want a recursive function we call **??** inside the function.
- Also since it is recursive, we need to make sure that there is a way to stop the recursion. This occurs in the first two parts of the if statement.
- The for loop between lines 9 and 11 sum the squares of each of the digits.
- the `return isHappy(sum)` statement on line 12 calls the function recursively on the sum of the digits.

8.4.2 Exercise

Try running the `isHappy` code on many positive integers.

8.4.3 Alternative forms

We can shorten the last block of code into the following:

```
?? PythonTeX ??
```

where instead of using a for loop, we sum the squares of an array using the `sum` command.

Lastly, we can write this a bit shorter again using the ternary `if-then-else` (but in this case, it is nested as

```
?? PythonTeX ??
```

and unless you're strange (like me) and like such if-then-else statements, this is much less readable.

8.4.4 Exercise

Test the three versions of `isHappy` presented here using the macro `@btime` from the package `BenchmarkTools` to see if there is a difference in speed.

8.4.5 Finding Happy numbers

One way to find happy numbers then is check a range of integers using the function you wrote above. For example, you can type

```
?? PythonTeX ??
```

which will return an array of true/false if the number is happy or not. Try running it.

This isn't so helpful. If your look down through the list, it's hard to tell which number are happy and which are not. A better way to do this is to use julia's built-in function `filter` that we saw in Chapter 6. For example,

```
?? PythonTeX ??
```

returns all integers between 1 and 100 that are happy or

?? PythonTeX ??

We saw the `filter` command with an array, but you can also use it with a range as shown above.

8.5 Primes can be big!!

If you do a quick google search on the largest prime number known, you'll see that primes are big. As of early 2020, the largest prime had 24,862,048 digits. Let's not go that big.

The number $2^{89} - 1$ is prime. However, if we type **??** into julia, we get `-1`. Does this imply that $2^{89} = 0$? Recall that the problem with this is that 2^{89} does not fit into `Int64` and we are getting an overflow error without being informed. You would need at least 89 bits to store this number, so it could be stored as a `Int128`.

As we saw in an earlier chapter, we can use `BigInt` so

```
?? PythonTeX ??
```

will give **?? PythonTeX ??**. However, don't throw this at your `isPrime` function because it will not complete this because we have used a fairly bad algorithm for finding factors or testing for primality.

8.6 Speeding up `isPrime`

Let's time the `isPrime` function that you wrote above on a reasonable-sized number. With

```
?? PythonTeX ??
```

and the results are **?? PythonTeX ??**Although this may not seem slow, 1 million is small if you are studying prime numbers and as we saw in the previous section, they get large fast. The reason this is slower is because `isPrime` relies on the `findAllFactors` function which checks every every number between 2 and $n - 1$ to see if it is a factor. This is very inefficient, so we will try some ways to speed things up.

8.6.1 Speedup #1: Rewrite `findAllFactors` to stop earlier

In the function `findAllFactors` we go to $n - 1$ to check for a factor. There's actually no reason to do this. For example, consider $n = 100$, the factors are (and you can check this) are $1, 2, 4, 5, 10, 20, 25, 50, 100$. The second largest factor is at most half of the value n. Let's consider then the following

```
?? PythonTeX ??
```

where `div(n,2)` is integer division. You will see in an exercise that doing `n/2` results in slowing down the function. This is because doing `n/2` results in a floating point, and all operations with floating points are slower than integers.

To show that this works:

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**whereas

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**and this is a significant speedup. We will also a new version of `isPrime` to take advantage of this new factors algorithm.

```
?? PythonTeX ??
```

If we run **??** we get **?? PythonTeX ??**which runs in about 50% faster than our original `isPrime` function.

8.6.2 Speedup #2: Rewrite `findAllFactors` to not check all integers as factors

Notice that in both of the cases, we go through of the integers up to $n-1$ or $n \div 2$, whereas that isn't necessary because most of the time, factors come in pairs. Consider $n = 200$, We can write the factors in the following way:

$$\begin{array}{cccccc} 1 & 2 & 4 & 5 & 8 & 10 \\ 200 & 100 & 50 & 40 & 25 & 20 \end{array}$$

and there are pairs of factors and unless the number is a perfect square, they always occur in pairs.(Check $n = 100$ as an example). We will go through integers, but can stop at $\sqrt{n}$. The following will compute this:

```
?? PythonTeX ??
```

and notice that if we try to evaluate time this:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

Since we really want to test the speed of an `isPrime` function, let's write:

```
?? PythonTeX ??
```

and

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

and to compare to the original that we did above, this is about 1000 times faster and about 700 times faster than version 2.

8.6.3 Speedup #3: Don't use `findAllFactors` at all

If we are only interested with determining if a number is prime or not, we don't really need to calculate all of the factors. Let's start with the `findAllFactors3` function, which seems fast, but we will get rid of the array. Consider the following:

```
?? PythonTeX ??
```

and

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

Note that this is just slightly faster that `isPrime3`. It may be that the number of operations is comparable, just done in a slightly diffrent way.

8.6.4 Speedup #4: Don't do unnecessary calculations

One thing I noticed while writing this function is that the for loops goes through all numbers up to $\sqrt{n}$. A quick way to eliminate many of these is to check 2 then all odds between 3 and $\sqrt{n}$. This is because if 2 isn't a factor of n, then any even cannot be either. Here's a way to do that.

```
?? PythonTeX ??
```

and checking the time on this

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**which is about twice as fast as the previous one. This makes sense in that one is eliminating almost all of the even numbers.

8.6.5 Speedup #5: check only primes as possible factors

We can continue the thought of the previous algorithm in that the only factors we need to check are prime numbers.

However, there's a bit of circular reasoning here that we are checking to see if a number is prime and we would use primes to check this, but here goes.

An exercise above asked to generated a list of primes up to some number. The Sieve of Eratosthenes is one that generates a list of primes up to a number m in the following way.

1. Write down all numbers between 2 and m.
2. Start with 2 and cross out all multiples of 2.

3. The next number greater than 2 not crossed out is 3. Cross out all multiples of 3.
4. The next number greater than 3 not crossed out is 5.Cross out all multiples of 5.
5. Continue this process until you reach $\sqrt{n}$.
6. The resulting list are all prime.

See the animation on the wikipedia page linked above for a nice visual.
Here's a julia version of this algorithm:

```
?? PythonTeX ??
```

and for example, **??** returns **?? PythonTeX ??**

Now, we can adapt the `isPrime` code to use only primes in the following way:

```
?? PythonTeX ??
```

Lastly, let's just test the speed of this function:

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**Surprisingly, this is actually slower than `isPrime5` by about a factor of 2. This shows that you that if speed is important that you always need to test.[2]

Noting that the allocations (about 13 kB) is significant whereas the previous algorithm had virtually no allocations, this probably results in the difference in speed.

8.7 Summary of `isPrime` functions

Since this was all a lot to take in, let's summarize all of these functions. We'll stick with **??**, which is a prime number

command	time
??	**?? PythonTeX ??**
??	**?? PythonTeX ??**
??	**?? PythonTeX ??**
??	**?? PythonTeX ??**
??	**?? PythonTeX ??**
??	**?? PythonTeX ??**

It appears that `isPrime5` is the fastest of the algorithms we developed here. Note that this just checks for a single number, whereas, the values may changes as one increases the value of n.

[2] I decided to leave this last example in. My expectations were just off but that is why we test. In fact after the first results, I figured it would pay off for larger numbers. It didn't appear that way.

8.8 Testing for Primes using the `Primes` library

Load the `Primes` library (and if you haven't added it yet, do so):

```
?? PythonTeX ??
```

The `Primes` library has the `isprime` function to test for primality. Let's test a few examples.

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**and this is about twice as fast as our algorithm, `isPrime5`. Note that the time is given in ns or nanoseconds and 1000 nanoseconds is 1 μs.

Note that the `isprime` function is quite fast. If we put in $2^{89} - 1$, it still finds the answer quite quickly.

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

Chapter 9

Solving Quadratics and Rootfinding in General

A **root** of an equation is a number $x^\star$, such that $f(x^\star) = 0$. For example, if $f(x) = x^2 - 4$, then both $x = 2$ and $x = -2$ are roots of the equation. Finding roots of functions is often impossible analytically, but quite important. In this chapter, we discuss how to find roots of both quadratic functions and general functions, but also use this to discuss both errors in a algorithm as well as round off error, which occurs often when solving problems.

9.1 Absolute and Relative Errors

Consider an algorithm that tries to find the value of $x^\star$. If the algorithm actually returns the value x, then there will be some error. The **absolute error** is

$$|x - x^\star|$$

and the **relative error** is

$$\left|\frac{x - x^\star}{x^\star}\right|.$$

Often, the **percent error** is helpful as well, which is just the relative error times 100.

9.1.1 Example

Consider an algorithm that returns $x = 0.0153$ and the actual answer is $x^\star = 0.0150$. Find both the absolute and relative errors.

The absolute error is $|0.0153 - 0.015| = 0.0003$ and the relative error is

$$\left|\frac{0.0153 - 0.015}{0.015}\right| = 0.02$$

or 2%.

We can do this is julia with

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??** and

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**.

To make thing easier in this chapter, we'll create functions for these:

```
?? PythonTeX ??
```

which now allows us to use **??** and **??** and we get the same results as above.

9.2 Rounding Errors and the Quadratic Formula

The following example shows how a well-know formula (the quadratic formula) can lead to incorrect results due to rounding errors.

Remember if you solve $ax^2+bx+c=0$, the the quadratic formula finds the solution:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Next, let's consider the quadratic equation

$$12.242x^2 + 42.382x + 0.0012 = 0. \tag{9.1}$$

We are going to solve this using the quadratic equation in julia. To study rounding effects we're going to solve these using both 16-bit and 64-bit floating point numbers. We'll assume that the 64-numbers are the actual answers in determining errors.

First, consider the quadratic formula as a julia function:

```
?? PythonTeX ??
```

We can solve the quadratic in (9.1) with

```
?? PythonTeX ??
```

leads to **?? PythonTeX ??**. Note that since the default floating-point type is `Float64`, these are 64-bit results.

To solve this using 16-bit floating-point numbers,

```
?? PythonTeX ??
```

leads to **?? PythonTeX ??**.

Let's assume that the 64-bit answers are correct. Find the absolute and relative errors for the two solutions.

The absolute errors are:

- **??** = **?? PythonTeX ??**.
- **??** = **?? PythonTeX ??**.

and the relative errors are:

- **??** = **?? PythonTeX ??**.
- **??** = **?? PythonTeX ??**.

The relative error 1 (or 100%) means that the answer is way off. The second answer is quite close though.

9.2.1 Exercise

Use the quadratic formula to solve $2.56632x^2 + 17.34x + 0.01734 = 0$ using both BigFloats and 64-bit floats. Assume that the BigFloat versions are exact and find the absolute and relative errors in the 64-bit numbers.

9.2.2 Be skeptical of numerical answers

Even though the example above was manufactured to produce bad results, you never know if a numerical solution is accurate or not. One way to look at the above example is to not use 16-bit floating point numbers. This is a good idea, but assume (like we did above) that 64-bit is the actual answer. There are cases where you will get the wrong answer with 64-bit.

9.2.3 Can we ever be confident of answers?

It's crucial to have a good sense of the problem (whether it be mathematical or a scientific question). Additionally, testing is an important part of the solution process. In Chapter 16, we will spend some time discussing testing of code.

9.3 Are We Sunk? Examining the Quadratic Formula

The above example shows that we had quite a large error. From what we've seen so far, we could try to use other data types, like `BigFloat` to solve this problem, but as we've seen they are slow and we should only use that type when needed. So what can we do.

If we're careful about things, we can often rewrite algorithms to do a better job with roundoff. The problem that occurred for the general quadratic equation

$$ax^2 + bx + c = 0$$

is if $d = \sqrt{b^2 - 4ac}$ and when $b > 0$, the formula has $-b+d$, which is basically subtraction. In the case above, $b = 42.382$ and $d = \sqrt{b^2 - 4ac} = 42.38130675663505$. The difference in these is quite small and that is where the round-off error is introduced.

Let's assume that $b > 0$ and let $d = \sqrt{b^2 - 4ac}$. The roundoff occurs when the value of b and d are close to each other. (Note: if b is negative, you can multiply the entire equation through by -1 to get $b > 0$ without changing the answer.) To change the quadratic formula we are going to exchange the addition with a subtraction (however we will not have the catastrophic subtraction error we saw above). "How can you do that you ask?" Here we go...

Start with the quadratic formula for the $+$ case:

$$\begin{aligned}
x &= \frac{-b + \sqrt{b^2 - 4ac}}{2a} && \text{Multiply by a convenient form of 1} \\
&= \frac{-b + \sqrt{b^2 - 4ac}}{2a} \cdot \frac{-b - \sqrt{b^2 - 4ac}}{-b - \sqrt{b^2 - 4ac}} \\
&= \frac{b^2 - (b^2 - 4ac)}{2a(-b - \sqrt{b^2 - 4ac})} \\
&= \frac{4ac}{2a(-b - \sqrt{b^2 - 4ac})} \\
&= -\frac{2c}{b + \sqrt{b^2 - 4ac}}
\end{aligned}$$

and there is a similar solution if the $-$ root is taken above.Note that we have exchanged a subtraction for an addition if $b > 0$. So an alternative quadratic formula is:

```
?? PythonTeX ??
```

9.3.1 Exercise

- Use the new quadratic formula (`qSol2`) to find the roots of $x^2 - x - 110 = 0$. Do you get the same results as above?

- Use the new quadratic formula to solve $12.242x^2 + 42.382x + 0.0012 = 0$. Find the absolute and relative errors if we assume that the 64-bit answers are exact and the 16-bit answers are approximate. Compare your results with that of the standard quadratic formula.

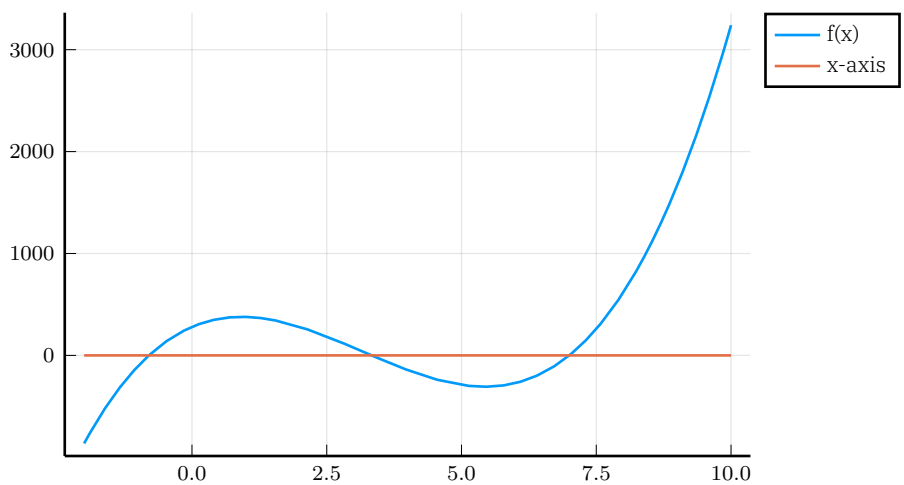
9.4 Newton's Method

Solving an equation is a very important part of mathematics and other scientific fields. However, finding general solutions to equations is not very easy. Consider a cubic equation like $x^3 - 10x^2 + 24x - 17$. In the spirit of (but not very simple) the quadratic formula, there is a cubic formula. Much of the wikipedia page spends time solving the cubic with all possibilities. In short, it's not very easy.

In lieu of using such a formula, a more robust approach is to solve it numerically. Let's consider

$$15x^3 - 143x^2 + 226x + 280 = 0$$

This cubic actually factors, but finding those factors is quite difficult to do in general. We then look at using Newton's method to solve this. Note the plot below:



and the three intersection points between the red curve (x-axis) and the blue line (the function, $y = f(x)$) are the three roots.

Newton's method starts with a "guess" at the root and then refines it. Let x_0 be the guess, then

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

Let's look at this in Julia for the function above.

```
?? PythonTeX ??
```

and we need the derivative so

```
?? PythonTeX ??
```

Let's say that `x0=0` and find `x1` using Newton's method:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**. We then use the result and perform another step of Newton's method or

```
?? PythonTeX ??
```

this time we get **?? PythonTeX ??**, which is closer. A few more iterations of this, you should get quite close to `-0.8`.

9.4.1 Code for Newton's Method

We are going to go through building a function to do this. First, let's consider the template of the function, which will have an arguments `f, df` and `x0`. Thus a good template is

```
?? PythonTeX ??
```

Here are considerations while building the function:

- You will need to do the two steps above many times so you will need a loop. Since you don't know how many times you will need to run through the loop use a `while` loop and your condition should be that the two steps `x0` and `x1` are apart from each other.
- Checking if two floating-point numbers are equal are generally not a good idea, because they have to be equal to all bits, so instead we will run the while loop while the two number `x0` and `x1` are larger than some default (like 10^{-4}).

Here's more a frame of the function:

```
?? PythonTeX ??
```

Using this we can now call this function as

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**.

Just to clean things up a bit we can write this as:

```
?? PythonTeX ??
```

9.4.2 Using Automatic Differentiation

If you have used Computational Algebra Systems like Maple or Mathematica, you know that computers have the ability to differentiate. Julia is not one of these, although it has some capability of doing some of the feature set of these programs. There is a system called **automatic differentiation** that will compute the exact derivative to a function at a given point. That is, if you have a function $f(x)$ and a number a, it will give you $f'(a)$. The package is called `ForwardDiff` and you may need to add it and then

```
?? PythonTeX ??
```

For example, if you define:

```
?? PythonTeX ??
```

to find $g'(3)$, type

```
?? PythonTeX ??
```

which will return **?? PythonTeX ??**. Try something much more complicated:

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**, and if you are careful with the derivative (or have access to a CAS), see if your answer is correct.

9.4.3 Newton's Method with Automatic Differentiation

With this nice package, let's rewrite Newton's method

```
?? PythonTeX ??
```

And note that if we run this with **??** we get **?? PythonTeX ??**

9.4.4 Exercise

1. Find the intersection point of the functions $y = x$ and $y = \sin x$. Hint set the two equations equal and use algebra to write it as a function $g(x)$ which is equal to 0.

2. Use your function above to find all three of the roots of the cubic function, $f(x) = 15x^3 - 143x^2 + 226x + 280$.

9.4.5 Adding an extra stopping condition

One problem that can arise is if you call `newton` on a function that does not have a root. For example, consider $f(x) = x^2 + 1$, which is always positive. If you run

```
?? PythonTeX ??
```

then you reach an infinite loop. You'll need to interrupt the process.

To prevent this, we need to add a stopping condition if this occurs. One way to do this is to add a counter and then a `return` statement if the counter hits some maximum steps. Here's a possible update to Newton's method:

```
?? PythonTeX ??
```

Now if we try:

```
?? PythonTeX ??
```

we get **?? PythonTeX ??**

Now, this still isn't perfect because when we run it, we only get a single number out. In this case, the value is not a root, but how do you know? We will see in Chapter 11 how to develop a new type which stores information about the root and if a solution was reached.

Chapter 10

Plotting Data and Functions

Plotting is crucial to understanding functions and visualizing data. There are many ways to do some plots using various packages. Instead of covering details of multiple packages, we will cover the Plots package, which is overlay over many other plotting packages.

10.1 The Plots Package

There is a relatively simple, but powerful plotting package called `Plots` and don't forget to download it as in Appendix B. The full documentation is at the Plots.jl website. Recall that once the package is added, enter **??**

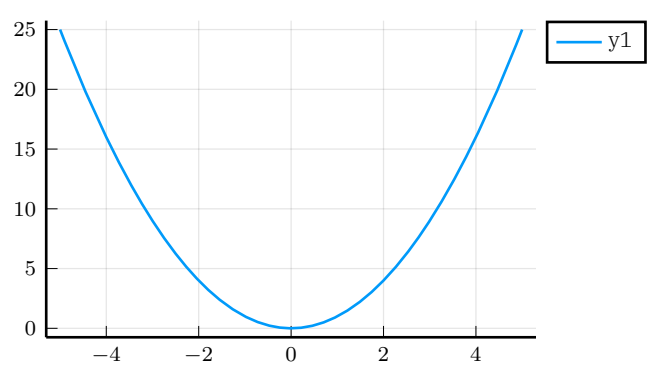
The Plots package tries to unify the syntax for plotting anything. The basic command for plotting data or functions in 2D is the `plot` command. The idea call `plot` on any type of object that can be plotted. The next few examples shows this.

10.1.1 Plotting Functions

For plotting a function, simply call plot on the function:

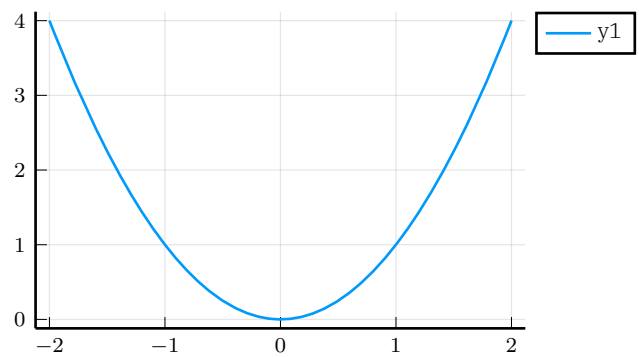
```
?? PythonTeX ??
```

produces the following plot:



Note: your plot may look a bit different than this one with different fonts. This is mainly due to using a different backend, which is explained below.

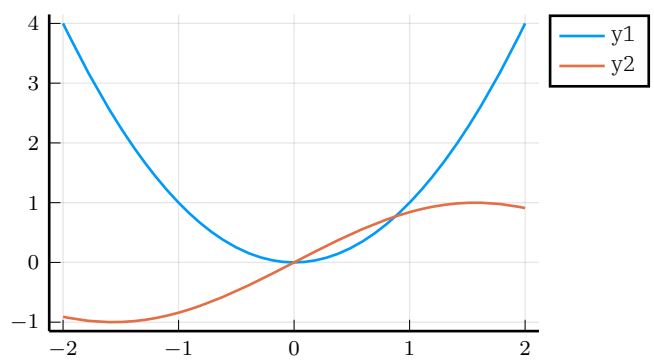
If you want to specify the x-range, try: **??** which generates:



If we want to plot 2 or more functions on the same axes, pass in an array of functions like:

```
?? PythonTeX ??
```

produces the following:



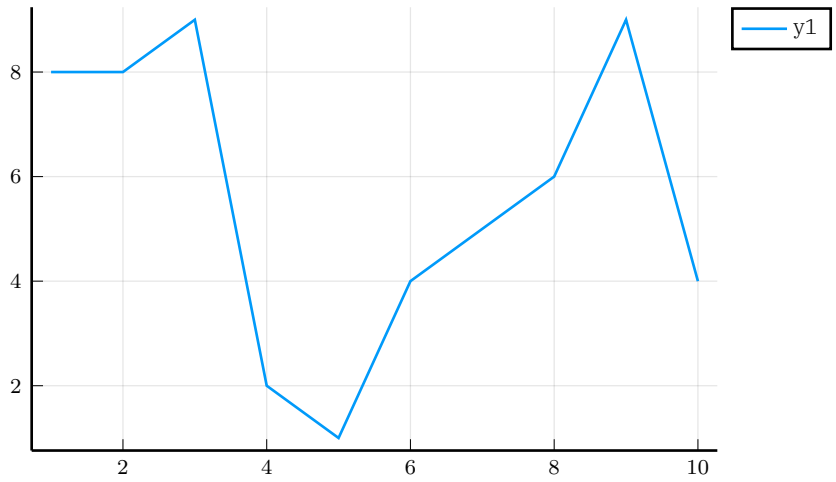
We will also see below how to change other aspects of the plot including the legend, title, labels on the axes, etc.

10.1.2 Plotting Data

First, let's start with some random data. Let

```
?? PythonTeX ??
```

then **??** will produce a scatter plot of the data, like

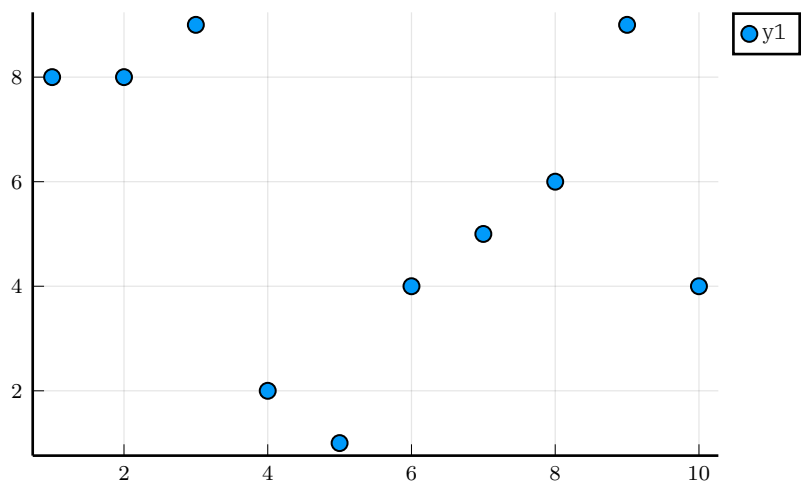


and note that since these are just random points, your plot will look different, but the style should be the same.

If we want to connect all of the points with points instead, type

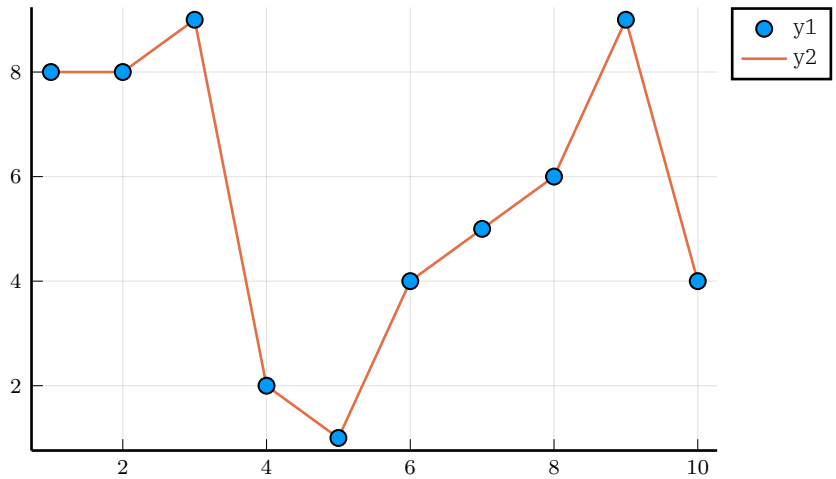
```
?? PythonTeX ??
```

and the plot will look like:



There is a shorthand or different version of this: **??**, which produces the same plot. If you want both, then type

```
?? PythonTeX ??
```



10.2 Backends of the Plots package

The `Plots.jl` package actually doesn't do the plotting. It leaves the details to other packages. By default, `Plots` uses the `GR` backend although in this text I have used the `PGFPlotsX` backend. There are a number of backends that you may want to try. The standard ones are:

- PyPlot (matplotlib): Slow but dependable
- GR: Feature-rich and fast, but new
- Plotly/PlotlyJS: Interactive and good for web
- PGFPlotsX: Native LaTeX rendering
- UnicodePlots: Plots to unicode for situations without graphics capabilities.

To switch the backend, you type the backend name in all lowercase with a set of (). Note: you will need to add and load the package. Here is the plot of x^2 on various backends.

```
?? PythonTeX ??
```

gives:

images/ch09/plot08.png

```
?? PythonTeX ??
```

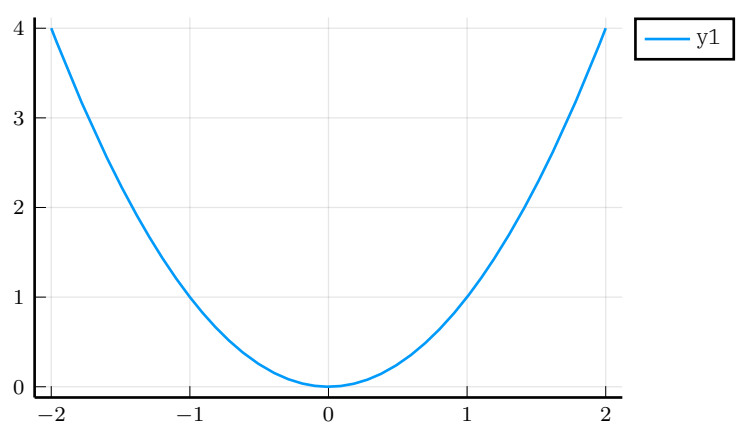
results in:

```
images/ch09/plot09.png
```

and then

```
?? PythonTeX ??
```

results in:



and finally

```
?? PythonTeX ??
```

results in:

images/ch09/plot11.png

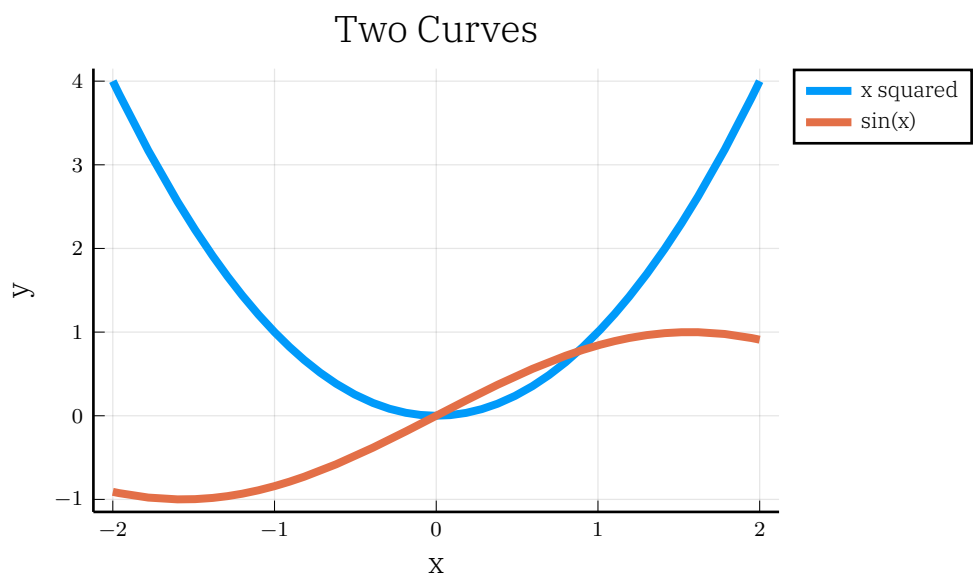
For additional information on the supported backends, visit the Plots.jl backend documentation

10.2.1 Changing the attributes of the plot

Let's return back to the function plots above (although this works for point/line plots as well) and change many attributes of the curve. As an example:

```
?? PythonTeX ??
```

results in



The format is fairly clear for the changing of the attributes. Note: in this example:

- the `title` changes the title of the plot
- the `label` changes the legend.
- the `xlabel` and `ylabel` changes the axes labels.
- the `lw` is the line weight.

To avoid duplicating tons of documentation, visit the Plots.jl page on attributes to find all of the information to get the plot the way you want.

10.2.2 Exercise

- Take the scatterplot above (with the random dots) and change the color of the dots to darkgreen, change the markers to diamonds and the size of the points to about twice the default size.
- On the function plot, make the line thicker and style to dashed.

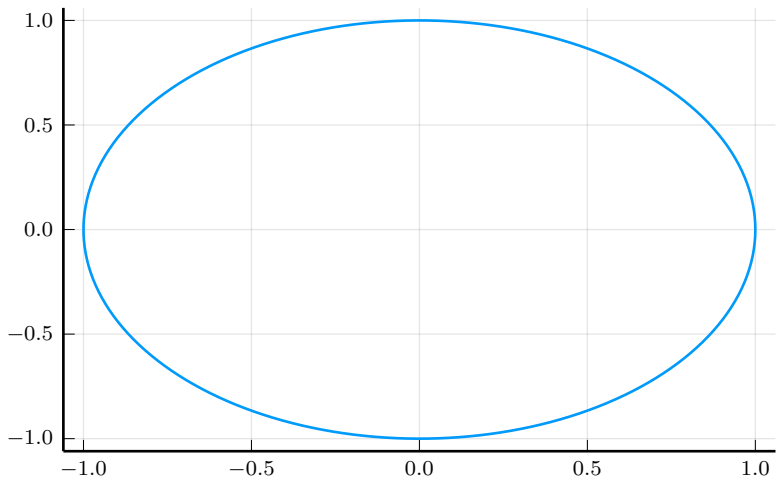
10.3 Other plots

10.3.1 Parametric Plots

To do a parametric plot, like the circle defined by $x(t) = \cos t, y(t) = \sin t$, then

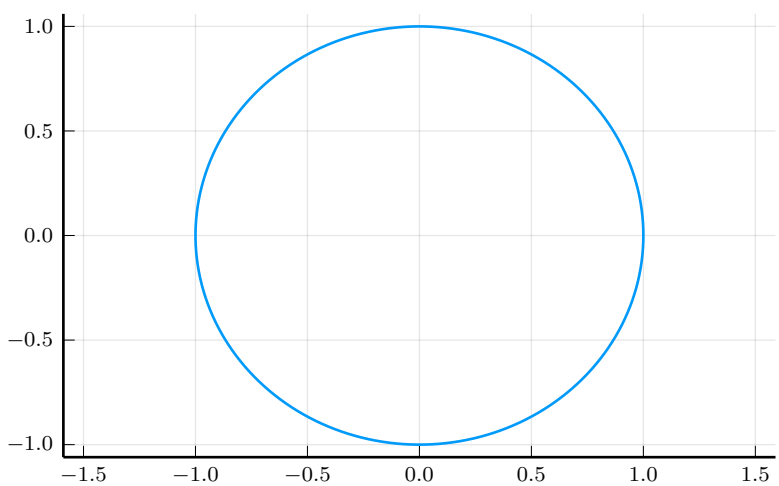
```
?? PythonTeX ??
```

where the legend is turned off, since with one curve, it doesn't make much sense. The result is



but notice that this should be a circle, but it looks like an ellipse due to the aspect ratio. If one instead adds the `aspect_ratio=:equal` option, as in

```
?? PythonTeX ??
```

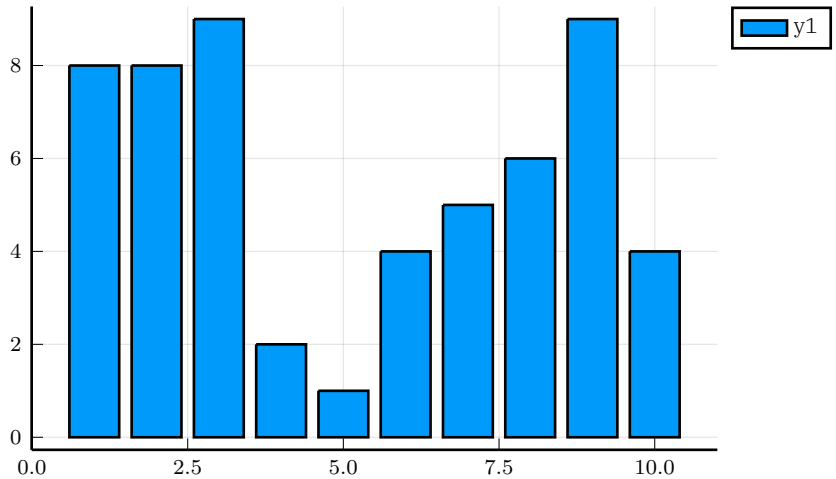


10.3.2 Bar plots

A bar plot can be made with the `bar` command. For example:

```
?? PythonTeX ??
```

where y was defined above for the scatter plot. The result is



10.3.3 Surface Plots

If we have a function of 2 variables, a surface plot is nice to use. For example, if we have the function $f(x, y) = 0.1e^{x^2+y^2}$ and we want to plot it from -3 to 3 in both directions, if we define

```
?? PythonTeX ??
```

and then plot with

```
?? PythonTeX ??
```

HIDDEN PLOT

10.3.4 Heat maps

Similar to above, we can make a heat map with

```
?? PythonTeX ??
```

which produces

HIDDEN PLOT

10.3.5 Animation

Another nice type of plot under Plots.jl is that of an animation, however you will need to have `ffmpeg` installed on your machine. If you then do

```
?? PythonTeX ??
```

which saves to a gif that the output will describe this results in:

10.4 Other Plots and subplots

This just is the tip of the iceberg for plotting. Take a look at the Plots.jl documentation or do some google-foo with the phrase 'Plots.jl' and what you're looking for and good spot for Q&A is a julialang.org discourse site

10.5 Exercises

Use the plotting techniques in this section to plot the following. For each, hide the legend when ther e is only one curve/set of data and label appropriate if more than one curve/set of data. Include a title as well.

1. The function $y = e^{-x^2}$ from $x = -3$ to $x = 3$
2. The functions $y = \sin x, y = \sin 2x, y = \sin 3$ from $x = -\pi$ to $x = \pi$.
3. A scatter plot of the following:

x	0	2	3	4	6	9	10	11	13	15	16	18	20
y	-1	4	3	6	2	0	2	9	5	-2	4	6	3

4. A surface plot of $z = \sin(x - y)$ with $0 \leq x \leq \pi, 0 \leq y \leq \pi$.
5. A heat map of $z = \sin(x - y)$ with $0 \leq x \leq \pi, 0 \leq y \leq \pi$.

Chapter 11

Creating New Data Types in Julia

Although there are many standard Julia data types that are useful for Scientific Computing, we can make new types called a composite data type. We will start with developing a `PlayingCard` type that simplifies the poker hand simulations that we will see in Chapter 14. We will also look at a `Polynomial` data type and finally create a `Root` type that will help with the rootfinding from Chapter 9.

11.1 Basics of Julia's Composite Datatype

The composite data type in Julia is called a `struct` and consists of some number of fields of an existing type. A simple one is

```
?? PythonTeX ??
```

which has the two fields `num` and `str`. We are using the standard naming convention of a struct starting with a capital letter. We can create an object[1] of type `Mystruct` by

```
?? PythonTeX ??
```

and we can access the fields of the struct with `m.num` and `m.str`. They will return **?? PythonTeX ??** and **?? PythonTeX ??**. Also, you can get an array of the names of the fields with **??**. Note that the `fieldnames` command needs the datatype, not a variable of the datatype. This example is not very realistic, but the rest of the chapter will include more-practical examples.

One cannot change the fields of a `struct`. If `m` is defined as above and you try

```
?? PythonTeX ??
```

then Julia will report the error: **?? PythonTeX ??**

This is exactly like trying to redefine a `const`. As we will see most of the examples in this text are these constant types of `struct`s, you can make non-constant `structs` using the `mutable` keyword. For example:

[1] Although a Julia struct is not an object in the sense of an object-oriented language, the terminology is pervasive in the Julia community.

```
?? PythonTeX ??
```

and then define an object such as

```
?? PythonTeX ??
```

Then the redefinition **??** is allowed. Note: that if you try to do **??**, the you will get the error: **?? PythonTeX ??**which happens because you are trying to assign the value 7.1 to an integer, since the field `b` is an integer.

11.2 A Card datatype

In Chapter 14, we will use simulations to determine the probabilities of certain poker hands. Although the calculations can be done with built-in types, the result would be difficult to read and hard to understand. Here, we will create a `Card` datatype that will help clarify the code by creating a type with a rank (1 to 13 corresponding to Ace, 2, through 10, Jack, Queen, King) and a suit (1 to 4 corresponding to the suits "spades", "hearts", "diamonds", "clubs"), which we define as the characters arrays:

```
?? PythonTeX ??
```

where the suits are the unicode characters for the suits. See the julia documentation on unicode. We will create the following datatype:

```
?? PythonTeX ??
```

and if we enter **??**, then this will create a card of rank 3 and suit 2 (hearts). To access the fields of the type, we use dot notation. For example **??** will return **?? PythonTeX ??** and **??** will return **?? PythonTeX ??**.

Julia allows the printing of any type of data in a specified way. To do this we will define `Base.show` for our type. To do this, enter

```
?? PythonTeX ??
```

where the arguments of `Base.show` should be of type `IO` and then a print should be called as above. The result will be the concatenation of the characters corresponding to the rank and suit of the card. Try this out by typing **??** and you should get **?? PythonTeX ??**.

Note: this is different than just a `print` or `println` within a function, which is highly discouraged. This function is called whenever a card type is displayed.

When julia calls `Card(3,2)`, actually, it executes a special function called a constructor, which creates an instance of the type, which we have called an object. Although Julia creates the basic constructor–that is the one that fills the fields of the type in the order defined, we may want another constructor that will take an integer between 1 and 52 and returns the appropriate card. The following example will do this.

```
?? PythonTeX ??
```

There are a number of things about this definition:

- It is still a struct with two members (rank and suit)
- There is a function **??** used. This will use the default constructors that is **??** would define rank to be 3 and suit to be 4.
- The lines 4–6 define a Card with the two arguments. However, it tests to make sure that you put in valid ranks and suits. This is a bit complicated in that there is a nested tenary `if-then-else` command.
- The lines 7–9 define the constructor that takes a single integer and checks if that is valid. It uses integer division and the modulo `%` to calculate the rank and suit.

Notice that there is a function `new` that is called. The first constructor is the default constructor. If a second one is made (as is in this example), you need to explicitly define the standard constructor as well. See additional information about constructors in the Julia Documentation.

If you enter the above `struct` we saw the error:

```
?? PythonTeX ??
```

Recall, from the beginning of this chapter, this is because a `struct` is fixed, much like if a constant is declared and then it is redefined. We want to replace the first `struct` with this one, so we will restart the kernel by selecting ***kernel*** in Jupyter and then select ***restart kernel***[2]. You will need to reenter all of the needed cells, like the one with the ranks and suits as well as the one that starts `Base.show`. In Chapter 16, we will put structs inside a module and not have to restart the kernel as you develop a new datatype.

The second constructor will create a card from a number between 1 and 52 (and throw an error if it is not in this range). We can now create a card from an integer. For example **??** will return **?? PythonTeX ??**.

As we will see in Chapter 14, a hand is also helpful in playing card games, we will define a hand in the following way:

```
?? PythonTeX ??
```

which is just an array of cards. (Note: there is nothing here that specifies that the Hand has to be 5 cards, but that could be included by doing some error checking in the constructor). An example hand would be

[2] In Chapter 16 we will put structs in a separate file as a module and use a package called `Revise` to automatically handle changes without these errors.

```
?? PythonTeX ??
```

and again, since this looks a bit ugly, we can define a `Base.show` method for a hand:

```
?? PythonTeX ??
```

now `h1` should now return **?? PythonTeX ??**

We will return to this in Chapter 14 where we will use this datatype to help with simulations.

11.3 Polynomials–A parametric data type

Recall that a polynomial is the sum of nonnegative powers of x with constant coefficients. That is anything of the form:

$$P(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

and we may want to solve problems with them and it may be helpful to have a data type of this form. We can represent any polynomial by the coefficients. For example, the following will construct a datatype that is a polynomial with integer coefficients[3]:

```
?? PythonTeX ??
```

We can then represent the polynomial $P(x) = 2 - x + 3x^2$ with

```
?? PythonTeX ??
```

Let's say that we want to construct methods to add, subtract, scalar multiply and plot polynomials. However, if we want to allow Polynomials with coefficients other than integer coefficients, such as rational or real or complex, then we would need to write a set of functions to do this for each datatype, which is 1) painfully dull, 2) hard to maintain since there are many copies of the same function and 3) unnecessary because Julia has a nice feature called a parametric type.

We could define the Polynomial as

```
?? PythonTeX ??
```

which has Polynomial that has a type `T`. This now allows with one definition to have a polynomial of integers, floats, complex and rationals. However, this would also allow us to define a polynomial of strings or `Card`s, which doesn't make any sense. We can restrict the coefficients to number types with

[3] As well will see in Chapter 19, a `Vector` is shorthand or an alias for a 1D array

```
?? PythonTeX ??
```

which allows the coefficient to be any type that is a subtype of `Number`, which is what the notation `<: Number` means. This defines `T` to be any type that is a Number[4].

For example, using the above definition of `Polynomial`, we can define the following

```
?? PythonTeX ??
```

and the result of the last will be **?? PythonTeX ??**The beginning of the output, `Polynomial{Int64}` explains that it is a type polynomial. The `Int64` within the `Polynomial` type explains what type the coefficients have. If you enter the variable name, like `poly3`, then you will see the type of the coefficient for that variable as well.

It would be nice if the result looked like a polynomial. In this case, we can use the `Base.show` command.

```
?? PythonTeX ??
```

Now, if we reevaluate `poly5`, then Julia returns **?? PythonTeX ??**

Another nice thing that we'd like to do is have an add command and use the symbol `+` for this. However, if we enter:

```
?? PythonTeX ??
```

If we try **??** we get the error: **?? PythonTeX ??**so we need to do:

```
?? PythonTeX ??
```

and then we can rerun the function `+` above. This is now a function that adds two polynomials. Note that the coefficients do not need to be the same type, so there are two types, `T` and `S` available. If we now enter **??**, the result is **?? PythonTeX ??**

Note that this now a polynomial with floating point coefficients.

11.3.1 Exercise

Similar to the function above, write functions that

- subtract two polynomials,
- multiply a polynomial by a constant.
- multiply two polynomials. Note: remember to multiply polynomials you need to distribute, not just multiply the coefficients.

Note: to use `-` and `*`, you'll need to do `import Base.-,Base.*`

[4]Recall that Chapter 3 explain the data typing system and how to determine subtypes of a type.

11.3.2 Evaluating the polynomial

Another helpful function is to actually evaluate the polynomial. The basic way to do this is to sum the terms of the polynomial. A possible way to write this is

```
?? PythonTeX ??
```

and now if we evaluate `poly1` when `x=-2`, we enter **??** and get **?? PythonTeX ??**.

An alternative to this is to use Horner's form of the polynomial. That is, we can write a polynomial of the form $a_0 + a_1x + a_2x^2 + \cdots a_nx^n$ as

$$((\cdots((a_nx + a_{n-1})x + a_{n-2}) + \cdots)x + a_2)x + a_1)x + a_0$$

and in julia this can be written as:

```
?? PythonTeX ??
```

Evaluating a polynomial using Horner's form is more efficient from a computational standpoint. Evaluating a polynomial of degree n in standard form takes $O(n^2)$ operations, where as in Horner's form is $O(n)$.

Go over this in detail

11.4 Plotting the Polynomial

Recall that using the `Plots` package, one can consider plotting the polynomial by creating a function that pulls out the coefficients. However, the `Plots` package has a related package called `RecipesBase` that allows one to create a plotting recipe. That is if `p` is a `Polynomial`, then we can write `plot(p)` and it will plot the polynomial. Let's see how and don't forget to add the package and then enter **??**.

The package `RecipesBase` includes the macro `@recipe` and although the documentation is sparse, this page is helpful in the background. The basic idea on a plot recipe is to do the following:

```
?? PythonTeX ??
```

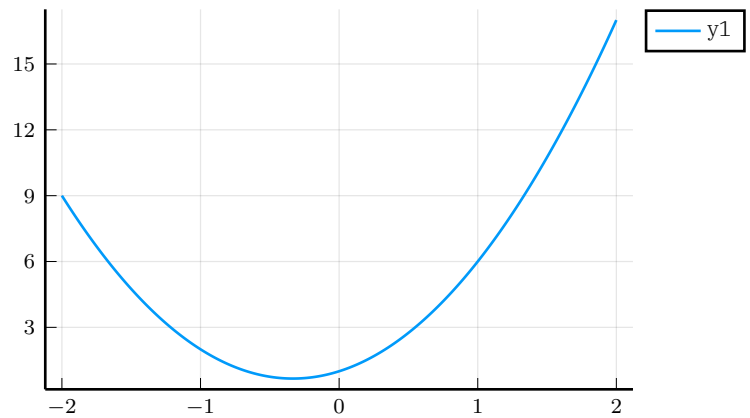
where `TheType` is any datatype (either built-in or user defined). The recipe needs to return some number of vectors of points (depending on if it is 1D, 2D or a 3D plot). A simple version of this for type `Polynomial` is

```
?? PythonTeX ??
```

where line 2 creates an array of x values of length 200, then line 3 creates the y values for each x value. Line 4 returns a tuple of the pairs of points. We can now use this to plot a polynomial and we will need to `using Plots` to:

```
?? PythonTeX ??
```

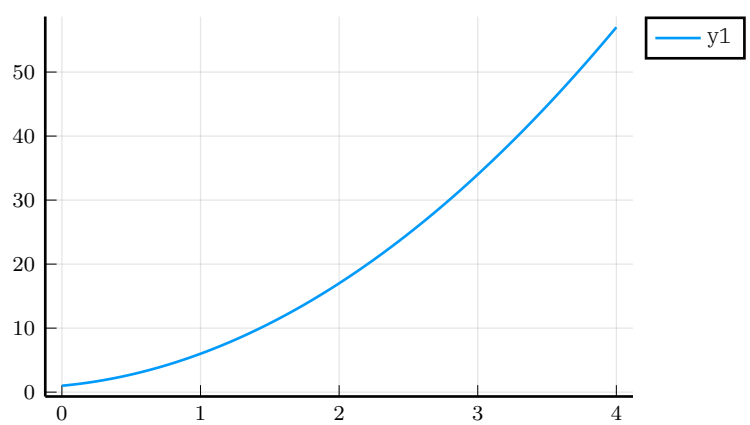
and since we defaulted the plot range from -2 to 2, we get the following plot:



and if we want to specify the plotting range:

```
?? PythonTeX ??
```

we get

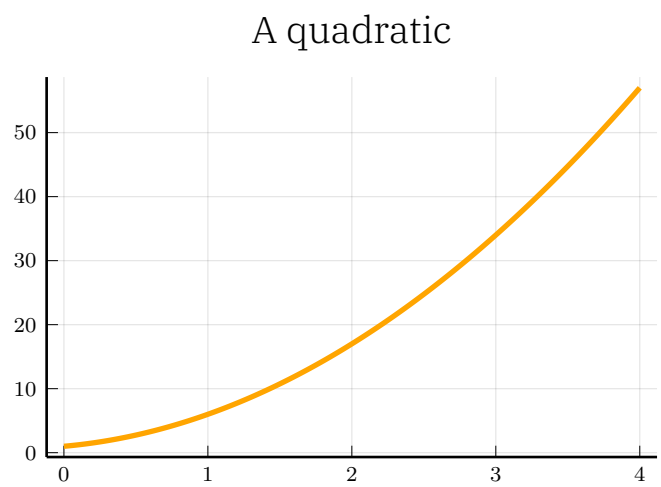


11.4.1 Changing other parameters

But wait... There's more... One of the fantastic things about using `RecipesBase` is that we can still use all of the other parameters associated with plot as we normally would. For example:

```
?? PythonTeX ??
```

produces the plot:

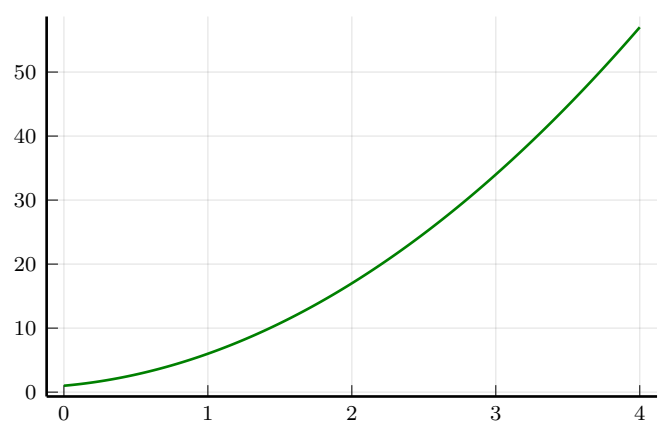


11.4.2 Setting Default parameters

Recipes also allow to set default parameters. Let's say that if we want to always plot a polynomial green without a legend that we can put these default parameters in the recipe:

```
?? PythonTeX ??
```

If we have this definition then **??** produces the following plot:



11.5 A Rootfinding datatype

The last example of a new data type will be related to finding the root of a function. We explored this in Chapter 9. One major issue with that function is that if Newton's method did not find the root, it wasn't clear. The method stopped if it reached 10 steps or actually found the root. There was no way that you (the user) knew which case occurred. We will use a `struct` to store the information about the rootfinding in this section.

In section 9.4, we developed the following function for solving Newton's method:

```
?? PythonTeX ??
```

Although if the function has a root and the method converges to it, then this method will probably find it, however, just running it, we don't know if it has exceeded the total number of steps (hardcoded as 10 steps in this example) or reached the root.

We are going to define a `struct` to deliver more information as follows:

```
?? PythonTeX ??
```

and then return an object of type `Root`. Thus we will replace the function `newton` above with

```
?? PythonTeX ??
```

and note that we store extra information that is returned. Let's run Newton's method on $f(x) = x^2 - 2$ as an example by evaluating

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**which again gives a lot of information, but perhaps not very easy to read since you would need to know each of the fields and what each means. We could then create a `Base.show` function to help with readability:

```
?? PythonTeX ??
```

which will print out different information if it converges or not. If we now run Newton's method on $f(x) = x^2 - 2$ with **??** then we get: **?? PythonTeX ??**and then if we run it on a function that does not have a root. Recall in Chapter 9, we ran Newton's method on $f(x) = x^2 + 1$. Since x^2 is nonnegative then the function $x^2 + 1$ is positive and so never is zero. If we enter

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

11.5.1 Exercise

Adapt the `Root` struct and the `newton` function to include the following:

- Extend the `Root` struct to include an array of the x values that save all of the steps of Newton's method. (Recall that you will need to restart the kernel to change the `struct`).
- Store the `x` values that Newton's method iterates through and then assign them to the `Root` datatype. You will need to update the `newton` method. Before the `while` loop, set the array equal to the initial point `x0` and each time in the while loop perform a push to the new value.
- Write a `showSteps` function that takes in an object of type `Root` and prints out a table of the x values.
- Test the new struct, `newton` and `showSteps` function.

Part II: Simulation

So far in this text, we have done very little scientific computation due to getting up to speed with some of the coding ideas and tools that we need. This part of the text shows examples of using simulation to solve certain problems. The heart of this is using random numbers to estimate the solutions of problems. The first chapter covers how computers generate pseudo-random numbers and the other section using them to solve problems.

Chapter 12
Probability Distributions and Pseudorandom Numbers

One big part of scientific computation is the subject of Monte Carlo simulations in which random numbers are used to model some situation. We'll spend the next part of this course covering this subject.

12.1 Probability

To fully understand how to use random numbers and how Monte Carlo simulations work, we need some basic probability. When understanding probability there are two types continuous and discrete, which for what we need will come down to using floating point or integers.

12.1.1 Discrete Probability

We'll first discuss discrete probability. A ***discrete probability distribution*** is a finite set (often will be a subset of integers.) For example, let's use the set

$$A = \{1, 2, 3, 4, 5, 6\}$$

and an event, X is subset of these numbers. For example, let $X = \{1\}$, then the probability that this event occurs in the ratio of the number of elements in each set or

$$P(X) = \frac{N(X)}{N(A)}$$

In this case, $P(X) = 1/6$ and that means that the probability that the number 1 comes up is 1/6. Think about this in the case of rolling a die. This says that the probability that 1 comes up is 1/6.

12.1.2 Continuous Probability

Another type of probability distribution is called a continuous distribution and in this case we'll only consider a type of continuous probability distribution called a uniform distribution.

Let's consider a set $A = \{x \mid 0 \leq x \leq 1\}$ or all real numbers between 0 and 1. Events are still subsets of the set A, however the probability that events occur is the fraction of the set.

For example, if $X = \{x \mid 1/3 \leq x \leq 1/2\}$, then $P(X)$ is the fraction

$$P(X) = \frac{\frac{1}{2} - \frac{1}{3}}{1 - 0} = \frac{1}{6}$$

12.2 Pseudo Random Number Generator

The Wikipedia Pseudo-random Number Generator page gives an overview of the subject and a lot of technical details. In short, a truly random number on a computer is very difficult to generate and generally not necessary because a pseudorandom number is sufficient. I also won't try to explain this in general term since you'd need a significant mathematical background.

A pseudo-random number generator is a function that produces a sequence of numbers that act like random numbers. Let's examine what this means in terms of the discrete probability with set $A = \{1, 2, 3, 4, 5, 6\}$.

If a pseudo-random number generator produces a sequence from this set then the sequence should have the following property:

- If the event $X = \{i\}$ for any i between 1 and 6, then $P(X) \approx 1/6$. And by approximately, as the sequence gets larger, the approximation becomes closer to 1/6.

Is this enough? No, the sequence

$$\{1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6, \ldots\}$$

satisfies the above property, but I don't think anyone would consider this random. Another property would be:

- If we know the sequence $\{a_1, a_2, a_3, \ldots, a_n\}$ then we can't predict the next number a_{n+1}.

This is obviously violated in the sequence above.

A little more technical definition of a sequence of pseudo-random numbers Let $(a_1, a_2, a_3, \ldots)$ be a random sequence. Typically we mean the following properties need to hold:

1. any number in the range 1 to 6 is equally likely to occur.
2. Take N random numbers and let s_n be the number of times the number n occurs. The fraction s_n/N should go to 1/6 in the limit as $n \to \infty$.
3. Knowing the sequence $s_1, s_2, \ldots, s_k$ does not allow us to predict s_{k+1}

If instead we use floating point numbers, there are a few different properties. Assume that the floating point number is in the range $0 < x < 1$. Then the sequence $(a_1, a_2, a_3, \ldots)$ is random if

3. Let $s_{[c,d]}(a_1, a_2, a_3, \ldots)$ be the total of the numbers that satisfy $c < a_k < d$. Then as the number of random numbers approach infinity $s_{[c,d]} = d - c$

12.2.1 Using Julia to simulate the rolling of a die

First, the main commands that are built-in to Julia are listed in the Julia Manual for Random Numbers. We can generate 100 random numbers between 1 and 6 using

```
?? PythonTeX ??
```

and notice that if you rerun the command, you'll get a different sequence of random numbers. We can check that this is doing what we expect by checking the probability that we get a 1 (or any other number).

To check the count of the number 1 that appears, try

```
count(a->a==1,S)
```

12.2.2 Exercise

Convert this to a probability. Is it close to what you expect? Try changing the number of random numbers used to larger numbers. Does your answer get closer to what you expect?

12.2.3 Floating Point random numbers

Let's look a floating point random numbers between 0 and 1. To generate a sequence (array) of such numbers, type

```
S=rand(100)
```

and we can check the number of values less than 0.1 with the command:

```
count(a->a<0.1,S)
```

and if we want the number of values between 0.4 and 0.7 then type

```
count(a->0.4<a<0.7,S)
```

12.2.4 Exercise

Change the number of random numbers used to much larger set and return the above commands. What are the results? Is this what you expect?

How to do find the fraction of number that are less that 0.25 and greater than 0.75?

12.3 Calculating π using pseudo random numbers

- Buffon's Needle Experiment (18th C.)
- Circle in the Square.

  Consider the square $\{(x, y) \mid 0 \leq x \leq 1, 0 \leq y \leq 1\}$ and the quarter circle that falls within $x^2 + y^2 \leq 1$. The area within the circle is the fraction of the cirle in the square or $\pi/4$.

  We can use this to estimate a value of π.

  1. Randomly choose points x and y in the range [0,1].
  2. Count all points that fall within the circle.
  3. Find the fraction of the points in the circle.
  4. Since this fraction should be $\pi/4$, multiply by 4 to estimate π.

  This can be done in julia using

  1. **??**
  2. **??**
  3. **??**
  4. **??**
  5. **??**

  which has the result . And all of these can be combined into one command:

  ```
  ?? PythonTeX ??
  ```

12.3.1 Exercise

- Write a function that take a positive integer n, in and performs the approximate-π calculation described above using n points. It should return the approximate value of π.
- Test it will high values of n and time it.
- for $N = 10^5$, $N = 10^6$ and $N = 10^7$ find the relative error of the estimate using your function and using the built-in value `pi`.

Chapter 13

Using Random Numbers and Probability Models

To understand basic probability, often problems are examined that use combinatorics or counting techniques to solve them. Consider

> A round table has 7 chairs around it. Mary and her friend Alisha and 5 other people are given seats at the table in a random manner. What is the probability that Mary and Alisha sit next to each other?

After practicing with problems like this, the solutions can be found quite simply, but in this chapter we examine some ways to use simulation to find the solution.

13.1 Rolling Dice

A relatively-simple example is that of rolling dice. As we saw in Chapter 12, if we have a fair, six-sided die, the the probability of any of the numbers coming up is 1/6. We can simulate that using random numbers in the following way:

```
?? PythonTeX ??
```

generated 100 numbers taken from the set $\{1, 2, 3, 4, 5, 6\}$. Running this command[1] results in for the first 10 elements.

As the number of time we roll a die increases, the probability of any one number appearing gets closer to 1/6. For example, to test this, try

```
?? PythonTeX ??
```

and then evaluating `p/l_000` results in **?? PythonTeX ??**, which is reasonably close to 1/6.

[1] don't forget that when running code with random numbers, you won't get the exact same results, but the spirit should be the same.

13.1.1 Exercise

- Trying increasing the number of random numbers used in the above example. You should notice that as that number gets larger, the resulting probability gets closer to 1/6.
- What does

```
?? PythonTeX ??
```

  measure in the sense of a rolling a fair 6-sided die? Does the result make sense?

13.2 Flipping Coins

Another probability model that arises often is that of flipping fair (and unfair) coins. There are a number of ways to model this, however, we will use random boolean values. For example, we can flip 100 coins by **??**. (Note: the result looks like an array of 0s and 1s, but look at the type, `Array{Bool,1}`, which says it is a 1-dimensional boolean array.)

We can determine the number of heads (when the result is 1), by

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**for me.

13.2.1 Flipping multiple coins

Another simple example is to flip multiple coins and generally count the number of heads or tails seen. Consider flipping 3-coins–say a penny, nickel and dime–and counting the number of heads. We then do that 3-coin flip a larger number of times.

We can do this with

```
?? PythonTeX ??
```

and the top 10 lines of this is: **?? PythonTeX ??**

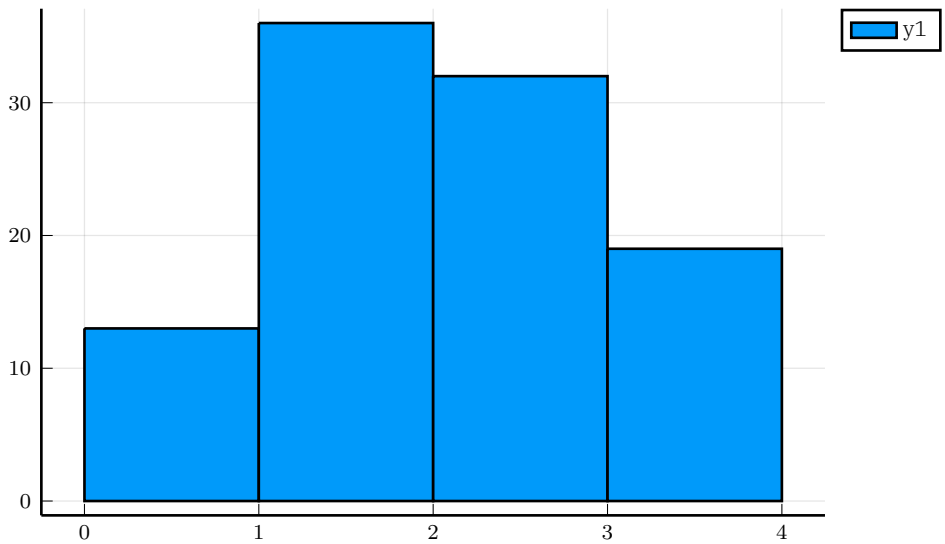
Each row contains the coin flips. For example the first row has heads on all three. The second row has a head and then two tails. If we are interested in the sum of the number of heads, we can do this with the `mapslices` functions seen in Chapter 6:

```
?? PythonTeX ??
```

The first few elements of the array is `[3; 1; 2; 2; 3; 2; 1; 1 ...]` We can plot the results of this with the function

```
?? PythonTeX ??
```

and the result is



13.3 Rolling 2 dice

How do we handle the rolling of two dice? Here's an array with each row having 2 dice.

```
?? PythonTeX ??
```

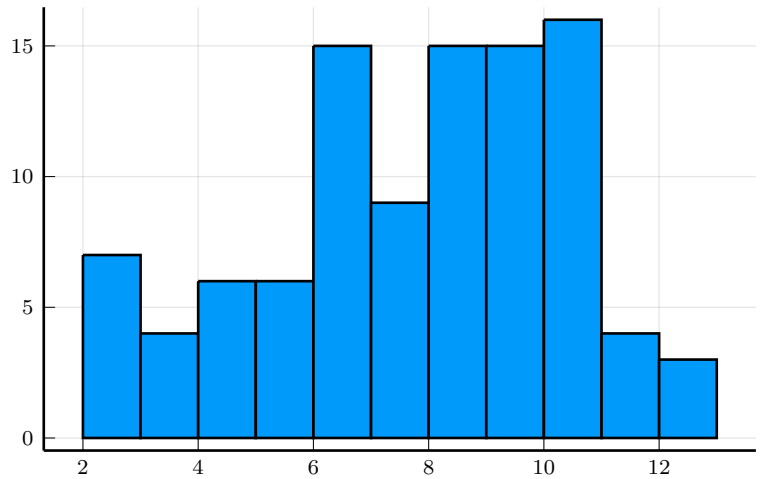
and then to find the sum of the dice:

```
?? PythonTeX ??
```

which sums along the rows. This is 100 rolls of 2 dice with the sum recorded. First, to get an idea of the distribution of the dice sums, let's plot the histogram

```
?? PythonTeX ??
```

where `nbins` is the number of bins and in this case, since it runs from 2 to 12, there's a total of 11 and this generates the plot:



We could find the number of 2's 3's, etc. using the `mapslices` function as above, however there is a nice way to do this use the `StatsBase` package. You may need to add the package and then load it with

```
?? PythonTeX ??
```

and then the `counts` function (Read the online documentation) can be used:

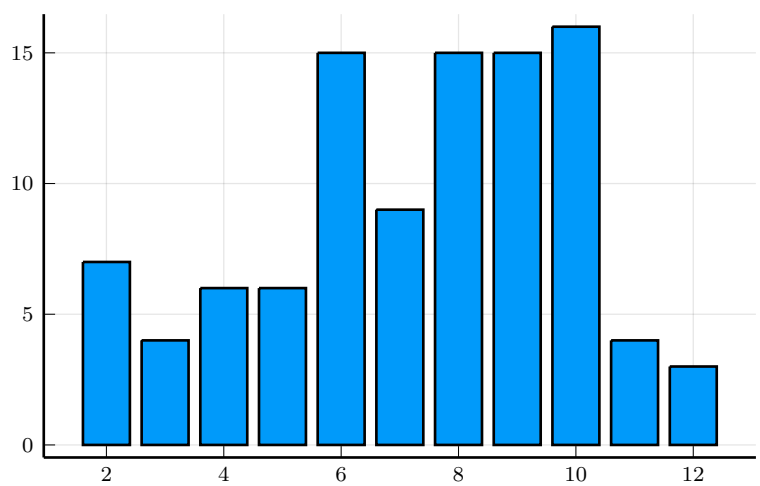
```
?? PythonTeX ??
```

returns a vector of how many of the dice sum fall into each number. When I ran this, I got

?? PythonTeX ??

and a nicer way to plot the histogram is

```
?? PythonTeX ??
```



13.3.1 Exercise

1. Change the code above to use 10,000 dice rolls. Estimate the probability that you
   (a) roll a 7.
   (b) roll a 10 or greater.
   (c) roll an even number.
2. Also, plot a the histogram using 1000 dice rolls.

13.4 Other Probability Models

Let's return to the problem that problem at the beginning of this section. We will solve this problem using the random modeling in this chapter.

First let's first store the names of the people at the take as an array

```
?? PythonTeX ??
```

where we use generic names for the other 5 people. We will take random permutations of this array below and the determine if Alisha and Mary are next to each other.

Before we find a random permutation, let's write a function that takes an array of names and returns `true` if they are next to each other and `false` if not.

```
?? PythonTeX ??
```

Like many aspects of coding there are many ways of doing this. Here's what we will do:

1. Find the position in the array where Alisha is sitting. This would be the place at the table she is in.
2. Find the position in the array where Mary is sitting.
3. Determine if the two numbers are next to each other. Don't forget that this could include positions 1 and 7.

The following is a relatively simple function is

```
?? PythonTeX ??
```

where the `findfirst` function returns the index of the array where the function is true (that is where the two friends are sitting). To test this:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

13.4.1 Random Permutations

We now return to the problem to study the probability. The `shuffle` command in the `Random` package[2] takes any array and shuffles (permutes) the contents in a random manner. For example:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

Then we can test if they are sitting next to each other. The follow repeats this a large number of times:

```
?? PythonTeX ??
```

and then the fraction of times is **?? PythonTeX ??**.

The true value of the probability can be found in the following way. Mary sits in one of the seven chairs. Alisha has a equal chance of sitting in one of the remaining 6 chairs. Two of the chairs are next two Mary, so the probability is 2/6 or 1/3. The result we see above is close to this value.

[2] The Random package is a built-in package and doesn't need to be added, but just enter **??**

Chapter 14

Probability of Poker Hands

This chapter will examine coming up with poker hands. If you are unfamiliar with poker or the hands of poker, here's a quick synopsis and the Wikipedia page on Poker Hands. And recall from Chapter 11 that a card has both a rank (Ace, 2–10, Jack, Queen, King) and a suit (hearts, diamonds, clubs and spades).

In this chapter, we are only concerned with 5 cards with no jokers and just determining if a hand satisfies one of the following:

- ***Royal Flush:*** the ranks of the cards are 10, J, Q, K, A and all cards have the same suit.
- ***Straight Flush:*** the ranks are sequential and all cards have the same suit.
- ***Flush:*** All cards have the same suit. Generally excludes straight flushes.
- ***Straigh:*** The ranks of the 5 cards are sequential. Generally exclude straight flushes.
- ***Four of a kind:*** four of the cards have the same rank
- ***Full House:*** two cards have the same rank, the other three cards have the same rank. The suit doesn't matter.
- ***Three of a kind:*** three of the cards have the same rank. The other two cards do not have the same rank. That is, don't count full houses.
- ***Two pairs:*** Two cards have the same rank. Two of the remaining cards have the same rank, but different than the first two pair. The 5th card does not make it a full house.
- ***One pair:*** two cards have the same rank. The remaining cards do not make it a different type of hand (full house, three of a kind, etc.)
- ***No pair or nothing:*** the cards don't form any other hand.

14.1 A user-defined package

In Chapter 16, we will create a module, but it is helpful for the topics in this chapter to use that module. Download `PlayingCards.jl` from ???. This module contains the `Card` and `Hand` types we developed in Chapter 11.

This is a module/package, which like other packages, need to be loaded with either the `using` or `import` keyword. Since this is just a file in the current directory, we first run the file and then load it

```
?? PythonTeX ??
```

where the `.` represents a local (current directory) module. Note that when running a module that has been added (downloaded), no . is needed.

Once we have the function written, we should test it on a few known and unknown full house hands. Try testing:

```
?? PythonTeX ??
```

The first 2 are full house hands and the last is not.

Now, let's perform a Monte Carlo simulation on a large number of poker hands and test if this gives the result we want. We'd like to have a function that tests many different hands. In general, this is called a trial, so we'll have following which passes in a function that takes a hand and a number of trials and returns the fraction of times that hand satisfies that function. First, here's the `runTrial` function.

```
?? PythonTeX ??
```

This returns **?? PythonTeX ??**. It would be helpful to put this in a function. We will do this and put all of these in a package together in the chapter 16.

Part III: Other Julia Features

Chapter 15

Advanced Function Features of Julia

This chapter covers a bit more on functions in julia. These feature allow the ability to write code that easier to read and debug.

15.1 Testing Arguments

Let's return to the factorial function that we saw in Section 4.8:

```
?? PythonTeX ??
```

We saw before that if we call this with a non-integer number that we will get an error, but what if we include a negative number? That's a challenge to just try it.

If you run **??** you will see **?? PythonTeX ??**

This occurs because when a negative number is put into the `fact` function that since n is not 0, it computes `n*fact(n-1)` which will evaluate `fact` again at a more negative number. This would never end except that recursive functions are evaluated on what is called the stack and there is a maximum number of items that can be put on the stack and thus this occurs. We can prevent it with the following:

```
?? PythonTeX ??
```

The first line of this evaluates `n >= 0`. If that is false, the second after the || is evaluated and an error is thrown. If you enter a nonnegative number for `n`, then the first line `n>=0` evaluates as `true` and the rest of the line doesn't get evaluated because the or || shortcircuits.

15.1.1 Exercise

In Chapter 8, the `findAllFactors` function doesn't make any sense if the number is less than 1. Add a line to the function that throws an appropriate error.

15.2 Optional arguments

Let's return to Newton's method, which we wrote before as

```
?? PythonTeX ??
```

Notice that we hard-coded the stopping criteria and the max number of steps. Let's adapt this function to include the following:

1. Create two optional parameters called `tol`, the error tolerance and `max_steps`, the maximum number of steps to perform.
2. Check to make sure that they are valid. That is, are there any restrictions on the two parameters, check like the previous section.
3. If the number of steps exceeds the maximum number of steps, then throw an `ErrorException` and a reasonable message.

We will build up all three of these:

Add Optional Parameters: instead of defining (or hard-coding) a value, it's generally better to assign it to be a optional parameter, which means the value will be defined in the argument list. Let's define the tolerance (`tol`) and the max number of step (`max_steps`) in the following way:

```
?? PythonTeX ??
```

Note that when entering this, it says there are 3 methods with this name. This is because julia will build three different function signatures. One with 2 arguments (and both `tol` and `max_steps` use the default values), one with 3 arguments (and `max_steps` using it's default value) and one with all 4 arguments. Also remember that `1e-3` means 10^{-3}.

This seems more robust in that we can now call Newton's method with different values of tolerance and steps. So:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**. But if we use a lower tolerance:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**.

If we want to change the number of steps, however, we need to include the tolerance as well like:

```
?? PythonTeX ??
```

which results in **?? PythonTeX ??**. We will see in the next section an alternative way to do parameters, called ***keyword parameters*** which requires names of parameters, not the order, which is helpful as the number of arguments/parameters for a function gets large.

Check that the parameters are valid: The `tol` and `max_steps` optional parameters should both be positive, so we will add checks on these as

```
?? PythonTeX ??
```

and including negative numbers for either of these two now throws an error.

Throw an error when exceeding the maximum number of steps If

```
?? PythonTeX ??
```

and we run:

```
?? PythonTeX ??
```

we get the response **?? PythonTeX ??**. If we evaluate the function at the root, with **??** the result is **?? PythonTeX ??**, which is definitely not 0 (or very close), so this doesn't appear to be a root. What happened? If we temporarily print out the values of `x1` within the loop, we'll see that the x values bounce all around and then just stops.[1] It's not clear, but what happens here is that the max number of steps is reached, but you are not alerted.

So to make this more clearn, before the last line of the function, let's include

```
?? PythonTeX ??
```

and then rerunnning `newton(f,25)` gives an error which explains to the user that a root was not reached.

15.3 Keyword Arguments

Although optional arguments are quite helpful, there are two situations that they can be annoying.

[1] In the Newton method, add the line **??** just after the line with the **??** statement. Rerun the function then and try **??** again.

1. If you want to change one optional argument without the others, we may not be able to. In the Newton's method example, if we want to change `max_steps`, which is the 4th argument without changing the 3rd argument, we can't just put in the value of `max_step`.

2. For more complex functions, there may be a lot of possible parameters, most of which could be optional. The order of the arguments are important, and if you mix them up, you may either get an error or get unexpected results.

Using keyword arguments can solving both of these problems and We can rewrite newton's method as

```
?? PythonTeX ??
```

and it is important to note that there is a semicolon separating the arguments from the keyword arguments.

Now we can use this more easily. The function call

```
?? PythonTeX ??
```

results in **?? PythonTeX ??** and if we only want to change the number of `max_steps` is

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**.

This was just a quick introduction to this and for further information, look at the julia documentation on keyword arguments.

15.3.1 Exercise

Later in Chapter 21, we'll see the trapezoid rule, which is used for numerical integration or area under a curve. The technique subdivides the interval $[a, b]$ into 10 equal pieces and approximates the area under the curve with the area of a trapezoid. We will explain the function

```
?? PythonTeX ??
```

later in that chapter, but note that we can estimate the area under the curve $y = x^3$ on the interval $[0, 4]$ by entering

```
?? PythonTeX ??
```

and the result is **?? PythonTeX ??**.

- Rewrite the code above to make a keyword argument of the number of subdivisions (10) and set the initial value to 10.
- Find a better estimate for the area under $y = x^3$ on $[0, 4]$ using 100 and 1000 subdivisions.

15.4 Variable Number of arguments

Recall that the geometric mean of a set of n numbers $x_1, x_2, \ldots, x_n$ is given by

$$\sqrt[n]{x_1 x_2 \cdots x_n}$$

Below is the code for `geom_mean` which computes the geometric mean for any number of elements. It would be unfortunate if we have to write different functions for different number of arguments. We can write a variable number of arguments with a ... trailing the last argument. The following is a generalized version of the geometric mean:

```
?? PythonTeX ??
```

and remember that we can write $\sqrt[n]{a}$ can be written as $a^{1/n}$. This allows the following:

```
?? PythonTeX ??
```

as well as

```
?? PythonTeX ??
```

to both work with the same function.

The parameter `x` is assigned to be a tuple, as discussed in Section 3.9. The individual elements of the argument are accessed in the same way. The following is a completely manufactured example.

```
?? PythonTeX ??
```

which adds the odd elements (not the odd numbers) in a list of numbers.[2] Running the command such as **??** returns **?? PythonTeX ??**, which is the sum of every other element.

See The Julia documentation on Variable Arguments or VarArgs for more information on variable arguments.

[2] A better way to write this would be **??**, but the above function shows better how the vararg x is like an array.

15.4.1 Exercise

15.5 Multiple Dispatch

Generally each function that you write (or is built-in to Julia) has a signature which is the number and types of arguments. For example, the function `newton` above has two arguments and two keyword arguments. The first is a function, the second a number and the keywords are a float and an integer.

Julia allows functions of the same name with different signatures. The execution of this is called ***multiple dispatch***. The classic example is the `+` function. If we only consider those with 2 arguments, then + can take 2 integers an integer and a float, two complex, a float and a complex, etc. Each one of those functions have the same name with different signatures. This makes life much easier to code. We can write **??**, **??** and **??**, which under the hood requires different code to add integers, a float and an integer and a pair of rational numbers and use + for each of these.

To view all of the methods, use the `methods` command. For example, `methods(+)` lists 171 methods and the signature of each.

Let's consider a function called `findMax` which return the maximum number of a whole bunch of possible arguments. Let's start going through them:

- 2 numbers

```
?? PythonTeX ??
```

- 3 numbers

```
?? PythonTeX ??
```

- variable number of numbers

```
?? PythonTeX ??
```

It is also nice to return the maximum value of a range (technically an `AbstractRange` type) which includes a `UnitRange`, `StepRange` and `LinRange`. These all have the same underlying code, so we don't need a separate function for each type. Therefore, we can put all of these together with the function

```
?? PythonTeX ??
```

and note that `r` is an abstract range, then `first(r)` returns the first element of the range, `last(r)` returns the last element and `step(r)` returns the step size of the range.

Write this function and test with

1. **??**

2. **??**
3. **??**
4. **??** which makes a range from 1 to 5 using 9 steps.

Once you have created these, and type `methods(the_max)` julia should return:

```
4 methods for generic function findMax:
```

and list the 4 methods with their types.

15.6 Parametric Methods

As we are thinking about writing other `findMax` functions with a different argument signature, a natural one would be an array of `Real`s (mainly because we will need to do something different for `Complex` arrays).

If we write:

```
?? PythonTeX ??
```

which is similar to that above for any number of numbers and then let's test it with the array **??**, we'll get the error:

```
?? PythonTeX ??
```

and this is because even though `x` is of type `Array{Int64,1}` that doesn't fit the signature `Array{Real,1}`.

One way to get around this is to write a function for each number type we may want to find the max of (`Int128`, `Int64`, ..., `BigInt`,`Float64`,`Float32`,`Float16`), but if you notice every function will be identical and this would be painful and a lot of code copying.

Instead, we will use something in julia called parametric functions which allows us to write a large number of nearly identical functions with the same code. To do this with the `findMax` function, we will write:

```
?? PythonTeX ??
```

and the expression `where T <: Real` is used as any type `T` that is a subset of the `Real` numbers (including integers, floats and rational)

and then **??** returns 10. Also if we have

```
?? PythonTeX ??
```

returns `10.0` and

```
?? PythonTeX ??
```

returns `10` (as a `BigInt`).

15.6.1 Exercise

Create a `findSum` method that is parametric. The template (signature) for this function should be

```
?? PythonTeX ??
```

by using a `for` loop. Start with the line:

```
?? PythonTeX ??
```

which creates the `sum` variable as a zero with type `T`, so depending on the data type the array is, it will have the appropriate sum data type.

Try you function with

- **??**
- **??**
- **??**

and make sure that the result is the expected type.

Chapter 16

Creating Modules and using Unit Tests

Throughout this text, we have added and loaded packages or modules. These were generally official ones. In this section, we will now learn how to write our own module. We will learn how to do this by creating a `PlayingCards` module with all of the types and functions associated with it. It is helpful to read through the julia documentation on modules.

In addition, much of content of this chapter was first presented in Chapter 14 and it is recommended that you review that material as needed.

16.1 Creating a Module

To begin with, let's look at a module template:

```
?? PythonTeX ??
```

As you can see above, a module has a name (in this case `PlayingCards`) and ends with the keyword `end`. We will next put a number of types and functions inside the module, but in order for someone loading the module, you need to export anything to be used.

The following is a more detailed version of the module:

```
?? PythonTeX ??
```

Open a new file in jupyter and copy-paste the above module into the empty file. It will need to be called `PlayingCards.jl`.

Here are some things to note about the module:

- the **??** line will import the `show` command from `Base`, the module that contains manly basic julia commands. We need to do this for the show command below.
- the **??** line says what will be available if the module is loaded.
- the `Card` and `Hand` structs are defined.

- the functions `Base.show` functions for both `Card` and `Hand` are defined. The command `Base.show` is automatically loaded, so these didn't need to be exported.
- the `isFullHouse` functions is defined.

16.2 Debugging a module

All is great now if you have a bug-free module, but even the best programmers are not capable of bug-free code the first time. Notice that if you make changes to the module, then the changes won't be present. And if you reload the file and the module, from the lines:

```
?? PythonTeX ??
```

The first line should report:

```
WARNING: replacing module PlayingCards.
```

and any syntax errors that you may have, and the second line reports:

```
WARNING: using PlayingCards.Hand in module Main conflicts with an existing
↪  identifier.
WARNING: using PlayingCards.Card in module Main conflicts with an existing
↪  identifier.
```

and any changes won't be made. (Try altering the Base.show function on a `Card`). If you want changes to be able to be used, you will need to restart the kernel.

So while debugging a module, we will use the `Revise` package instead. Make sure you add it first, then restart the kernel, then enter

```
?? PythonTeX ??
```

where `includet` is a command that tracks changes and automatically reloads after any changes.

16.3 Unit Tests

Writing a module is important, but making sure it does what it is supposed to is just as important. At first, when you start writing code, you typically make sure there are no bugs, but after time, code changes and you're not sure that the code still works. The notion of a unit test is a set of tests to determine if you get out from the code what you expect. This is a language-independent idea and should be created once you write any level of substantial code.

To run a test in Julia, first import the `Test` package:

```
?? PythonTeX ??
```

To do a test, we'll use the **??** macro and it's a good idea to check out the Julia documentation on Test. For this module, let's first test that the constructor is working using the **??** function.

```
?? PythonTeX ??
```

should return **?? PythonTeX ??**. Recall also that we can pass in an integer between 1 and 52, so

```
?? PythonTeX ??
```

should also return **?? PythonTeX ??**. If we try to create a card that is not valid, then we won't get a Test Passed. For example:

```
?? PythonTeX ??
```

will return `Error During Test`. A better way to test for this is using the `@test_throws` method to test if an error is thrown:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**

Here's another nice test that will check if the two different ways to create Cards are the same. For this we will need a way to test if two Cards are equal.

```
?? PythonTeX ??
```

then create two cards

```
?? PythonTeX ??
```

and test if they are the same with

```
?? PythonTeX ??
```

and this returns **?? PythonTeX ??**.

16.3.1 Exercise

Write a test that the `isFullHouse` method is working. To do this:

1. Create a hand that is a full house, called `h1`. Run **??**.
2. Create another hand called `h2` that is a full house and test it.
3. Create a third hand called `h3` that is not a full house and test it. To get the test to return passed, perform **??**.

16.4 Creating a test suite

Often in a set of tests, there are mulitple tests that go together. For example, if we just want to test that the construction of Cards are working, we can create a test set in the following:

```
?? PythonTeX ??
```

where all three methods to construct a card are used and all should work. If you run this, you should get:

?? PythonTeX ??which just shows that we passed all of the tests. If one of them doesn't pass, you will get information about the one that wasn't passed. Try changing one of the tests above to get an illegal card.

16.4.1 Putting all tests in a file

The Playing Card test file is a better way to run a set of tests. To use this, download the file, put it in the current directory of jupyter and then run it with:

```
?? PythonTeX ??
```

and you should see:

```
?? PythonTeX ??
```

printed out.

16.5 Modules and Unit Tests

Once you have enough code to write a module, the first thing should be to start writing unit tests to make sure it is correct. In fact, good programming practice is to write the API (Application Programming Interface), which is just the function signatures with no funtion bodies, then the test cases before any working code is written.

In any case, once you have unit tests working, you should write and revise any code and after any changes are made, rerun any unit tests.

Chapter 17

Parallel Computing

Briefly, parallel computing is a method of running code on multiple processors (or multiple cores of the same processor) at the same time. In general, this is a difficult task depending on where data is stored and retrieved. The Julia Documentation on parallel computing is a good place to start.

Let's start with something relatively simple. Consider the following code:

which mimics flipping a coin `n` times. We can simulate flipping 2 billion coins and finding the fraction that is heads by the following:

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**

17.1 Running parallel code

We now wish to take advantage of the fact that today's processors often have multiple cores and multiple threading. There are some helpful things in the `Distributed` package:

```
?? PythonTeX ??
```

Let's say that we wish to run it simultaneously on 2 cores (even if we know we have more) If we type

```
?? PythonTeX ??
```

this would then allow julia up to 2 "processors". There are a couple of ways to now run code. The first gives us more control, while the second allows for some easy code writing.

17.1.1 Assigning code to workers

First, we need to make sure that the function `countHeads` is available to each core of the machine. To do this, we will start the function with the `@everywhere` macro:

```
?? PythonTeX ??
```

Any function that you will need on multiple cores will need to be prefaced with the macro `@everywhere`. After this, we can now send code to different cores. Type:

```
?? PythonTeX ??
```

and you should notice that it doesn't return the number of heads as expected and it goes almost instanteously. Don't worry right now about the type of object that is returned.

Even though it returns quickly, code is already running and you can see this with your machine's process manager (Activity Monitor on MacOS). To get the results, we type

```
?? PythonTeX ??
```

and it will return the number of heads flipped. To test the timing, put all three of these in a block:

```
?? PythonTeX ??
```

and the results are: **?? PythonTeX ??**

This is perhaps a bit disappointing in that if we use two cores, then might expect half of the time. It's not quite that simple in that there is some overhead for using multiple cores. However, if you use more cores like the exercise below, you'll probably get better results and for longer processes, it will pay off.

17.1.2 Exercise

1. Add two more cores to julia with the `addprocs` command.
2. Rerun the block of code `@everywhere function countHeads` code above.
3. Create 4 spawning lines. Call them `a, b, c` and `d` and use `5*10^8` for each.
4. Time the sum of the four.
5. Note: if you truly have 4 cores, then you will see a further halving of the time. If not, you will probably only see the same results.

17.1.3 Adding all of the cores

Since most computers have multiple cores, this is helpful, but what about adding all of the cores? If we just call

```
?? PythonTeX ??
```

then it adds everything available. You will get an array of the workers.

17.1.4 Parallel for loops

One issue with what we did above is that we have to think about spawning to individual processors or cores. That is fairly annoying. Fortunately, another helpful feature of julia is that of a parallel for loop. Try the following code:

```
?? PythonTeX ??
```

and the results are: **?? PythonTeX ??**

where julia distributes the load in the for loop across the available processors and the time should be a fraction of the original head counter code where the fraction is the number of actual number of possible processors or cores.

17.2 Writing a parallel card simulator

For this, we need to reload the PlayingCard module:

```
?? PythonTeX ??
```

(or you may need to put in a different path to your module)

If we have the following code to simulate a large number of hands:

```
?? PythonTeX ??
```

And let's just check to make sure it works:

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**

We now wish to write a distributed version of this in which we will replace the for loop with a distributed for loop. First, the module needs to be loaded for all cores by

```
?? PythonTeX ??
```

and then the following will be the parallel version of the hand count function:

```
?? PythonTeX ??
```

and running this:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**, which cuts the time by about quarter.

17.2.1 parallelizing the ....

17.3 A Parallel Map Function

In Chapter 7, we saw the `map` function, which is the standard way if you have an array and need to do the same thing to each element of the array, returning the result. If the function that is applied to each element of the array is costly and you have multiple processors/cores to work with, doing this in parallel can be helpful.

We demonstrate how to do this with just the coin flip right now using the `pmap` function. We first make an array of length 12 (although this isn't an important number):

```
?? PythonTeX ??
```

so each element is 1 billion. We wish to run the coin flipper on each element of the array. Don't forget that the `countHeads` function must be declared everywhere. The following runs the function:

```
?? PythonTeX ??
```

and the resulting time is: **?? PythonTeX ??**

In comparison, if we run the regular map function:

```
?? PythonTeX ??
```

the result is **?? PythonTeX ??**. Again, this is about 4 times slower.

17.3.1 When to use `pmap`?

It seems like the `pmap` function should always be used if it speeds up calculations. However, note that in this example, the size of the array was small but the function on each element took a relatively long time. If we have an array with millions of elements, though, the `pmap` function may actually be slower.

17.4 Shared arrays

The last example shows that we may have to do something on an array. For simplicity, let's say we have a fairly large array:

```
?? PythonTeX ??
```

A common thing to do is to smooth an array (and is often done to images) by a windowed mean, which means that every element is replaced by a mean of the points around it in some window. We first define a windowed mean by the following function:

```
?? PythonTeX ??
```

which first determine a window (being careful with the first and end of the array) and then just calculating the mean.

Then if we have a new array of zeros:

```
?? PythonTeX ??
```

we fill the new array with the windowed mean:

```
?? PythonTeX ??
```

will fill the array with the smoothed version in about 43 seconds.

Note: the astute reader is probably thinking that using `map` to do this is the right way to go, however, we can't use map in this instance because the `window_mean` function doesn't just use a single number (from an array), it uses the entire array.

A natural way to speed this is is to send this to individual processor/cores and work on this in pieces. One problem with this is that we would have to send the entire array to each worker and that is expensive. Since a 64-bit integer is 8 bytes, the array of 100 million is about 800 Mb of memory and that is reasonable expensive to pass around. Instead, we are going to use a shared array in the package `SharedArrays` and you will need to add this.

```
?? PythonTeX ??
```

Part IV: Data Analysis

Chapter 18

Introduction to Data Analysis

Data is ubiquitous these days and being able to analyze it is crucial. In this section, we will first load a dataset that is available as a Julia package, plot the data and do some basic analysis of `DataFrame`s. Then, data will be loaded from a file. Lastly, a package named `Query` will be used to take a dataset, filter it and parse using some notation that is different than other Julia notation, but common to datasets.

18.1 Data from the `RDatasets` package

The statistical software/language called R has many built-in and loadable datasets. A julia package called `RDatasets` has many of these datasets and additionally we will use the `StatsPlots` package. You will probably need to add these packages before

```
?? PythonTeX ??
```

The datasets are subdivided into package and you can see the entire list of the packages with

```
?? PythonTeX ??
```

and then a listing of all of the datasets in each subpackage can be found with

```
?? PythonTeX ??
```

as an example. For example, we can load the classsic `iris` dataset from this to view:

```
?? PythonTeX ??
```

The dataset returns as: **?? PythonTeX ??**

The first line of the output of the dataset shows the size (number of rows and then number of columns). The third line is the column names, the fourth line is the datatype for each column and then the first 4 rows are shown. The first column is row number (not part of the dataset). The last column is a categorical datatype and more will be said about this later.

18.2 Dataframes

In some ways, a DataFrame is like a 2D array in terms of accessing parts of the array, but in many ways it is very different. Notice that the columns of the `iris` `DataFrame` are different types, but it is important that each column has a fixed data type.

This is very different than a 2D array in that the entire array must have the same type. Julia has the ability to have an array of `Any` and then anything can go there, but anytime julia doesn't know a data type, it slows down calculations tremendously. A better way to think of a DataFrame is that of a collection of 1D arrays, where each array (column) has a name.

We can access the DataFrame using some of the syntax from arrays. For example, **??** will return the element in the 2nd row and 3rd column. Many of the array accessing methods from Chapter 6 also work.

There are other nice features though as typically we want to do things on columns. For example: `iris[!,:PetalLength]` returns a 1D Array (of `Float64`). The ! in the first slot means to use all rows. If we want the 11th through the 20th row of the `SepalWidth` column, enter

```
?? PythonTeX ??
```

A perhaps-confusing result of using a `DataFrame` with `IJulia`, is that there is a limit on the number of columns and rows shown. Check out the Julia DataFrame documentation for information on this. You may see at the top of the output for a DataFrame that

18.2.1 Constructing a DataFrame

Although nearly always, you will use a DataFrame by loading in a dataset, a DataFrame can be created by specifying the columns. Consider:

```
?? PythonTeX ??
```

which will create a DataFrame with column names `A, B` and `C` and Julia interprets the data types of each column. The result is **?? PythonTeX ??**

There's a number of helpful functions associated with DataFrames:

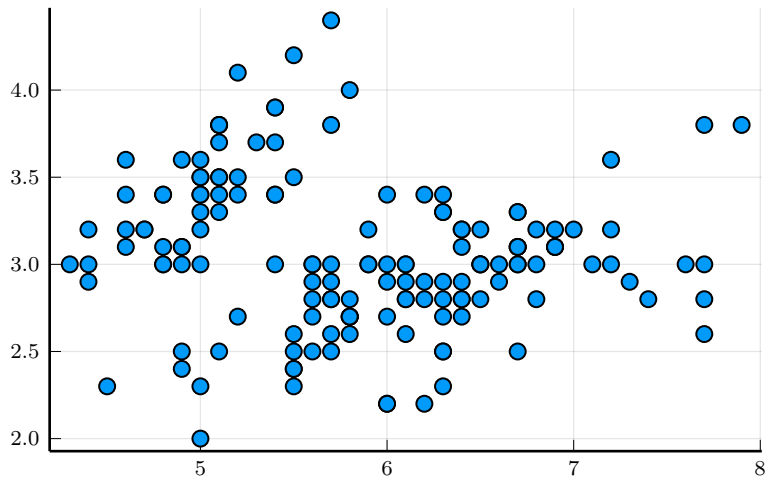
- `size(df)`: similar to an array, returns the number of rows and columns.
- `names(df)`: Names of the columns as an array
- `eltypes(df)`: the types of each of the column as an array.
- `first(df,n)` and `last(df,n)`: prints the top n and bottom n rows of the dataframe.

18.2.2 Plotting with `StatsPlots`

We can do a scatter plot of the first two columns in the following manner:

```
?? PythonTeX ??
```

and the resulting plot is



There are some nice features using the package `StatsPlots`. For example, the following will produce the same plot:

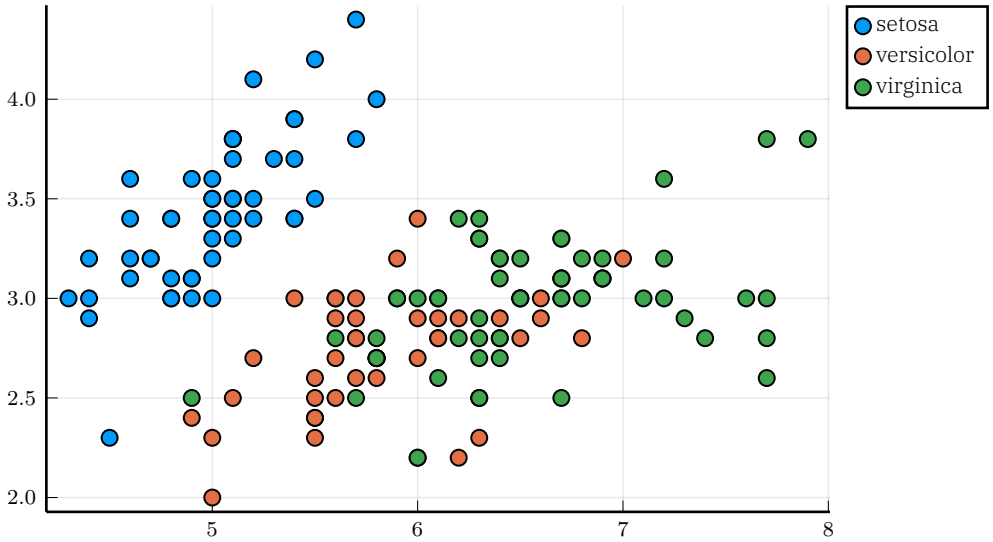
```
?? PythonTeX ??
```

where the `@df` macro says to use the DataFrame `iris` and then any column information is interpreted to be of that DataFrame.

Recall above that there was a Categorical datatype in the column `Species`. The following scatter plot delineates the species with different colors:

```
?? PythonTeX ??
```

and this results in



18.2.3 Basic Statistics

First, we'll load in the `StatsBase` package (which you may have to add) with the command **??**. Now, we can find the mean and standard deviation of columns with

```
?? PythonTeX ??
```

will compute both and return: **?? PythonTeX ??**.

MORE OF THIS??

18.3 Manipulating DataFrames using the `Query` package

Many of the commands in this section require the `Query` package. The command within this package allow `DataFrame`s to be manipulated in nice way. You may need to add this to your Julia installation and then don't forget to enter **??**.

Recall that if you want to apply some function to each element of an array, the built-in `map` function is useful. For example, in Chapter 4, we saw the following example:

```
?? PythonTeX ??
```

which returns the array `[1,4,9,16,25]`. Using `Query`, we can get this the same way by

```
?? PythonTeX ??
```

where we have used the pipe `|>` command. This can be thought of as sending the array into the function `@map` which applies the square function. This gets quite nice when stringing a number of commands together. Consider

```
?? PythonTeX ??
```

This is a shorthand for the following steps:

1. **??** creates an array from 1 to 10
2. **??** squares each element
3. **??** keeps only the element in the array that are even.
4. **??** finds the mean of the result or **?? PythonTeX ??**.

We are going to use the pipe operator throughout the rest of this chapter.

18.4 Analyzing Census Data

```
?? PythonTeX ??
```

Before we begin, make sure that you have added and loaded the following packages: `DataFrames, Query, StatsBase, CSVFiles` and `StatsPlots`. Since many of the more-difficult aspects of the code below are in the `Query` package, I would recommend checking out the documentation.

Download the file Gaz_ua_national.csv and save it somewhere that you can access it from Julia. This file has a lot of census data that we will try to find answers. Information about the data is on the census website

We will start by loading the file with

```
?? PythonTeX ??
```

and although this looks like a DataFrame, it isn't. Note that using `typeof(census_data)` returns `CSVFiles.CSVFile` type, which isn't as useful as a DataFrame. So instead we can pipe the output from the file to the `DataFrame` constructor.

```
?? PythonTeX ??
```

and checking the type shows that it is now a `DataFrame`.

18.4.1 Answering Census Questions

There are a number of questions that we can answer. The questions are listed and then the approach to finding the answer.

1. What are the top 10 areas in population?
2. How many population areas are west of 120 degrees longitude?

3. Give a histogram plot in terms of population? (What are good bin sizes?)
4. What is the total population of all areas?
5. What the top 10 area in housing units?
6. What is the total number of housing units?
7. What is the average number of people per housing units for all areas?
8. For the top 10 area in population, find the average number of people per housing unit?
9. What are the top 10 areas in land size?
10. What are the top 10 areas in water size?
11. What are the Massachusetts areas in the data?
12. What is the average population, median and standard deviation of the areas?

18.4.2 Sorting Dataframes

We can answer the first question with a simple sort, which is from the `Query` package.

```
?? PythonTeX ??
```

returns: **?? PythonTeX ??**

1. `census_data` starts with the original DataFrame.
2. `|> @orderby_descending(_.POP10)` sorts by the column `POP10`. The `_` is like we saw above an anonymous function and the dot notation is used like a struct, but in this case returns the column of that name.
3. `|> DataFrame` then switches it back to a dataframe.
4. `|> df->head(df,10)` returns only the first 10 columns of the dataframe.

Filtering the data

Let's try to find all of the location to the west of 120 degrees west longitude. This is store in the column INTPTLONG. We can find all of these by using the `@filter` macro.

```
?? PythonTeX ??
```

which returns 279.

The new parts of this is the `@filter` command which returns only the rows of the dataframe in which the INTPTLONG column is smaller that -120. Also the `nrow` command returns the number of rows in the dataframe.

18.4.3 Histogram of the Population

To produce a histogram of the population, we will use the `histogram` function of the package `StatsPlots`. Check out the documentation. If we do:

```
?? PythonTeX ??
```

then we get the following plot:

images/ch16/plot04.png

The trouble with this is that there are many many census areas with very little population. We can fix this with a log scale. If you enter

```
?? PythonTeX ??
```

then the result is:

images/ch16/plot05.png

Adding another column to the Dataframe

We are interested in finding the highest housing density, that is the number of people per housing units. We will create a new column to do this:

```
?? PythonTeX ??
```

and now sorting:

```
?? PythonTeX ??
```

shows the result as **?? PythonTeX ??**

18.5 Analyzing information about Olympic Athletes

Let's look at another data file. Download the file OlympicAthletes_0.csv, which lists all olympic medals between 2000 and 2012 and save it somewhere that you can access it from Julia. It is a comma separated file (CSV). Load it as a dataframe.

Here's some questions to answer:

1. What is the total number of medals given in all Olympics in the dataset?

2. Who had the most olympic gold medals in the Summer 2000 games? How many medals?
3. Collectively taking each olympics, give the top 10 athletes by number of medals.
4. Who has the most Olympic Silver medals in the data set (Collectively over multiple olympics)
5. Produce a new DataFrame that lists total number of medals by country. Produce a histogram of the total number of medals.
6. Plot the number of medals that the U.S. collected over each olympic year.
7. What is the age with the most number of total medals.
8. Produce a new data set with total number of medals per sport per year. What are the top five sports over the past 4 olympics.

18.5.1 Loading the file

Since it it comma delimited, the following will load the file:

```
?? PythonTeX ??
```

and view the first few rows with

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**

It's helpful to have column names without spaces in them, so we will replace all of the spaces with underscores.

```
?? PythonTeX ??
```

The dataset is ready now and we'll go through each of the questions.

1. What is the total number of medals given in all Olympics in the dataset? This will be the sum of the `Total_Medals` column:

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**.

2. Who had the most olympic gold medals in the Summer 2000 games? How many medals? In this case, we will get a subset of the DataFrame and sort by Total Medals or

```
?? PythonTeX ??
```

?? PythonTeX ??. which shows that Aleksey Nemov of Russia had the most with 6 medals.

3. Collectively taking each olympics, give the top 10 athletes by number of medals. We need to do a bit of work for this one. Since over all olympics for each athlete we need to add up all of the medals. This is known as grouping a dataset and the `Query` package does this. Before we answer the question about the athletes, consider a simpler example:

```
?? PythonTeX ??
```

lists 3 people and the cars they own. We want to summarize the data as the total number of cars each person has. If we

```
?? PythonTeX ??
```

and although a bit hard to read, it is now a list of the 3 people and collectively the cars they own:

```
?? PythonTeX ??
```

Let's now do the same with the Olympic Athlete dataset:

```
?? PythonTeX ??
```

shows the top list of the Sports. We can then find the number of athletes listed for each sport in the following way:

```
?? PythonTeX ??
```

Lastly, to actually answer the question at the top of this section, how we find the total number of medals for each athlete.

First, we group the data by Athlete's name (in the Athlete column) To see what happens with this, do

```
?? PythonTeX ??
```

4. Who has the most Olympic Silver medals in the data set (Collectively over multiple olympics) This is very similar to that above except that the Silver Medals are examined instead. I will leave it up to you to figure this out.

5. Produce a new DataFrame that lists total number of medals by country. Produce a histogram of the total number of medals. This is similar to the total numbers by athletes:

```
?? PythonTeX ??
```

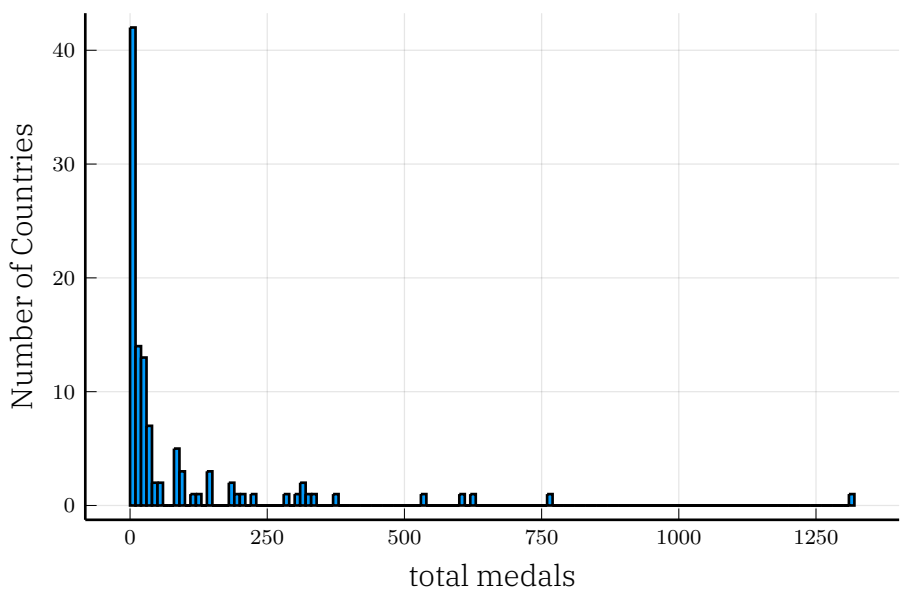
which does the following

1. groups the dataset by country.
2. summarizes the data into a new dataframe which is country and total medals
3. sorts them in decreasing order.

and the following (which uses `StatsPlots`):

```
?? PythonTeX ??
```

which results in

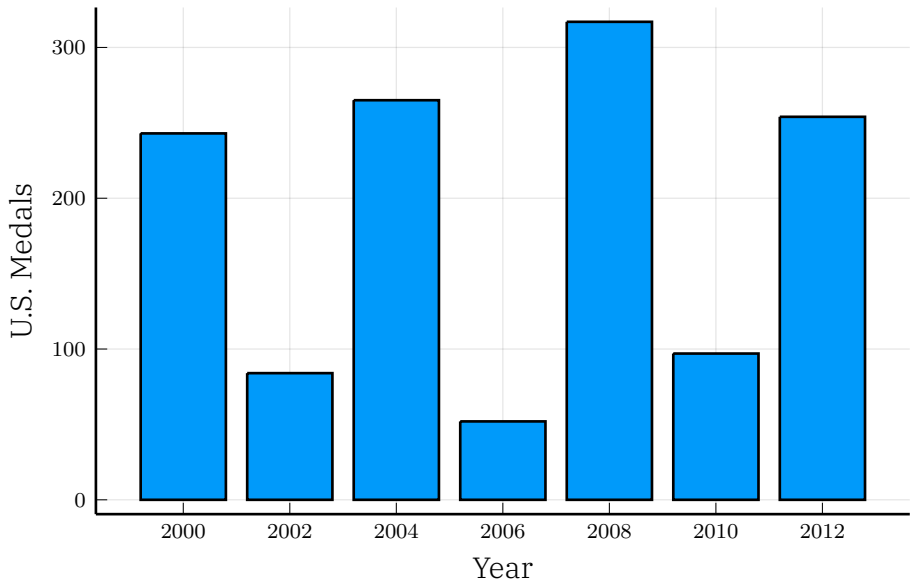


6. Plot the number of medals that the U.S. collected over each olympic year. In this case, we need to just get a subset of the U.S. medals:

```
?? PythonTeX ??
```

and the following gives a decent plot:

```
?? PythonTeX ??
```



7. What is the age with the most number of total medals? In this case, we group by age and sum total medals:

```
?? PythonTeX ??
```

show that age 24 has the most medals.

8. Produce a new data set with total number of medals per sport per year. What are the top five sports in medals for 2002,2012? First, we'll consider the 2002 year:

```
?? PythonTeX ??
```

results in Ice Hockey, Cross Country Skiing, Short-Track Speed Skating, Biathlon and Alpine Skiing.

And in 2012:

```
?? PythonTeX ??
```

This shows the top 5 are Swimming, Athletics (Track and Field), Rowing, Football (Soccer) and Hockey (probably field hockey).

18.6 Missing Data

Before finishing the questions, we need to deal with missing data in the dataset. It's a very common situation in a real dataset and does in this situation. Julia has a standard data type called `Missing` and it has a single value of that data called `missing`. The julia Documentation on Missing Data is a good place to read up on details of this.

There is some interesting properties of this. For example

```
?? PythonTeX ??
```

returns `missing` and pretty much any operation on a missing value returns missing because the idea is that if you don't know the value, how can calculate it.

18.6.1 Missing Data in a Dataset

Take the following dataset:

```
?? PythonTeX ??
```

where a number of missing values have been put in. We'll notice something different about the data types. If we look at the salary column:

```
?? PythonTeX ??
```

we see that the type is **?? PythonTeX ??**which means that the type of elements in the array are `Union{Missing,Int64}` which is julia's way of saying that the type can be either `Missing` or `Int64`.

Now, let's get down to some analyzing of the data. If we want to find the largest age by

```
?? PythonTeX ??
```

which returns 45, but if we want to find the mean salary:

```
?? PythonTeX ??
```

we get `missing`, which makes sense from the above discussion that nearly every operation with missing returns missing. Instead, we may want to ignore the missing values, so we can use the `skipmissing` function.

```
?? PythonTeX ??
```

we get **?? PythonTeX ??**, which seems quite complicated in that we get another array, however, if we

```
?? PythonTeX ??
```

then it finds the mean of the non-missing values and returns **?? PythonTeX ??**.

18.6.2 Missing values in the Olympic data

If we just find the average age of all in the Olympic database, then

```
?? PythonTeX ??
```

we get `missing` however, if we

```
?? PythonTeX ??
```

then the result is **?? PythonTeX ??**.

Part V: Linear Algebra

Chapter 19

Introduction to Linear Algebra

Linear Algebra is probably the most important mathematical field for Scientific Computation[1]. This chapter covers the basics of Linear Algebra in Julia and how it can be used to solve scientific problems.

19.1 Vectors and Matrices

Vector and matrices are the building blocks of linear algebra. In short a Vector is simply a 1D array and in julia the type `Vector<T>` is shorthand for `Array{T,1}`. We can make a vector just like we did with an array. For example, **??** makes an array of length three and the output: **?? PythonTeX ??**shows that it is a 1D integer array of length 3.

Similarly, there is a `Matrix` type which is an alias for a 2D array. If

```
?? PythonTeX ??
```

then we get the results **?? PythonTeX ??**

19.1.1 Addition and Subtraction of Vectors and Matrices

All of the construction methods for arrays of both 1D and 2D as seen in Chapter 6 can be used. If we have

```
?? PythonTeX ??
```

then **??** returns **?? PythonTeX ??**and **??** returns **?? PythonTeX ??**

Similarly if **??** and **??** then **??** returns **?? PythonTeX ??**and **??** returns **?? PythonTeX ??**

19.1.2 scalar multiplication

Scalar multiplication of vectors and matrix work as expected using the * operator, which is implied if multiplying on the left by a number. Using `A` and `x` above, **??** returns **?? PythonTeX ??**and **??** returns **?? PythonTeX ??**

[1] And if you ask someone in the field, they will say most important field PERIOD!

19.1.3 Matrix-vector products

Let **??**, and `A` and `B` as defined above. The vector-matrix product **??** returns **?? PythonTeX ??**and the matrix-matrix products **??** is .

19.1.4 Matrix Transpose

Recall that the tranpose of a matrix A is a matrix where the ith row and j column of A becomes the ith column and jth row of the transpose.

19.1.5 Identity Matrix

19.2 Matrix Inverse

19.3 Eigenvalues and Eigenvectors

Chapter 20

Advanced Topics in Linear Algebra

20.1 Sparse Matrices

We saw in the previous chapter, some of the matrices contained many zeros. As matrices get larger, the structure of them (like banded matrices), results in a matrix that is almost all zeros.

Consider the example,

Part VI: Calculus and Differential Equations

Chapter 21

Numerical Integration

Numerical integration is a very important component of scientific computation. In tradition calculus classes, the integrals presented were ones that had antidervatives that could be found relatively easily. However, this is generally not true. For example, if trying to determine

$$\int_0^2 e^{-x^2}\,dx$$

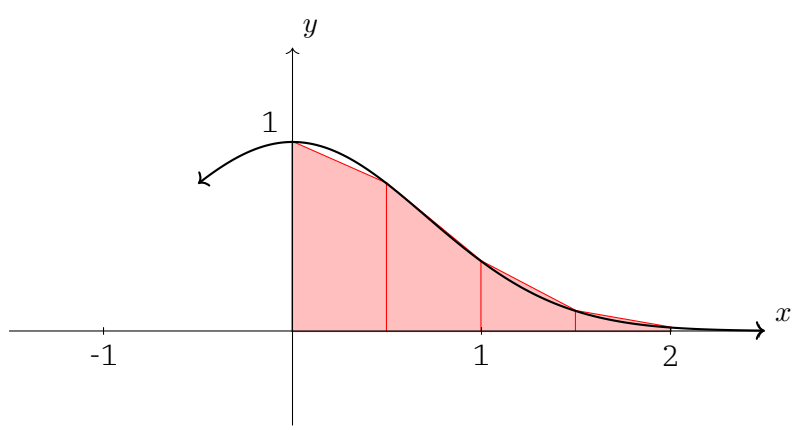
the function $f(x) = e^{-x^2}$ does not have an antidervative in terms of standard functions. This however it a very important antidervative in that it is related to the normal probability distribution. Because of its importance, we define

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}}\int_0^x e^{-t^2}\,dt$$

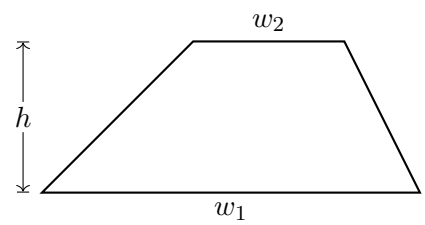
as the error function it is important to calculate values for it for any value of x.

21.1 Trapezoid Rule

Instead of approximating the area with rectangles, we can use trapezoids to do this. A plot of the approximate integral of e^{-x^2} on the interval $[0, 2]$ is given by



where 4 trapezoids have been used. The area of the a trapezoid like:



is

$$A = \frac{1}{2}h(w_1 + w_2)$$

In the trapezoids in the function plot above, the trapezoid are oriented sideways to this one, so the value of h is the horizontal distance or 0.5 for each one and the w_1 and w_2 are the function values, so the area of the four trapezoids above are:

$$\begin{aligned} A &= \frac{0.5}{2}\left(e^0 + e^{-0.5^2}\right) + \frac{0.5}{2}\left(e^{-0.5^2} + e^{-1}\right) + \frac{0.5}{2}\left(e^{-1} + e^{-1.5^2}\right) + \frac{0.5}{2}\left(e^{-1.5^2} + e^{-2^2}\right) \\ &\approx \textbf{??} \end{aligned}$$

The basic part of the trapezoid method is to use the area of a trapezoid on an interval with width h. The area of one of the trapezoids is

$$A_i = \frac{h}{2}(f(x_i) + f(x_{i+1}))$$

and then a full approximation will be the sum of these:

$$\begin{aligned} A &= \sum_{i=1}^{n-1} \frac{h}{2}(f(x_i) + f(x_{i+1})) \\ &= \frac{h}{2}\left(f(x_0) + f(x_1) + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + f(x_n)\right) && (21.1) \\ &= \frac{h}{2}\left(f(x_0) + \left(2\sum_{i=1}^{n-1} f(x_i)\right) + f(x_n)\right) && (21.2) \end{aligned}$$

Let's take a look at doing this in julia. To calculate the value we need a few things: 1) a function, 2) the interval we're doing the approximation on and 3) the number of trapezoids or subintervals. We presented the following in Chapter 15 as

```
?? PythonTeX ??
```

1. The parameter h is $(b - a)/n$.
2. Line (21.1) can be written as the map of two arrays. And then summed.

This is short and (maybe) sweet, but note that there are two separate arrays created. We have learned throughout this text, that the creation of arrays is often the most expensive part of a calculation. So instead, we will do the following:

```
?? PythonTeX ??
```

Although this looks to be nearly the same, we haven't created arrays. The `LinRange` function creates a range, but julia is savvy enough to not create the array to sum over it. See the exercise below.

Let's first redo the example above:

```
?? PythonTeX ??
```

which returns **?? PythonTeX ??**.

21.1.1 Exercise

- Find $\int_0^1 \sin(x^2)\,dx$ using the trapezoid method for $4, 10$ and 20 points.
- Use the `BenchmarkTools` and the `@btime` macro to compare the time of the trapezoid rule using the two functions shown here. Try integrating $\sin(x^2)$ on $[0, 1]$ for $N = 10,000$ subintervals.

21.2 Simpson's Rule

The trapezoid rule uses straight lines to approximate the function and then integrates the line (or finds the area of the trapezoid). To find a better approximation if we use a quadratic function (parabola) to estimate the function. If we do this, then integrate the resulting parabola, this is called Simpson's Rule. There are numerous places to find the derivation of this, but the result is

$$\begin{aligned} I &= \frac{h}{3}\big(f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + \cdots + 2f(x_{n-1}) + f(x_n)\big) \\ &= \frac{h}{3}\left(f(x_0) + f(x_{2n}) + 4\sum_{k=1}^{n} f(x_{2k-1}) + 2\sum_{k=1}^{n+1} x_{2k}\right) \end{aligned}$$

where $h = (b-a)/n$, $x_k = a + kh$ and it is important that n is an even number.

Similar to the trapezoid rule, we can write Simpson's rule in julia as

```
?? PythonTeX ??
```

Applying Simpson's Rule to $\int_0^3 x^2\,dx$ as **??** results in **?? PythonTeX ??**. It's interesting to note that this reports the exact value. In fact, Simpson's rule will numerically integrate any cubic (and lines and quadratics) exactly.

If we return to trying to find $\int_0^2 e^{-x^2}\,dx$ and use Simpson's Rule, then

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**, which is within 5 digits of the true answer.

21.2.1 Plotting the Error Function

We can use numerical integration to evaluate the error function at any value, so let's go ahead and plot it.

First, let's make a function that defines the error function.

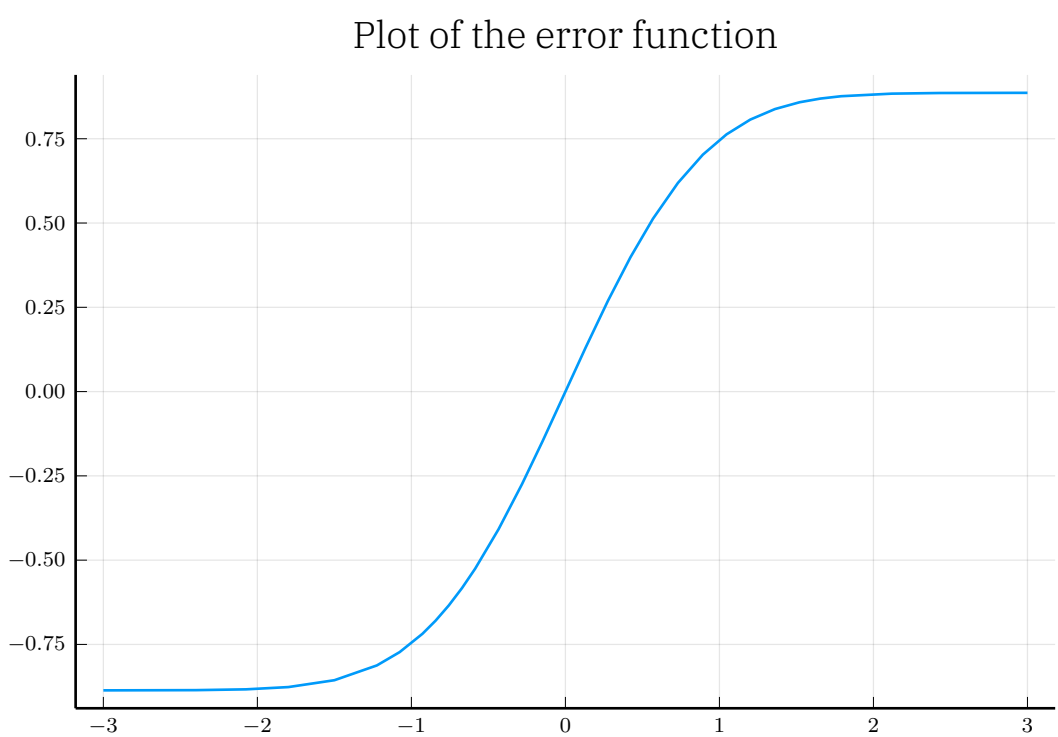
```
?? PythonTeX ??
```

where we have used the symmetry of the error function to evaluate it if `x` is positive or negative. If we don't check for this, we will get an error because `simpsonsRule` has a check that the interval needs to be put in as $a < b$.

Now, we can plot it with:

```
?? PythonTeX ??
```

which results in



21.3 Using the NumericalIntegration package

This package does some basic integration using both Trapezoid and Simpson's rule discussed above.

```
?? PythonTeX ??
```

and the result is **?? PythonTeX ??**

not impressed with this package. Maybe I shouldn't include this. If we decide to, need to expand and show some of the details.

Chapter 22

Differential Equations

Another extremely important topic in Scientific Computation is that of differential equations. These arise obviously in mathematics, but also in most other scientific fields including biology, chemistry, engineering, and economics. Also, it is rare that a differential equation has a nice closed form solution, so often numerical methods must be used to get a solution.

We will discuss two often-used methods for solving differential equations, Euler's method and Runge-Kutta methods and then delve into a very nice robust julia package.

A **differential equation** is simply an equation that has derivatively in it. For example

$$y'' + 4y = 0$$

It is important to note that the **order** of a differential equation is the degree of the highest derivative, so in the example above, it is two. The following procedure can covert non order-1 differential equation to one that is a first-order system.

Let $y_1 = y$, then $y_2 = y' = y_1'$. Lastly $y'' = y_1'' = y_2'$ and using the differential equation $y'' = -4y$, this can be written:

$$\begin{aligned} y_1' &= y_2 \\ y_2' &= -4y_1 \end{aligned}$$

or in matrix form:

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix}' = \begin{bmatrix} y_2 \\ -4y_1 \end{bmatrix}$$

and as long as $y^{(n)}$, the highest-order term, can be isolated algebraically, one can write the differential equation as

$$\mathbf{y}' = F(\mathbf{y})$$

22.1 Euler's Method

We start with one of the simplest methods to solve a differential equation. If we have are solving

$$y' = f(y,t), \qquad y(0) = y_0 \tag{22.1}$$

with the understanding that we will solve this at a discrete set of points, so we let $y_n = y(t_n)$ and $t_k = t_0 + k\Delta t$, where Δt is a small time step. We can then replace y' with the approximation:

$$y' \approx \frac{y_{n+1} - y_n}{t_{n_1} - t_n}$$

and the differential equation in (22.1) is

$$\frac{y_{n+1} - y_n}{t_{n_1} - t_n} = \frac{y_{n+1} - y_n}{\Delta t} = f(y_n, t_n)$$

and solving for y_{n+1}, we get:

$$y_{n+1} = y_n + \Delta t f(y_n, t_n)$$

and this iterative (or difference equation) is known as **Euler's Method**.

22.1.1 Implementing Euler's method in Julia

Let's take a look at Euler's method in Julia. Of course, we'll write a function to evaluate this and there are a few things that we need for the function:

- The function f, which is the right-hand side. Note that it takes in two variables.
- An initial point, (t_0, y_0). There will be two real arguments for this.
- A step size Δt.
- Either a final t value or a number of steps to run Euler's method. We'll use the number of steps, a positive integer.

We'll return two arrays for the `t` and `y` values.
We'll also implement this as a for loop. Although there are some

```
?? PythonTeX ??
```

22.1.2 Example

We'll solve $y' = y$, $y(0) = 1$ for `dt=0.1` and 10 steps.

```
?? PythonTeX ??
```

and this returns **?? PythonTeX ??**

This results the t and y values based on Euler's method. However, perhaps it's more interesting to see a plot. Also, note that the analytic solution to this is $y = e^x$, so the plot of the approximate solution using Euler's method as well as the analytic solution is

```
?? PythonTeX ??
```

plotplots/diffeq/euler1.texdiffeq

22.2 Runge Kutta methods

There is a family of other methods called the **Runge Kutta methods** that are relatively simple with a high order of accuracy. One version that was often a standard version is what is called the 4th-order Runge-Kutta or RK4 method is

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

where

$$\begin{aligned}
k_1 &= f(t_n, y_n) \\
k_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\
k_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\
k_4 &= f(t_n + h, y_n + hk_3\Big)
\end{aligned}$$

22.2.1 RK4 in julia

The following is the `rk4` method in julia. It is very similar to euler's method...

```
?? PythonTeX ??
```

and we can use this to solve $y' = y$, $y(0) = 1$ by

```
?? PythonTeX ??
```

and the results are **?? PythonTeX ??**

22.3 Using the `DifferentialEquations` package

There is a very mature and robust package for solving differential equations aptly called `DifferentialEquations`. We will show a few examples to get a feeling for how to use the package, but as you can tell from the documentation, that there are a lot of

Chapter 23

Complex Numbers and Analysis

23.1 Complex Numbers

23.2 Fourier Transforms

Part VII: Advanced Topics

Appendix A

Details of Using Julia

A.1 using the Repl.it wesite

A.2 Downloading and Installing JuliaPro

We will talk about getting julia installed on your local machine using JuliaPro. First, go to the JuliaPro website and click the ***free download*** button or scroll down. Select the platform under ***Current Stable Release*** and click the black button.

Note: for Mac users, you'll need to do an extra step to get JuliaPro installed. Download the ***QuickStart*** guide on the same page and read through the instructions.

After downloading, open the JuliaPro and follow the instructions. There is little intervention, except you will get a dialog asking about the package server. Leave it blank. Hopefully all will install smoothly.

Open up the app ***JuliaPro*** as you would normally do for other apps and you should see the ***Atom*** browser launch and look like:

images/getting-started/juno.png

A.3 Using Juno and the Atom Editor

Make sure you have installed JuliaPro and opened the app which will launch the Atom editor. This section will give an overview of using this.

images/getting-started/juno-details.png

There are 4 regions that are important to get familiar with as well as the Juno Menu bar. Let's get to know these.

A.3.1 The Julia REPL

The ***Julia REPL*** is where you enter single julia statements. A REPL is short for Read-Eval-Print-Loop and is often a interpretative shell where you can enter commands in some language. To get started, click on the REPL part of Atom and hit ENTER. This starts up julia and you should see:

images/getting-started/REPL-startup.png

and notice that it's ready for you to enter something with the `julia>` prompt.

We will enter a few commands to get started. All julia commands throughout the book will be written in a `monospace font` and it is a good idea to type along at the same time.

Enter `a=2` at one prompt and `b=3` at another prompt. You will see the response of 2 and 3 as well. (Additionally, you may notice that the ***Workspace*** tab in the upper right has changed. We will discuss that later.). Now enter `a+b` in a prompt and you should see the result 5.

Later in this chapter we do a simple plot, but need to load a package first. Type `]` at a prompt and the julia prompt will turn into `(@1.4) pkg>` which is the package manager mode. First, type `up` to update all packages. You will need to authenticate first. Your default browser will automatically open and you will need to login. After you authenticate, the browser will say that it was successful and you can close that tab. Return to Atom and you should see that julia is updating some packages. This may take a minute or so.

Next, we will install the ***Plots*** package. At the `(@1.4) pkg>` prompt, type `add Plots`. Julia will need to add a lot of other packages, but you won't need to worry about this. We will use this in the next section. We will also learn much more about packages in Chapter B.

To exit the package manager mode, enter CONTROL-C. You will leave this mode and return to julia prompt.

A.3.2 The Plots Pane

The Plots pane is one of the tabs in the upper right corner. In the julia REPL, type `using Plots` which will load in the ***Plots*** package. Often, the package will need to pre-compile before you can use it. This may take a minute or so at most. Then type `plot(rand(10))`

at the julia prompt. You will see something similar to

images/getting-started/random-plot.png

A.3.3 The Workspace Pane

Also in the upper right corner, the ***Workspace*** pane. If you have been typing in all of the commands in this appendix, your workspace pane should look like:

images/getting-started/workspace.png

This shows all of the current objects/values stored in julia. The first three are some base packages that are always loaded. The bottom one `register_auth_handler` is how Juno authenticates with the package manager. The other three are those entered by

you. You stored values into the variables `a` and `b` and the last result (which is a plot) is always stored in the variable `ans`.

A.3.4 The Documentation Pane

The documentation pane (also in the top right part of Atom) allows you to search the documentation, although personally, I prefer using the website docs.julialang.org. Trying typing `rand` into the search box at the top to search for information about the rand function. You should see:

`images/getting-started/doc-example.png`

A.3.5 The Editor Pane

The upper-left corner of Atom is the Editor Pane. This is a nice interactive way of running julia commands than the REPL. In the editor, enter the following three lines:

```
a=2
b=3
a+b
```

You can evaluate each line by typing CTRL-ENTER (or CMD-ENTER on MacOS) to evaluate each line. Along the right side, you'll see the output. In the first two lines, just the values of the variables and for the last line, the number 5.

A.4 The kernel

In many ways, julia is a program like any other that you run on your computer, however because of the nature of it parsing statements and then giving output, you have to think

about it a bit differently. When you start julia, we often will say that you are starting the kernel, which is a basic state which you can feed variables and statements to julia and get output. There are three different ways that this can occur:

A.4.1 The REPL

If you open julia directly from your computer (either clicking on an icon or using some sort of application launcher), you'll set the REPL as explained above. This is a very simple, but often helpful, way to enter julia code. When this starts up you enter commands like:

```
?? PythonTeX ??
```

This all goes to the julia kernel.

A.4.2 Juno and the Atom Editor

As we saw above, the Atom Editor and the embedded Juno packages allows you to have much more control over julia code.

A.5 Restarting the Kernel

Appendix B

Loading Packages

As with many software packages, additional functionality is available in other libraries or packages. If you take a look at https://juliaobserver.com there are about 2000 packages (as of Fall 2018). Clicking on any of the package names will give additional information about the package including documentation (hopefully).

If you don't have a package installed and try to use it, like:

```
?? PythonTeX ??
```

you will get an error that it isn't installed–unless you actually installed it previously. You can add the package by

```
?? PythonTeX ??
```

and after fetching the package, rerunning

```
?? PythonTeX ??
```

should no longer give an error. Then, for example, try

```
?? PythonTeX ??
```

which returns `true` because 17 is a prime number. This is a nondeterministic function that determines if a number is true. The documentation for the Primes package is at `https://github.com/JuliaMath/Primes.jl` and demonstrates 4 commands including `isprime` and `factor` which returns all prime factors (and the order).

A few other packages that we have seen or will see:

- IJulia – provides the browser interface to Julia. You are probably already using this.
- ForwardDiff – does Automatic Differentiation (see Chapter 8)
- Plots – A plotting packages (See Chapter 11)

- GR – A backend to the Plots package.
- PlotlyJS – Another nice backend to the Plots package.
- DataFrames – a way to nicely handle datasets generally loaded from external files.

Chapter 14 will show how to create our own packages also called a module.

B.1 Managing Packages in Julia

Although we can use the `Pkg` package to handle packages, this section will use the terminal REPL to handle any non-standard packages. Additional documentation on this is given in https://docs.julialang.org/en/latest/stdlib/Pkg/ First, open up a terminal version of julia (generally by opening the application that you downloaded). You will get:

```
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.0.0 (2018-08-08)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org/ release
|__/                   |
```

or similar and then

```
julia>
```

which means we're ready to handle julia commands. If we type `]`, then the prompt turns into:

```
(v1.4) pkg>
```

where the 1.4 will be the version of julia that you are running. There are a number of commands that we will cover here:

- add
- remove (rm)
- develop (dev)
- status
- update (up)
- test

- build
- precompile

and the commands in parentheses are the shortcut.

B.1.1 Adding a package

In the package command line, type `add` ***package_name*** to add the package. For example, to add the `ForwardDff` package:

```
add ForwardDiff
```

and it is not installed, you will get something like:

```
Updating registry at `~/.julia/registries/General`
  Updating git-repo `https://github.com/JuliaRegistries/General.git`
  Updating `~/.julia/environments/v1.0/Project.toml`
  [f6369f11] + ForwardDiff v0.9.0
  Updating `~/.julia/environments/v1.0/Manifest.toml`
  [9e28174c] + BinDeps v0.8.10
  [bbf7d656] + CommonSubexpressions v0.2.0
  [163ba53b] + DiffResults v0.0.3
  [b552c78f] + DiffRules v0.0.7
  [f6369f11] + ForwardDiff v0.9.0
  [77ba4419] + NaNMath v0.3.2
  [276daf66] + SpecialFunctions v0.7.0
  [90137ffa] + StaticArrays v0.8.3
```

A few things to note:

- You results will vary depending on version numbers avaiable and what subpackages (like `DiffRules` or `StaticArrays` are needed for the current version of the package you want to load.
- The line after the 2nd `Updating` line is the package (and version) that you are installing.
- All of the lines after the 3rd `Updating` line is all of the packages that this depends on.
- The + sign means that the package is being added.

If you want to add multiple packages at the same time, say packages A, B and C, type `add A B C`. You can also add particular versions of a package (often for testing or to avoid a bug). For example, if you want version 0.3.0 of `ForwardDiff` type:

```
add ForwardDiff@0.3.0
```

You will then get info on the dependencies on that version.

B.1.2 Package Status

The `status` command (or `st`) will just list all of the main packages installed. For example,

```
Status `~/.julia/environments/v1.0/Project.toml`
[c52e3926] Atom v0.7.6
[336ed68f] CSV v0.3.1
[a93c6f00] DataFrames v0.13.1
[f6369f11] ForwardDiff v0.9.0
[7073ff75] IJulia v1.12.0
[e5e0dc1b] Juno v0.5.3
[1a8c2f83] Query v0.10.0
```

and note that these are just the packages added by the `add` command, not all of the dependencies. If you want all of the dependencies as well, type `st --manifest` and I get a huge list of packages.

B.1.3 Removing a Package

You can remove a package by typing `remove` or `rm` then the package name. If I want to remove the `ForwardDiff` package, then

```
remove ForwardDiff
```

we get the following:

```
Updating `~/.julia/environments/v1.0/Project.toml`
 [f6369f11] - ForwardDiff v0.3.0
 Updating `~/.julia/environments/v1.0/Manifest.toml`
 [49dc2e85] - Calculus v0.4.1
 [c5cfe0b6] - DiffBase v0.2.0
 [f6369f11] - ForwardDiff v0.3.0
 [77ba4419] - NaNMath v0.3.2
```

Note:

- The `rm` command removes the package from the list of available packages, but doesn't remove them from your harddrive.
- If you want to see everything installed, navigate to the `~/.julia/packages` directory, which is where they are stored.

B.1.4 Updating packages

If you type `update` or `up` you will update all of the installed packages (and dependencies). For example:

```
Updating `~/.julia/environments/v1.0/Project.toml`
 [7073ff75] ↑ IJulia v1.11.1 ⇒ v1.12.0
 Updating `~/.julia/environments/v1.0/Manifest.toml`
 [7073ff75] ↑ IJulia v1.11.1 ⇒ v1.12.0
 [b85f4697] ↑ SoftGlobalScope v1.0.5 ⇒ v1.0.7
 [5e66a065] ↑ TableShowUtils v0.1.1 ⇒ v0.2.0
```

and all updates will be with an ↑. If you only want to update a single package, type the name after `update`.

B.1.5 Building Packages

Generally a package is built after it is installed. Building a package might include running code (or unpacking files) after it is installed. Sometimes if things get wonky, rebuilding is a good thing to do.

```
build
```

or if you only want say `IJulia` built,

```
build IJulia
```

B.1.6 Precompiling packages

When a package is used, often it requests to be compiled. For example, when `using Primes`, then following is shown:

```
[ Info: Precompiling Primes [27ebfcd6-29c5-5fa9-bf4b-fb8fc14df3ae]
```

and basically some code is compiled beforehand, generally to speed up code. You can precompile all code with

```
precompile
```

and it may take a while, but you won't have to wait, when you load the package with the `using` command.

B.1.7 Testing a Packages

To test a packge, say the `ForwardDiff` package, then

```
test ForwardDiff
```

It list all of the dependencies first, and then runs a number of tests (and we will show how to write tests soon) and timing information. After a while, it finishes sucessfully.

Appendix C

Binary Representation of Numbers

In Chapter 3, numeric data types were introduced. Some details of integers and floating points were given, however, some of the gorier details were left to here.

C.1 Integer Representation

We discussed the binary representation of non-negative numbers in Chapter 3. Negative numbers is more difficult. This method of data storage in called ***2's complement***. A nice description of this is Ryan's Tutorial on negative binary numbers.

Although seems a bit strange, let's first discuss the conversion of binary to integer. Let the 8-bit binary number be written as $x = sa_6a_5a_4a_3a_2a_1a_0$, where the first bit is the sign and the other 7-bit compose the number then representation of x in decimal is

$$x = -s \cdot 2^7 + a_6 2^6 + a_5 2^5 + a_4 2^4 + a_3 2^3 + a_2 2^2 + a_1 2^1 + a_0 2^0 \tag{C.1}$$

As an example, note that if $s = 0$, then this is identical to the integer representation of positive integer. Again, `00010010` would be

$$x = -0 \cdot 128 + 0 \cdot 64 + 0 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 18$$

Let's look at an example with a negative number. Let `x=11010111`, then in decimal:

$$\begin{aligned} x &= -1 \cdot 128 + 1 \cdot 64 + 0 \cdot 32 + 1 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 \\ &= -128 + 64 + 16 + 4 + 2 + 1 = -41 \end{aligned}$$

C.1.1 Converting from decimal to binary

Again, like above, let's concentrate only on 8-bit numbers. That is, we will take a decimal number `x` in the range $[-128, 127]$ and return an array of length 8 of bits (you can even think of booleans) or we need to find $s, a_0, a_1, \ldots, a_6$ from (C.1).

First, let's examine only non-negative numbers, so $s = 0$. And let's use the number $x = 47$. We'll work from the larger digits to the smaller ones, so a_6 to a_0.

- Let **??**.
- Let **??**=**?? PythonTeX ??**. **??**=**?? PythonTeX ??**.
- Let **??**=**?? PythonTeX ??**. **??**=**?? PythonTeX ??**.
- Let **??**=**?? PythonTeX ??**. **??**=**?? PythonTeX ??**.
- Let **??**=**?? PythonTeX ??**. **??**=**?? PythonTeX ??**.
- Let **??**=**?? PythonTeX ??**. **??**=**?? PythonTeX ??**.
- Let **??**=**?? PythonTeX ??**. **??**=**?? PythonTeX ??**.
- **??**=**?? PythonTeX ??**.

This shows that the result is 00101111. Again, **??** returns **?? PythonTeX ??**, the same result.

A function that does this is as follows.

```
?? PythonTeX ??
```

Note that first this uses recursion as explained in Chapter 4. In addition, the error checking is explained in chapter 15. This function returns a boolean array (ones and zeros). For example:

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**. And note that since the array returns the digits in the order from small to large, this result is the same.

C.1.2 Representing Negative Integers

To find the representation of a negative integer, we return to (C.1) and note that $s = 1$. A little algebra results in

$$x + s \cdot 2^7 = a_6 2^6 + a_5 2^5 + a_4 2^4 + a_3 2^3 + a_2 2^2 + a_1 2^1 + a_0 2^0$$

and with $s = 1$, we define the left side as $y = x + 127$ and now the right side is the same representation of a positive number and if $-128 \leq x \leq -1$, then $0 \leq y \leq 127$.

Let's check out an example. Consider $x = -67$, then $y = 128 + x = 61$. Using the algorithm to find the binary representation of 61 or **??** which returns **?? PythonTeX ??**recalling that the order is reversed from the function above.

Since we are looking for the decimal representation of -67, not 61, the sign bit is 1, so we have `10111101` and checking with the built-in function **??** **?? PythonTeX ??**.

This method of storage of negative integers is called *2's complement*.

C.1.3 Exercise

Write out by hand the binary representation of a) -10, b) -120 and c) -1.

C.1.4 Including negative Integers

If we extend the function above to include negative numbers, the following:

```
?? PythonTeX ??
```

Calling **??** results in **?? PythonTeX ??**and this is the same as above. The output is a little tricky so we have another function that will take this boolean array to a string.

```
?? PythonTeX ??
```

where the `reduce` function in Chapter 4 is used and the function `reverse` takes an array and reverses the elements. This is must more convenient with

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**, the same as above and the same as `bitstring` for 8-bit integers.

C.1.5 Exercise

Use the functions `dec2bin` and `arr2str` to find the binary represenation of a) -10, b) -120 and c) -1 as you did above.

C.1.6 A binary to decimal function

It's also nice to have a function that reverses this.

```
?? PythonTeX ??
```

To use this, we need to start with a binary array, so let's start with

```
?? PythonTeX ??
```

and then we can use the `bin2dec` function as

```
?? PythonTeX ??
```

returns **?? PythonTeX ??**.

And the following is a function that will take a binary string and turn it to an array

```
?? PythonTeX ??
```

If we use this in conjunction with `bin2dec`, we have some convenient functions. That is

```
?? PythonTeX ??
```

results in **?? PythonTeX ??**.

C.2 Operations on Binary Integers

Adding two positive binary integers is much like adding decimal integers. An example of $54 + 19$ could be:

$$\begin{array}{r} 00110110 \\ \underline{+\,00010011} \\ 01001001 \end{array}$$

where the standard algorithm is used

The result of this in decimal is **??** or **?? PythonTeX ??**.

Note that if we try to add `01001100` and `01100010` we get `10101110` (try it!) and if we use the conversion to get back to decimal results in **??** or **?? PythonTeX ??**, which clearly is not the sum of two positive numbers. The reason this occurred is that the sum of the two numbers does not fit in an 8-bit integer, whose max value is 127. We can tell this directly from the binary result in that the first digit is a 1 (due to the sum of the two one's in the second position). Recall that this first digit is a sign bit. This is an overflow error.

C.2.1 Unary minus

The unary minus operator negates a number. To see how to do this in binary, let's look at what a bit flip looks like with a few examples. Let's take the decimal number 54 or `00110110` and flip all of the bits (changes 0s to 1s and 1s to 0s) to get `11001001` and using **??** which returns **?? PythonTeX ??**. If we use `10101010` which is -86, the bit flips are `01010101` which is 85. Note that the sum of a binary number and it's bit flips is `11111111` which is -1.

To determine the negation of a binary number, we will 1) flip the bits and 2) add one. So for example, `00101101` or **?? PythonTeX ??**. The bit flip of this is `11010010` and then add one to get `11010011`, which is **?? PythonTeX ??**.

C.2.2 Exercise

- Show that the unary minus of 0 in binary is still 0. (Note: this is one of the very nice features of using 2's complement for storing integers.)

- There is one 8-bit integer than does not have negative (that is an 8-bit integer). What is it?

C.3 Subtraction of binary integers

There are a couple of ways to think about subtraction. One can use the column algorithm like is standard for subtraction of decimal numbers and adapt to binary integers. However an easier way is to thing of $a - b$ as $a + (-b)$ and use the negative from the previous section. For example, look at $78 - 85$.

1. The two integers in binary are **?? PythonTeX ??** and **?? PythonTeX ??**.
2. The negation of 85 is `10101010` + `00000001` or `10101011`
3. The sum of `01001110` and `10101011` is **?? PythonTeX ??**
4. converting back to decimal results in -7.

C.4 Multiplication of Binary Integers

$$\begin{array}{r} 00001010 \\ \times\ 00000110 \\ \hline 00000000 \\ 00010100 \\ +\ 00101000 \\ \hline 00111100 \end{array}$$

which shows that **?? PythonTeX ??**$\times$ **?? PythonTeX ??**equals **?? PythonTeX ??**
Discuss multiplication of negative integers...

C.5 Extended Exercise

Put all of these functions in a module as in Chapter 16. This will include

- Creating a type called `Integer8` which is an alias for a binary array of length 8.
- Create a `Base.show` method for `Integer8` to display as a string. See Chapter 16 to do this and use the function in this chapter
- Create a function `parseBin`, which takes in a string of length 8, and returns a `Integer8` type. Use the `str2arr` function from this chapter.

- Create a unary minus, plus and minus on the type `Integer8` types. Use the example of Polynomials in section 11.3 for examples on how to do this.
- Create a test suite to thoroughly test all of the functions that you write.

C.6 Scientific Notation

Recall that any number written in decimal form with only a finite number of digits can be written in scientific notation that is in the form:

$$a \times 10^b$$

where $1 < |a| < 10$ and b is an integer. For example 4003.23 can be written as 4.00323×10^3, so $a = 4.00323$ and $b = 3$.

In this form the number a is often called the **significand** or **mantissa** and the number b is **exponent**. This example has the base 10, however other bases are common (generally base 2).

One major advantage to using numbers in this form is the simple multiplication and division. Consider multiplying $x = 3.4 \times 10^2$ and $y = -4.7 \times 10^{-3}$. Using properties of exponentials we get

$$xy = (3.4)(-4.7) \times 10^{2-3} = -15.98 \times 10^{-1}$$

and typically we would like to put this back into scientific notation by shifting the exponent so $xy = -1.598 \times 10^0$.

Division can be done in a similar manner and perhaps surprisingly, addition and subtraction are more difficult due to the fact that the exponents of the two numbers need to be equal before adding and subtracting.

C.7 Floating Point Numbers of a given size

The reason for using floating point numbers in calculations is twofold. First, there is a finite size of storage for a number and secondly, routines for performing operations on floating-point numbers are fast and usually encoded on a computer chip.

Consider a floating point of a given size, say 64 bits generally called a ***double precision floating point number***. The first bit is generally used for the sign, the next 11 are the exponent and the final 52 bits store the mantissa. A floating point number has two limitations and that is the precision (how many digits that can be stored) and the magnitude (the largest number). Double precision numbers are store in binary and converted to decimal with the form:

$$(-1)^s 2^{c-1023}(1+f)$$

where s is the sign c is the exponent and f stores the mantissa. For example, consider the following number:

$$0\ 10000000101\ 0111011010000000000000000000000000000000000000000000$$

where spaces separate out s, c and f. Converting c to decimal:

$$c = 1 \cdot 2^{10} + 0 \cdot 2^9 + \cdots + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 1029$$

The mantissa is calculated in the following way

$$\begin{aligned} f = 0 \cdot \left(\frac{1}{2}\right)^1 + 1 \cdot \left(\frac{1}{2}\right)^2 + 1 \cdot \left(\frac{1}{2}\right)^3 + 1 \cdot \left(\frac{1}{2}\right)^4 + 0 \cdot \left(\frac{1}{2}\right)^5 + 1 \cdot \left(\frac{1}{2}\right)^6 \\ + 1 \cdot \left(\frac{1}{2}\right)^7 + 0 \cdot \left(\frac{1}{2}\right)^8 + 1 \cdot \left(\frac{1}{2}\right)^9 = \frac{237}{512} \end{aligned}$$

and thus the floating point number is:

$$(-1)^0 2^{1029-1023} \left(1 + \frac{237}{512}\right) = 93.625$$

The double precision number system falls into a class of number systems that we can commonly call floating-point number systems.